Table of Contents

The CMS Bible

The Complete Technical Reference for the ZWECK TUKULA Ltd Company Management System

Version documented: 1.27.0 **Last updated:** 12 August 2026 **Status:** Living document — update this file in the same session as any code change (see "How to Keep This Document Alive" near the end).

Credits

This system and this document were built collaboratively by **Ian** (owner, product direction, testing, and every real-world business decision about how ZWECK TUKULA Ltd actually operates) and **Claude** (Anthropic's AI model, acting as the sole developer — architecture, implementation, database design, and documentation), working together across many sessions inside Claude's Cowork mode.

Every feature in this document exists because Ian described a real need of the company and Claude implemented it end-to-end — there is no separate development team, no outside contractor, and no boilerplate template this was built from. This is a from-scratch, purpose-built system for one real SACCO/investment club, and this document is its single source of truth.

Table of Contents

---

1. Introduction

ZWECK TUKULA Ltd is a Uganda-based SACCO/investment club (Wakiso, Uganda) whose members pool capital, save, borrow, lend, invest, and govern themselves collectively. This system — internally called the "CMS" (Company Management System) — is the software the club runs its entire financial and administrative life through: every contribution, loan, grant, investment, dividend, savings deposit, transfer, meeting, and document that matters to the company passes through it and leaves a permanent, auditable trail.

This document exists to answer, in as much depth as is useful, three questions about any part of the system: - **What does this feature actually do**, in plain language? - **Exactly how does it work** — which database tables, which API endpoints, which business rules, which edge cases? - **What can go wrong or is already known to be imperfect** — so nobody rediscovers a bug that's already been found, or is surprised by a deliberate design tradeoff?

It is meant to be read by three different audiences: **Ian** (or any future company officer) who needs to understand what a feature does and how to operate it; **a future developer** (human or AI) who needs to safely extend or fix the system without re-deriving its architecture from scratch; and **Claude**, in a future session, who should treat this document as the fastest way back up to full context on a system this large.

This document must be kept current. See Section 10 for the exact rule: any session that changes a feature, adds one, fixes a bug, or changes the deployment must update the matching section of this file in the same session, not "later."

---

2. System Overview & Architecture

2.1 Technology stack

Layer	Technology
Backend	Node.js + Express 5, raw pg driver (no ORM)
Frontend	React 19, Tailwind CSS, React Router, Recharts (charts), Heroicons
Database	PostgreSQL (managed, hosted on Render)
Authentication	JWT (access + refresh tokens), bcrypt password hashing, optional TOTP 2FA
Email	Nodemailer via Gmail SMTP (App Password)
Scheduled jobs	node-cron, running in-process inside the backend web service
PDF/document generation	Client-side HTML-to-print for most documents; server-side Puppeteer (headless Chrome) for emailed share certificates
Hosting	Render (Blueprint-based: one Postgres database + one Node web service + one static React site, per company)

2.2 Repository layout

company management system/ (repo root)
├── render.yaml Company A's Render Blueprint
├── render.company-b.yaml Company B's Render Blueprint (second, independent deployment)
├── DEPLOYMENT_GUIDE.md Step-by-step first deployment walkthrough
├── GOING_LIVE_GUIDE.md Clearing test data / going live with real data
├── SYSTEM_DIAGNOSTIC_REPORT.md Point-in-time bug audit (19 July

```
2026) — see Section 8
├── docs/
│   └── CMS_BIBLE.md                    This document
├── cms/                                Backend (Node/Express)
│   ├── server.js                       Entry point — middleware, routes, startup
sequence
│   ├── schema.sql                      Canonical, always-current full database schema
│   ├── migration_v1.X.0.sql            One file per schema change, idempotent, run
once per database
│   └── src/
│       ├── config/                     database.js, email.js, logger.js
│       ├── controllers/                One file per module — all business logic lives
here
│       ├── routes/                     One file per module — route definitions +
permission gates
│       ├── middleware/                 auth.js, upload.js, validate.js
│       ├── services/                   Cross-cutting logic: auditService,
notificationService,
│       │                               emailTemplates, referenceService, loanService,
reportService,
│       │                               certificateService
│       ├── jobs/scheduler.js           All node-cron scheduled jobs
│       └── utils/                      errors.js, response.js
└── cms-frontend/                       Frontend (React)
    └── src/
        ├── api/endpoints.js            Every backend call, grouped by module (e.g.
`accountsAPI`)
        ├── pages/                      One folder per module, matching the backend
module split
        ├── components/                 layout/ (Sidebar, TopBar, AppLayout,
ProtectedRoute, GlobalSearch),
        │                               common/ (DataTable, PageHeader, Avatar, etc.)
        ├── contexts/                   AuthContext.js, BrandingContext.js
        └── utils/                      exportUtils.js (every printable/generated document
template),
                                        helpers.js (formatDate, formatNumber, etc.)
```

2.3 Core architectural patterns (apply everywhere, not just in one module)

- **Every module follows the same shape**: a controller file (business logic, wrapped in asyncHandler), a routes file (URL → controller mapping + permission gate), and a matching frontend/src/pages/<module>/ folder. If you understand one module's file layout, you understand all of them.
- **withTransaction(callback)** (cms/src/config/database.js) is the atomic-operation wrapper used for anything that touches money or must succeed/fail as one unit — it BEGINs, runs the callback with a locked transaction client, COMMITs on success or ROLLBACKs on any thrown error.
- **postTransaction()** (transactionsController.js) is the single choke point every real debit/credit in the entire system funnels through — it locks the account row, enforces

"never go negative" and "never breach the floor limit," and writes the ledger row. No module posts money to an account any other way.

- **Reference codes** (MODULE-CATEGORY-YYYYMM-SEQUENCE, e.g. PA-CONTRIB-202601-00001) are generated by one shared service (referenceService.js) for every record type in the system — transactions, transfers, loans, grants, documents, events, certificates, audit submissions. See Section 5.3 for the full mechanism.
- **Audit logging** (logAction() in auditService.js) is the one function that writes to the permanent audit_log table, called explicitly at the end of nearly every state-changing controller action. See Section 4.24.
- **Notifications** (notify()/notifyMany() in notificationService.js) is the one function that creates an in-app bell notification and, optionally, sends a matching email — always called *after* the real business transaction has committed, and never allowed to roll back or block it.
- **Response envelope**: every API response is { success, message, data, meta? } via sendSuccess/sendCreated/sendPaginated (utils/response.js).
- **RBAC enforcement**: authenticate (verifies JWT + reloads current roles/permissions from the DB on every request) runs first on every protected route, then either requireRoles([…]) (role-name allow-list) or requirePermissions([…]) (permission-code allow-list) gates the specific action. The Auditor role additionally gets blockAuditor applied to almost every route file, since it's the system's one external, non-member account type. See Section 3 and Section 4.23.

2.4 Frontend design system (v1.27.0)

A full visual/UX pass across the entire frontend — dark mode, gradient banners, mobile responsiveness, scrollable-table ergonomics, and in-app back navigation. Implemented at the shared-component/CSS level so it applies application-wide rather than page by page. The five pieces:

1. Light/dark theme — follows the operating system, no in-app toggle.
tailwind.config.js sets darkMode: 'media', which compiles every dark:-prefixed Tailwind class into a @media (prefers-color-scheme: dark) rule. There is no stored preference and no toggle switch anywhere in the UI — the app always matches whatever light/dark setting is active on the device (Windows, macOS, iOS, Android all expose this setting natively). Two layers make this work everywhere: - **Shared component classes** (cms-frontend/src/index.css, @layer components) — .card, .btn-primary/secondary/danger, .input, .label, .badge-*, .table-header, .table-cell, .page-title, .section-title all now carry dark: variants. Since most pages compose these instead of writing raw colour utilities, this one file change covers the majority of the app. - **Blanket override for raw utility classes** — many pages also reach for Tailwind colours directly in JSX (bg-white, text-gray-700, bg-green-50 for an alert box, etc.) — over a thousand occurrences across ~40 files, too many to rewrite individually. index.css adds a @media (prefers-color-scheme: dark) { .bg-white { … } … } block placed *after* @tailwind utilities, so in the compiled CSS these rules win the cascade (same specificity, later in the file) without needing !important. It re-targets the common neutral classes (bg-

white, bg-gray-50/100/200, text-gray-400–900, border-gray-100–300, divide-gray-*) and the bg-{colour}-50 / border-{colour}-200 / text-{colour}-800 trio used for inline alert boxes (blue/green/red/yellow/amber/purple). Any page using these standard classes gets dark mode automatically, including pages written before or after this change. - **Layout components with inline style={{}} colours** (TopBar.jsx) don't use Tailwind classes for their neutral surfaces (they already used inline styles for other reasons, e.g. the sticky header). These now reference CSS custom properties (--cms-surface, --cms-surface-hover, --cms-surface-divider, --cms-border, --cms-text-primary, --cms-text-secondary, --cms-text-muted, defined in index.css :root and flipped in the same dark-mode media query) instead of literal hex codes, so they follow the theme too. Sidebar.jsx is unchanged — its background is always the company's branding colour (a runtime value from the database, not a neutral surface), which reads fine in both modes. - **Known gap**: a handful of standalone auth pages (Login, Register, Forgot/Reset Password) already had their own gradient background and a floating white card design predating this change — they benefit from the blanket override for their bg-white/text-gray-* classes but weren't given a bespoke dark redesign, since a floating branded card is a reasonable "always the same" treatment for a pre-login screen.

2. Gradient banners and tiles. tailwind.config.js adds an accent indigo palette next to the existing primary blue one, plus three reusable backgroundImage tokens (brand-gradient, brand-gradient-soft, brand-gradient-soft-dark). PageHeader.jsx (used at the top of 23 of the ~31 page files) now renders as a .page-banner — a rounded gradient panel (blue → indigo → teal) with white title/subtitle text — instead of plain text on the page background. Since it's one shared component, every page using it picked up the banner treatment without a per-page edit. The two Dashboard pages' StatCard icon chips were also changed from flat colour tints to small gradients (e.g. blue→indigo, emerald→teal) for the "colourful tiles" part of the request. InvestmentDetailPage.jsx and LoanDetailPage.jsx (the two detail/sub-pages that don't use PageHeader) already had a bespoke gradient header from an earlier session and were left as-is.

3. Dual scrollbar tables — reachable from the top, not just the bottom. DataTable.jsx (used directly by 14 page files) now renders two synced horizontal scroll strips around the table: a 14px-tall "phantom" scrollbar directly under the search box/header, and the real one at the bottom (native, under the rows). A hidden 1px-tall spacer div inside the top strip is resized on mount/data-change/window-resize to match the table's true scrollable width (scrollWidth), so the top strip's scrollbar thumb is proportional. Dragging either one updates scrollLeft on the other via a small guarded onScroll handler (a syncingRef flag stops the two handlers from re-triggering each other in an infinite loop). This means a long table with many columns no longer requires scrolling all the way to the bottom of a tall page just to find the horizontal scrollbar.

4. Mobile responsiveness. Largely already in place from earlier sessions — Sidebar.jsx is an off-canvas drawer below the md breakpoint (slides in over a backdrop, closes on backdrop-click or nav-pick), TopBar.jsx shows a hamburger button and hides the centre account-balance strip and search icon below lg/md. This pass added: PageHeader's action buttons wrap onto a new line instead of overflowing on narrow screens (flex-wrap), and two hard-coded grid-cols-2 form layouts (Service Fees agreement form) were changed to

grid-cols-1 sm:grid-cols-2 so paired fields stack on very small screens instead of squeezing two <select>s into one row.

5. Back button on sub-pages. PageHeader.jsx accepts two new optional props: showBack (renders a left-arrow button before the title) and backTo (a specific route to navigate to; omitting it falls back to browser history via navigate(-1)). Wired into GenerateDocumentPage.jsx (backTo="/documents") as the one PageHeader-based sub-page reached by drilling into a list. InvestmentDetailPage.jsx and LoanDetailPage.jsx already had their own "Back to Investments"/"Back to Loans" buttons predating this change and were left as they were. Top-level list pages (Accounts, Loans, Investments, etc. themselves) deliberately do **not** get a back button — there's nowhere meaningful to go "back" to other than the sidebar they're already inside.

Known follow-up, not done in this pass: the blanket dark-mode CSS override (point 1) covers the dominant neutral/alert colour classes but not every colour utility in the app (e.g. less common shades, or hover:/focus: variants beyond the handful listed) — a page using an unusual colour combination may still render a light-mode element inside an otherwise dark page. Worth a targeted pass if any specific page is reported as looking wrong in dark mode.

---

3. Roles & Access Control (RBAC) Summary

The system ships with **10 seeded system roles** (is_system_role = TRUE — their names can't be renamed, though their permission grants can be freely edited). A member can hold multiple roles at once.

Role	Intended purpose
Admin	Full system access and configuration
Director	Company director — financial oversight and approvals (e.g. 3 Directors must jointly approve a Secondary→Primary transfer)
Treasurer	Primary financial approver and accounts manager — the role most money-moving actions are gated to
Assistant Treasurer	Supports the Treasurer with financial recording and contribution acknowledgement
Secretary	Events, documents, and meeting management
Assistant Secretary	Supports the Secretary with events and documents
Coordinator	Operational coordination and project tracking

Shareholder	Capital contributor — personal and general dashboard, the default role for an ordinary member
Auditor	The system's only **external, non-member** role — read-only access to a specific, scoped audit engagement and nothing else (see Section 4.27)
Administrative Officer	Hired/contracted staff (v1.21.0) — meetings, minutes, and correspondence; blocked from all company financial data except individual documents an Admin explicitly grants (see Section 4.28)

Two enforcement mechanisms exist side by side, and which one a given route uses is inconsistent across the codebase (a known, still-open issue — see Section 8): - **requireRoles(['Treasurer', 'Director'])** — checks the user's role *names* directly. Fragile to role renames; can't be reconfigured without a code change. - **requirePermissions(['FINANCE_TRANSACTION_CREATE'])** — checks fine-grained permission *codes*, which are freely assignable to any role (including custom roles) through Settings → Roles → Permissions.

Critical operational fact: role_permissions (the table mapping roles to permissions) ships with **zero seed data**. No role — not even Admin — has any permission granted at install time. Every single permission grant must be made by hand, once, through **Settings → Roles → (shield icon) → Permissions modal**. Until that's done for a role, every requirePermissions(…)-gated action is unreachable for anyone holding only that role. This is the single most important post-install step and is called out again in Section 7.

The **Auditor** role is architecturally different from most others: it is granted *zero* permissions by design, and instead of permission-based access, every legitimate action it can take lives under /api/audit/* and is scoped to whichever specific audit engagement(s) an Admin explicitly attached it to (see Section 4.27).

blockRoles(roleNames, message) (v1.21.0, middleware/auth.js) is the generic factory both finance-blocking middlewares are now built from — added so a future restricted role doesn't require inventing another near-duplicate block function: - **blockAuditor** — Auditor only. Applied to events.js and documents.js, since an Administrative Officer legitimately needs (partial) access to both. - **blockFinanceRestricted** — Auditor **and** Administrative Officer. Applied to every other finance-adjacent route file (Accounts, Transactions, Transfers, Grants, Loans, Investments, Requisitions, Savings, Side Fund, Dividends, Reports, Search, Shares, Certificates, Exchange Rates, System, /users/shareholding).

This is a defense-in-depth measure, not the only line of defense — role_permissions shipping empty already means an unconfigured role reaches nothing gated by requirePermissions, but a hard deny-list at the route layer means a mis-click while

configuring permissions in Settings can't accidentally expose company-wide financial data to a role that should never see it.

Zero-role accounts (v1.21.1) — requireAssignedRole (middleware/auth.js). Registering and verifying an email is not the same as being approved: a freshly verified account holds **no role at all** until an Admin assigns one (via Users → Assign Role, or by approving a role request made at registration). blockFinanceRestricted and blockAuditor are deny-lists for two *specific* known roles — they do nothing to stop a zero-role account, since it isn't Auditor or Administrative Officer either. requireAssignedRole is the allow-list half of that: it rejects any request from a user with an empty roles array, and is applied to every route file that has "open to any authenticated user" endpoints (Accounts, Transactions, Transfers, Grants, Loans, Investments, Events, Documents, Reports, System, Dividends, Savings, Side Fund, Requisitions, Shares, Exchange Rates, Certificates, Search, Audit, Staff Access, Service Fees, Categories, plus GET /users/shareholding). Deliberately **not** applied to GET/PATCH /users/me, GET /users/me/role-request, GET /users/roles, notifications.js, or settings.js's company-branding read — a pending account still needs to see/edit its own profile, check whether its role request has been reviewed, and render a branded page.

On the frontend, AppLayout.jsx checks the same condition centrally (same pattern as the Auditor force-redirect) and sends a zero-role account to /pending-approval (PendingApprovalPage.jsx) — a standalone page with no Sidebar/TopBar, showing their requested role and its status, a "Check again" button, and a 30-second background poll that takes them into the real app automatically the moment a role lands. This closes the gap between "email verified" and "Admin approved" that previously landed a brand-new account straight on the Dashboard, which — like the other endpoints above — shows real company account balances to anyone who gets that far.

Not-yet-consented accounts (v1.23.0) — requireConsent (middleware/auth.js), applied immediately after requireAssignedRole in the same route files. The next step after a role is assigned: draw a signature and consent to the Membership Agreement, once, ever (Section 4.29 in full). AppLayout.jsx redirects any role-assigned-but-not-consented account to /consent (ConsentPage.jsx) before the Sidebar/TopBar/Dashboard, same shape as the pending-approval redirect above. This applies to every existing member too on their next login after the v1.23.0 migration runs, not just new sign-ups — there's no way to infer past usage already implied agreement.

---

4. Module Reference

This section documents every functional module in the system: purpose, database schema, business rules, API endpoints, frontend behavior, and known issues. Modules are grouped the way the codebase groups them.

1. ACCOUNTS

1.1 Purpose

ZWECK TUKULA holds its cash in a small set of ledger “accounts” rather than one pot: a **PRIMARY** account (the club’s main EUR account), zero or more **SECONDARY** accounts (e.g. an “Investment Reserve Fund” or a bank sub-account in another currency), and exactly one dedicated **SAVINGS** account (the sole target of every member savings deposit/handout, kept deliberately separate from operational Primary funds). This lets the club: enforce a minimum operating balance (“floor limit”) on any operational account so the Treasurer can’t spend it to zero; track bank details per account; give every account’s transaction trail a distinguishable reference prefix; and support a “side fund” allocation that is carved out of an account’s balance for display purposes without touching the real ledger total.

1.2 Data model

accounts (cms/schema.sql:235) | Column | Type | Notes | |—|—|—| | id | SERIAL PK | | | account_type | VARCHAR(20) | CHECK IN (‘PRIMARY’,‘SECONDARY’,‘SAVINGS’) | | name | VARCHAR(150) NOT NULL | | | currency_id | INTEGER NOT NULL | FK → currencies(id) | | description | TEXT | | | current_balance | NUMERIC(20,4) NOT NULL DEFAULT 0 | CHECK ≥ 0 | | is_active | BOOLEAN NOT NULL DEFAULT TRUE | | | created_at, created_by | | created_by FK → users(id) | | reference_prefix | VARCHAR(10) UNIQUE | optional short code (e.g. IRF) used instead of generic PA/SA in reference codes | | is_virtual | BOOLEAN NOT NULL DEFAULT FALSE | true = no real bank behind it | | bank_name, bank_branch, bank_account_number, swift_routing_code | VARCHAR | | | — | CONSTRAINT check_balance_not_negative | current_balance ≥ 0 | | — | CONSTRAINT bank_details_required_unless_virtual | is_virtual = TRUE OR (bank_name AND bank_account_number NOT NULL) |

Two DB-level partial unique indexes enforce singleton accounts (not just app logic, so they can’t race): - idx_one_primary_account — unique on account_type WHERE account_type='PRIMARY' AND is_active=TRUE - idx_one_savings_account — unique on account_type WHERE account_type='SAVINGS' AND is_active=TRUE

primary_account_floor_limits (schema.sql:293) — despite the name, applies to any account type except SAVINGS as of v1.14.0: | Column | Type | |—|—| | id | SERIAL PK | | account_id | FK → accounts(id) | | floor_amount | NUMERIC(20,4), CHECK ≥ 0 | | effective_from | DATE NOT NULL | | effective_to | DATE (NULL = currently active) | | set_by | FK → users(id) | | approved_by | FK → users(id), nullable | | notes | TEXT |

currencies (schema.sql:66): id, code (unique), name, symbol, is_active, created_at, created_by.

side_fund_config (referenced via sfc join, defined elsewhere) supplies current_balance per parent_account_id when is_active = TRUE — used to split an account’s ledger balance into a “general” display balance and a separately-shown “side fund allocation.”

Relationships: accounts.currency_id → currencies.id; primary_account_floor_limits.account_id → accounts.id; transactions.account_id → accounts.id; transfers.from_account_id/to_account_id → accounts.id.

1.3 Business rules / key logic

- **Only one PRIMARY and one SAVINGS account** can exist at a time — enforced by DB partial unique indexes, not just controller checks.
- **SAVINGS account exemptions**: can never take part in a transfer (transfer legs are only ever PRIMARY↔SECONDARY, so SAVINGS is automatically excluded), is permanently exempt from floor limits, and may sit at exactly zero at any time.
- **Floor limit**: any account except SAVINGS may have one. updateFloorLimit (accountsController.js:527) closes out the previous floor-limit row (effective_to) and inserts a new one — **history is never overwritten**, only superseded. A floor limit can only be changed once every **6 months** (rolling from effective_from of the current row); attempting sooner throws a badRequest naming the next allowed date. Any account may only have a floor limit set with permission FINANCE_FLOOR_LIMIT_UPDATE.
- **Reference prefix**: optional per-account short code (max 10 chars, letters/numbers only, globally unique across accounts.reference_prefix). If left blank on creation, deriveReferencePrefix (accountsController.js:28) auto-derives one from the first 6 alphanumeric uppercase characters of the account name, appending a digit suffix on collision (giving up after 20 tries, in which case the account falls back to the generic SA prefix at transaction time). resolveModuleCode() (referenceService.js:176) resolves which prefix a given account's transactions actually use: the account's own reference_prefix if set, else PA for PRIMARY, SAV for SAVINGS, SA for SECONDARY.
- **Bank details**: required (bank_name, bank_account_number) unless is_virtual = TRUE, enforced both by a DB CHECK constraint and in the controller (so a virtual/internal tracking account needs no real bank).
- **Side fund display split**: current_balance returned to the frontend on getAllAccounts/getAccountById/getAccountSummary is the account's real ledger balance **minus** any active side fund allocation (side_fund_config.current_balance where is_active=TRUE); the true, untouched ledger figure is separately exposed as ledger_balance. This is display-only — postTransaction always operates on the real ledger balance.
- **Available balance** (getAccountSummary) = general (side-fund-excluded) balance minus the account's current floor limit (0 for SAVINGS, which has no floor concept).
- The PRIMARY account is always created against **EUR** specifically (createPrimaryAccount hard-codes lookup of currencies WHERE code='EUR'); SECONDARY/SAVINGS accounts can be any active currency chosen by the admin.
- Updating an account (updateAccount) cannot change account_type or currency_id — those are treated as foundational and immutable once transactions exist against the account.
- Currencies can be added/edited by Admin (SYSTEM_CONFIG); editing a currency's code/name/symbol/active flag never breaks existing accounts since they reference currency_id, not the code string.

1.4 API endpoints (cms/src/routes/accounts.js, prefix /api/accounts, all require authenticate + blockAuditor)

Method	Path	Permission	Description
GET	/currencies	any authenticated user	List active currencies
POST	/currencies	SYSTEM_CONFIG	Add a new currency
PATCH	/currencies/:id	SYSTEM_CONFIG	Edit/deactivate a currency
GET	/summary	any authenticated user	Dashboard balance summary (all active accounts, with floor/available balance)
POST	/primary	SYSTEM_CONFIG	One-time creation of the Primary EUR account
POST	/savings	SYSTEM_CONFIG	One-time creation of the Savings account
GET	/	FINANCE_VIEW_ALL	List all accounts with balances
POST	/	SYSTEM_CONFIG	Create a Secondary account
GET	/:id	FINANCE_VIEW_ALL	Full account detail incl. floor-limit history
PATCH	/:id	SYSTEM_CONFIG	Update name/description/reference prefix/bank details
POST	/:id/floor-limit	FINANCE_FLOOR_LIMIT_UPDATE	Set/replace floor limit (6-month cooldown; SAVINGS rejected)

1.5 Frontend — cms-frontend/src/pages/accounts/AccountsPage.jsx

- **Main list view**: Share Price card, Exchange Rates card, then a responsive grid of AccountTiles (one per account) showing balance, floor-limit progress bar (if applicable), side-fund allocation callout, reference prefix chip, and a hover-revealed "Floor Limit" shortcut (gated on FINANCE_FLOOR_LIMIT_UPDATE).
- **"New Account" button** (top of page) — visible only with SYSTEM_CONFIG; opens CreateAccountModal, which toggles between creating a **Secondary** account and

(only if no SAVINGS account exists yet) the one-time **Savings** account setup, with a virtual/bank-details toggle.

- If no savings account exists yet, an amber banner ("No savings account has been set up yet…") is shown to SYSTEM_CONFIG holders with a shortcut button.
- **Clicking a tile** opens AccountDetailView: bank details card, gradient balance header (Available / Total Inflows / Total Outflows stat tiles), a balance-over-time area chart and inflow/outflow pie chart (Recharts), and a paginated transaction ledger table for that account.
  - canRecordTransactions (role Treasurer/Assistant Treasurer) shows "Record Inflow" / "Record Expense" buttons (hidden entirely for the SAVINGS account, which instead shows a note redirecting to the Savings page).
  - canEdit (SYSTEM_CONFIG) shows an "Edit Account" button → EditAccountModal.
- **Share Price card** (SharePriceCard) and its edit modal (SetSharePriceModal, gated on isTreasurer()) live on this same page even though share pricing is logically a Shares module concern — flagged for completeness.
- **Exchange Rates card** (ExchangeRatesCard) and SetExchangeRateModal also live on this page (see Section 4.5).

1.6 Known issues (Accounts)

- The diagnostic report's floor-limit permission mismatch (route gated by role vs. frontend gated by permission) is listed as **fixed** in SYSTEM_DIAGNOSTIC_REPORT.md, and the code confirms it — accounts.js:206 now uses requirePermissions(['FINANCE_FLOOR_LIMIT_UPDATE']), matching the frontend's hasPermission check.
- **Still present**: accountsController.js:594, inside updateFloorLimit's audit log call — description: \Floor limit updated to ${floor_amount} EUR`` hard-codes "EUR" regardless of the account's actual currency. Since floor limits now apply to any account (not just the EUR Primary), a floor-limit change on a non-EUR secondary account produces a misleading audit trail entry.
- POST /accounts/primary has no confirmed frontend entry point (the "New Account" modal only creates Secondary/Savings) — logically sound but presumably a one-time manual/API-only setup step, per the diagnostic report.

---

2. TRANSACTIONS

2.1 Purpose

The single, immutable ledger of every money movement on any account — contributions, expenses, generic inflows, and the individual debit/credit legs posted by other modules (transfers, loans, grants, investments, savings). This is the system of record the Treasurer, Directors, and Admin rely on for the club's real financial position.

2.2 Data model

transactions (schema.sql:407) | Column | Type | Notes | |—|—|—| | id | SERIAL PK | | | reference_id | FK → references_registry(id) NOT NULL | | | account_id | FK → accounts(id) NOT NULL | | | transaction_type | VARCHAR(30) | CHECK IN ('CREDIT','DEBIT','REVERSAL_CREDIT','REVERSAL_DEBIT') | | inflow_type | VARCHAR(30) | CHECK IN a fixed list (CONTRIBUTION, GRANT, LOAN_RECEIVED, LOAN_REPAYMENT_IN, INTEREST_IN, INVESTMENT_RETURN, TRANSFER_IN, OTHER_INCOME, SAVINGS_DEPOSIT_IN, TRANSFER_OUT, LOAN_DISBURSED, LOAN_REPAYMENT_OUT, INTEREST_OUT, EXPENSE, SAVINGS_HANDOUT_OUT, GRANT_REFUND, SIDE_FUND_CONTRIBUTION_IN, SIDE_FUND_DIRECT_IN, SAVINGS_POOL_OTHER_IN) | | amount | NUMERIC(20,4) NOT NULL, CHECK > 0 | | | currency_id | FK → currencies(id) | | | balance_before, balance_after | NUMERIC(20,4) NOT NULL | running balance snapshot at post time | | category_id | FK → categories(id) NOT NULL | | | description | TEXT NOT NULL | | | value_date | DATE NOT NULL | | | transaction_date | TIMESTAMPTZ DEFAULT NOW() | | | reversal_of | FK → transactions(id) nullable | set on a reversal entry, points to the original | | is_reversal | BOOLEAN DEFAULT FALSE | | | is_reversed | BOOLEAN DEFAULT FALSE | set on the *original* once reversed | | transfer_id | FK → transfers(id), added via ALTER TABLE | | | contribution_id, grant_tranche_id, loan_received_id, loan_given_id, investment_id | nullable FKs — "only one populated per transaction" | | | status | VARCHAR(30) DEFAULT 'PENDING' | CHECK IN ('PENDING','APPROVED','POSTED','REVERSED','REJECTED') — in practice postTransaction always inserts status 'POSTED' directly | | created_by | FK → users(id) NOT NULL | | | contributed_by | FK → users(id) nullable | member the transaction is recorded *on behalf of*, distinct from created_by | | approved_by, approved_at, posted_at | | postTransaction sets approved_by = created_by and both timestamps to NOW() immediately — there is no separate approval step at the transaction level (approval, where required, happens upstream, e.g. transfers) |

shareholder_contributions (schema.sql:307) — one row per contribution event, separate from the transactions row it generates: reference_id, user_id, account_id, amount, currency_id, contribution_date, category_id, notes, status (PENDING/APPROVED/REJECTED/REVERSED, though recordContribution inserts directly as 'APPROVED'), created_at, created_by. CHECK amount > 0.

Relationships: every transactions row belongs to exactly one accounts row and one categories row, is traceable to a references_registry row for its human-readable code, and optionally links back to whichever module record generated it.

2.3 Business rules / key logic

postTransaction (transactionsController.js:52) is the single core function every credit/debit in the system funnels through (contributions, expenses, inflows, transfer legs, bank charges, reversals — and other modules like loans/grants/investments call it too). Its rules: 1. Locks the target account row with SELECT … FOR UPDATE to serialize concurrent balance updates. 2. Computes balanceAfter = before + amount (CREDIT/REVERSAL_CREDIT) or before − amount (DEBIT/REVERSAL_DEBIT). 3. **Rule: no**

account may go below zero — throws badRequest “Insufficient funds” if balanceAfter < 0. 4. **Rule: floor limit enforcement** — for any non-SAVINGS account, if a floor limit is currently set (getFloorLimit, looks up the row with effective_to IS NULL), a transaction that would push the balance below that floor is rejected with the current balance and “available to spend” shown in the error. SAVINGS is hard-coded to floor limit 0 (never enforced). 5. Inserts the transaction row with status='POSTED', approved_by = created_by, approved_at/posted_at = NOW() — i.e. posting and approval are the same event at this layer. 6. Updates accounts.current_balance to the new balance. 7. Returns { transactionId, balanceBefore, balanceAfter } to the caller, which is responsible for linking the reference code to the new transaction row (linkReferenceToRecord) and writing its own audit-log entry.

Contributions (creditShareholderContribution, shared by recordContribution and the requisitions module’s contribution-acknowledgement approval path): - Verifies the contributor is an active user; auto-creates a shareholding_registry row (0 shares) on first-ever contribution if none exists. - Always posts against the single active PRIMARY account. - Generates reference PA-CONTRIB-YYYYMM-00001 (or the Primary account’s custom prefix). - **Recalculates every shareholder’s percentage** after each contribution: sums shareholder_contributions (status APPROVED) per user, divides each member’s total by the grand total, and writes the resulting shares_held/percentage back to their shareholding_registry row (4-decimal percentage). This is a full recompute across all shareholders, not an incremental update. - Sends a bell notification + email to the contributor (best-effort — never rolls back the transaction on notification failure).

Side fund split (v1.26.0) — recordContribution accepts an optional side_fund_amount, sliced out of the total amount before either path runs: the side fund portion is credited via creditSideFundContribution (its own ledger transaction into the side fund’s parent account, then applySideFundPayment applies it to that same member’s own dues oldest-unpaid-first — Section 4.10.3), while only the remainder (amount − side_fund_amount) goes through creditShareholderContribution above as the capital contribution. If the remainder is zero, no contribution row is created at all. Requires the side fund to be active; the two halves are logged and notified independently.

General inflow / expense (recordInflow, recordExpense): posted against any active account chosen by the caller, with inflow_type fixed to OTHER_INCOME / EXPENSE respectively. Reference module code resolved via resolveModuleCode(account).

Reversal (reverseTransaction): - **Corrections are reversal entries only** — nothing is ever edited or deleted. - Cannot reverse a transaction already marked is_reversed, and cannot reverse a reversal entry itself (is_reversal = TRUE) — reversals are one level deep only. - Posts a brand-new transaction of the opposite type (CREDIT↔REVERSAL_DEBIT, DEBIT↔REVERSAL_CREDIT) for the same amount/account/category, described as REVERSAL of <original description> — Reason: <reason>, dated today (not the original’s value_date), linked via reversal_of. - Marks the original row’s is_reversed = TRUE (the original is never modified otherwise). - Runs through the same postTransaction floor-limit/zero-balance checks as any other transaction — a reversal can itself be rejected if it would violate those rules.

2.4 API endpoints (cms/src/routes/transactions.js, prefix /api/transactions)

Method	Path	Role/Permission	Description
GET	/	FINANCE_VIEW_ALL	Paginated ledger; filters: account_id, inflow_type, from_date, to_date
GET	/:id	FINANCE_VIEW_ALL	Single transaction with full joins (reference, category trail, account, currency)
POST	/contributions	requireRoles(['Treasurer','Assistant Treasurer'])	Record a shareholder contribution; regular members can no longer post their own — they submit a CONTRIBUTION_ACKNOWLEDGEMENT requisition instead. Optional side_fund_amount (v1.26.0) slices a side fund payment out of the total (Section 4.10.3)
POST	/expenses	requireRoles(['Treasurer','Assistant Treasurer'])	Direct expense from any account
POST	/inflows	requireRoles(['Treasurer','Assistant Treasurer'])	Direct miscellaneous inflow to any account
POST	/:id/reverse	requireRoles(['Treasurer'])	Reverse a posted transaction (reason required)

All routes also pass through authenticate and blockAuditor.

2.5 Frontend — cms-frontend/src/pages/transactions/TransactionsPage.jsx

- Full ledger table (DataTable) with columns: Reference (click to preview a printable document + shows public_id), Description/Account, Type (inflow_type label), Category (full trail), Amount (green/red by CREDIT vs DEBIT), Balance After, Date, Status badge, and an actions column.

- **Filter bar**: account, inflow type (dropdown covering the common types: Contribution, Expense, Transfer In/Out, Grant, Loan Received, Investment Return), from/to date, with a "Clear filters" link.
- **"Record" split-button** (visible only with FINANCE_TRANSACTION_CREATE) opens a small dropdown menu with "Record Contribution" and "Record Expense" options, each launching its own modal:
  - ContributionModal — amount/date/category/notes; a "Contributing Member" selector appears only for users with FINANCE_VIEW_ALL (i.e. Treasurer/Directors/Admin can record on behalf of any shareholder; otherwise defaults to the logged-in user).
  - ExpenseModal — account/amount/category/description/date.
- **Export**: "Export" button prints/exports the whole filtered ledger via transactionTemplate; each row also has a per-row export icon.
- **Reverse action**: a red reverse-arrow icon appears per row only for users with role Treasurer, and only when the row is neither already a reversal (is_reversal) nor already reversed (is_reversed) — opens ReverseTransactionModal, which requires a typed reason and explicitly warns the reversal is a new equal-and-opposite entry, not an edit/delete.

2.6 Known issues (Transactions)

- Diagnostic report item confirmed **fixed**: a Treasurer-gated Reverse button now exists in TransactionsPage.jsx (previously flagged as a dead/no-UI backend endpoint).
- Diagnostic report item confirmed **fixed**: the savings-handout category_id: null crash (savingsController.js) is outside this module's files but directly interacts with postTransaction's category_id NOT NULL requirement — worth cross-referencing in the Savings section.
- Minor/cosmetic, confirmed by diagnostic report: creditShareholderContribution (transactionsController.js:269-270) passes a contributedBy key into postTransaction's options object, but postTransaction's destructured parameter list has no contributedBy field — it's silently dropped. The actual contributed_by column write happens via a separate immediate UPDATE transactions SET contributed_by = … right after. Functionally harmless, but the dead parameter in the call is misleading and worth removing.
- transactions.status supports a PENDING/APPROVED/REJECTED lifecycle at the schema level, but postTransaction always inserts 'POSTED' with approved_by/approved_at set immediately — there is no actual pending-transaction workflow at this layer (approval workflows exist upstream, e.g. Transfers, not on the transaction row itself). Not a bug, but worth noting the schema is more general than the current usage.

---

3. TRANSFERS

3.1 Purpose

Moves money between the club's own accounts (Primary ↔ Secondary) under an approval workflow proportional to risk — a single Treasurer sign-off suffices to push money *out* of Primary into a secondary account, but pulling money *back into* Primary from a secondary account requires three separate Director approvals. This asymmetry protects the club's main account from being drained by a single compromised or mistaken approval while still allowing routine funding of secondary accounts.

3.2 Data model

transfers (schema.sql:473) | Column | Type | Notes | |—|—|—| | id | SERIAL PK | | | reference_id | FK → references_registry(id) | | | from_account_id, to_account_id | FK → accounts(id) | CHECK no_self_transfer (from ≠ to) | | transfer_type | VARCHAR(30) | CHECK IN ('PRIMARY_TO_SECONDARY','SECONDARY_TO_PRIMARY') | | amount_sent | NUMERIC(20,4) NOT NULL | CHECK > 0 | | currency_sent_id | FK → currencies(id) | | | amount_received | NUMERIC(20,4) NOT NULL | CHECK > 0 | | currency_received_id | FK → currencies(id) | | | exchange_rate | NUMERIC(20,8) NOT NULL | manually entered per-transfer, not sourced from currency_exchange_rates | | exchange_rate_entered_by | FK → users(id) | | | category_id | FK → categories(id) NOT NULL | | | description | TEXT | | | value_date | DATE NOT NULL | | | sending_bank_charge, receiving_bank_charge | NUMERIC(20,4) DEFAULT 0 | CHECK ≥ 0 each | | status | VARCHAR(30) DEFAULT 'PENDING' | CHECK IN ('PENDING','AWAITING_APPROVAL','APPROVED','POSTED','REJECTED','REVERSED') — controller only ever uses AWAITING_APPROVAL → POSTED/REJECTED | | debit_transaction_id, credit_transaction_id | FK → transactions(id) | populated once posted | | sending_charge_tx_id, receiving_charge_tx_id | FK → transactions(id), nullable | populated only if a charge > 0 | | created_at, created_by | | |

approval_workflows (schema.sql:522): generic approval engine reused across modules. Relevant columns: workflow_type (PRIMARY_TO_SECONDARY_TRANSFER / SECONDARY_TO_PRIMARY_TRANSFER for this module), record_type='transfers', record_id, required_approvals, current_approvals, status (PENDING/APPROVED/REJECTED/CANCELLED), initiated_by/at, completed_at, notes.

approval_actions (schema.sql:551): one row per individual approval/rejection — workflow_id, actor_id, action (APPROVED/REJECTED/ABSTAINED), role_id (which role the actor approved *as*), notes, acted_at. UNIQUE (workflow_id, actor_id) — a person cannot approve the same transfer twice.

3.3 Business rules / key logic

- **Every transfer records the exchange rate manually** — the Director initiating the transfer types in a rate (or it's auto-locked to 1 when both accounts share a currency; the frontend disables the rate field in that case too). This rate is independent of, and not sourced from, the currency_exchange_rates table (Section 4) — that table is display-only for share values.

- **Transfer type is derived, not chosen** — only PRIMARY↔SECONDARY combinations are legal; any other pairing (e.g. involving SAVINGS, or Secondary↔Secondary) throws badRequest. This is also how SAVINGS is automatically excluded from transfers without extra code.
- **Approval requirement**: PRIMARY_TO_SECONDARY needs exactly **1** approval, from a user holding the Treasurer role. SECONDARY_TO_PRIMARY needs exactly **3** approvals, each from a user holding the Director role (any three distinct directors — the code checks req.user.roles.includes('Director') per approval, not that the *same* director hasn't already approved twice, which is separately enforced by the UNIQUE (workflow_id, actor_id) constraint on approval_actions).
- **Edit before approval only**: editTransfer only permits changes while status = 'AWAITING_APPROVAL'; once even one approval has landed, the transfer is locked (must be rejected and recreated instead). Editable by whoever created it, or by anyone holding FINANCE_TRANSFER_APPROVE. From/to accounts cannot be changed on edit (frontend disables those selects; only amount/rate/category/description/value_date/charges are editable).
- **Dual-post accounting pattern**: once the required approval count is reached, approveTransfer posts **two** transactions atomically inside the same DB transaction — a DEBIT leg on from_account_id (inflow_type='TRANSFER_OUT') and a CREDIT leg on to_account_id (inflow_type='TRANSFER_IN') — each going through the same postTransaction floor-limit/zero-balance checks as any other transaction. **Either both legs succeed or neither does** (single withTransaction wrapper).
- **Bank charges posted separately**: if sending_bank_charge > 0, an additional DEBIT/EXPENSE transaction is posted on the *source* account; if receiving_bank_charge > 0, an additional DEBIT/EXPENSE transaction is posted on the *destination* account. Each gets its own reference code (BANK-CHG category abbreviation) and its own postTransaction call — so a transfer with both charges set posts up to **4** transaction rows in total.
- **Reference codes**: transfer itself gets TRF-P2S-YYYYMM-00001 or TRF-S2P-… (module code TRF, fixed regardless of account prefixes). Each transaction leg gets its own reference using its own account's resolveModuleCode() (own reference_prefix if set, else generic PA/SA) with abbreviation TRF-OUT / TRF-IN / BANK-CHG.
- **Reject**: only while AWAITING_APPROVAL; sets both transfers.status and the approval_workflows row to REJECTED, records the reason, notifies the initiator (bell + email).
- **Notifications**: partial approvals send a bell-only "approval progress" notification to the initiator (no email, to avoid noise); the final approval that posts the transfer sends both bell and email with the full transfer detail; rejection sends both.

3.4 API endpoints (cms/src/routes/transfers.js, prefix /api/transfers)

Method	Path	Permission	Description
GET	/	FINANCE_VIEW_ALL	List transfers; filters status, transfer_type

GET	/:id	FINANCE_VIEW_ALL	Full detail incl. approval history
POST	/	FINANCE_TRANSFER_CREATE	Initiate a transfer (starts AWAITING_APPROVAL)
PATCH	/:id	requireAnyPermission(['FINANCE_TRANSFER_CREATE','FINANCE_TRANSFER_APPROVE'])	Edit before any approval lands
POST	/:id/approve	FINANCE_TRANSFER_APPROVE	Record an approval; posts both legs once quorum reached
POST	/:id/reject	requireAnyPermission(['FINANCE_TRANSFER_APPROVE','FINANCE_TRANSFER_APPROVE_REVERSE'])	Reject with reason

Note: role checks for *who specifically* can approve which transfer type (Treasurer vs Director) are additionally enforced **inside the controller**, not just at the route-permission layer.

3.5 Frontend — cms-frontend/src/pages/transfers/TransfersPage.jsx

- DataTable of all transfers: Reference (click-to-preview + public_id), From→To (with a "Primary → Secondary"/"Secondary → Primary" subtitle), Amount Sent (red), Amount Received (green), Rate, Charges (Send/Recv, only shown if non-zero), Approvals (current/required), Date, Status badge, Actions, Export.
- **"New Transfer" button** — gated on FINANCE_TRANSFER_CREATE — opens TransferModal for initiation.
- TransferModal doubles as the edit modal (editingRecord prop): From/To account selects are disabled when editing (accounts are locked after creation); the Exchange Rate field is replaced with a disabled "1 (same currency)" input whenever the selected accounts share a currency; live preview of "Amount to be received" as the user types; separate optional Sending/Receiving Bank Charge fields.
- **Row actions**: Edit icon shown via canEdit(row) — status is AWAITING_APPROVAL AND (the user is the creator OR holds FINANCE_TRANSFER_APPROVE). Approve (green check) / Reject (red X) icons shown when status is AWAITING_APPROVAL and the user holds FINANCE_TRANSFER_APPROVE — note the frontend does not further distinguish Treasurer-vs-Director-only visibility client-side; that final gate is

enforced server-side inside approveTransfer. Reject uses a plain window.prompt() for the reason.

3.6 Known issues (Transfers)

- The frontend's Approve/Reject buttons are shown to *anyone* with FINANCE_TRANSFER_APPROVE, without checking client-side whether the user actually holds the specific Treasurer (for P2S) or Director (for S2P) role required by the backend for that particular transfer — so a permission holder without the matching role would see and click the button only to get a 403 from the server. Not corruption-risk (the backend enforces correctly) but a UX rough edge worth flagging, in the same family as the already-documented floor-limit permission mismatch.
- rejectTransfer's route accepts FINANCE_TRANSFER_APPROVE_REVERSE as an alternate permission to reject a transfer, but that permission's stated purpose in schema.sql is "Approve secondary-to-primary transfer" — reusing it to also gate *rejection* of any transfer type is a slightly surprising overload, worth double-checking against the intended access model.
- transfers.status schema CHECK permits PENDING, APPROVED, and REVERSED values that the current controller code never actually sets (it only ever moves AWAITING_APPROVAL → POSTED or AWAITING_APPROVAL → REJECTED) — likely legacy/future-proofing, not a bug, but means there is no reversal mechanism at the transfer level (a posted transfer's underlying transactions could presumably each be reversed individually via the Transactions reverse endpoint, but that's not a transfer-aware operation).

---

4. EXCHANGE RATES

4.1 Purpose

Lets the Treasurer/Assistant Treasurer/Admin record a monthly, company-declared conversion rate between currency pairs (e.g. EUR→UGX) purely so the club's share price/value can be *displayed* to members in a currency other than the one it was set in. This is explicitly **cosmetic/display-only** — it never affects how any contribution, transaction, or transfer amount is actually recorded or calculated (transfers between differently-currencied accounts use their own manually-entered, transfer-specific rate — see Section 3.3).

4.2 Data model

currency_exchange_rates (schema.sql:358) | Column | Type | Notes | |—|—|—| | id | SERIAL PK | | | base_currency_id | FK → currencies(id) NOT NULL | | | target_currency_id | FK → currencies(id) NOT NULL | | | rate | NUMERIC(20,6) NOT NULL | CHECK > 0; "1 unit of base = rate units of target" | | effective_from | DATE NOT NULL | | | effective_to | DATE nullable | NULL = currently active for that pair | | set_by | FK → users(id) NOT NULL | | | notes | TEXT | | | created_at | TIMESTAMPTZ | | | — | CHECK positive_exchange_rate | rate > 0 | | — | CHECK different_currencies | base_currency_id ≠ target_currency_id |

Index: idx_fx_rates_current on (base_currency_id, target_currency_id) WHERE effective_to IS NULL — supports fast lookup of the currently active rate per pair.

4.3 Business rules / key logic

- **Per-pair history, same pattern as primary_account_floor_limits/share_price_history**: setExchangeRate first closes out (effective_to = effectiveDate) whatever row currently has effective_to IS NULL for that **exact** base→target pair, then inserts the new rate. Other currency pairs are untouched — each pair maintains its own independent history.
- Base and target currency must differ (enforced both by controller check and DB CHECK constraint).
- rate must be a positive number > 0.
- effective_from defaults to today if not supplied.
- No automatic reverse-rate creation — setting EUR→UGX does not also create/update a UGX→EUR row; each direction is set independently if needed.
- Audit-logged under MODULES.SYSTEM (not MODULES.FINANCE) with action SYSTEM_CONFIG_CHANGED.

4.4 API endpoints (cms/src/routes/exchangeRates.js, prefix /api/exchange-rates)

Method	Path	Role	Description
GET	/current	any authenticated user	All currently-active rates (one per pair, effective_to IS NULL)
GET	/history	any authenticated user	Paginated full history across all pairs, newest effective_from first
POST	/	requireRoles(['Treasurer','Assistant Treasurer','Admin'])	Set/replace the current rate for a base→target pair

4.5 Frontend

There is **no standalone Exchange Rates page** — the UI is embedded directly in cms-frontend/src/pages/accounts/AccountsPage.jsx as the ExchangeRatesCard component, shown near the top of the Accounts page (below the Share Price card, above the account tiles): - Displays each active pair as a small tile: BASE → TARGET, 1 = <rate>, “since <effective_from>”. - “Set Rate” button — gated on hasRole(['Treasurer','Assistant Treasurer','Admin']) — opens SetExchangeRateModal: From Currency / To Currency selects (populated from all active currencies), Rate input, Effective From date, Notes. The modal explicitly tells the user this “does not change any recorded amount.” - There is no dedicated history view in the frontend for GET /exchange-rates/history — only the current-rate list is surfaced.

4.6 Known issues (Exchange Rates)

- GET /exchange-rates/history exists on the backend but has no confirmed frontend consumer — a real but low-severity gap flagged generically in the diagnostic report ("A handful of backend endpoints have no confirmed UI surface").
- Because this rate is entirely separate from the rate a Director manually types into a Transfer, there is real potential for the two numbers to diverge (e.g. the "official" monthly EUR→UGX rate shown on the Accounts page vs. whatever rate was actually used for a specific bank transfer) — this is a deliberate design choice per the code comments, not a bug, but worth stating clearly so nobody assumes the two are reconciled automatically.

---

5. CATEGORIES

5.1 Purpose

A single, unlimited-depth hierarchical category system shared by every module that needs to classify a record (Finance transactions/contributions/expenses/transfers, Documents, Events, Investments, and a catch-all "General"). Centralizing this avoids five different bespoke category systems and lets reference codes and reports use a consistent, human-readable category trail (e.g. "Finance > Operations > Utilities").

5.2 Data model

categories (schema.sql:176) | Column | Type | Notes | |—|—|—| | id | SERIAL PK | | | parent_id | FK → categories(id) nullable | self-referential, enables unlimited nesting | | module | VARCHAR(50) NOT NULL | CHECK IN ('FINANCE','DOCUMENT','EVENT','INVESTMENT','GENERAL') | | name | VARCHAR(150) NOT NULL | | | abbreviation | VARCHAR(20) NOT NULL | used to build reference codes (e.g. CONTRIB, EXPENSE) | | description | TEXT | | | is_active | BOOLEAN DEFAULT TRUE | | | created_at, created_by | | | | — | UNIQUE (parent_id, name, module) | prevents duplicate sibling names within the same module |

category_paths (schema.sql:190) — a denormalized, precomputed cache table, one row per category: | Column | Type | |—|—| | category_id | INTEGER PK, FK → categories(id) | | full_path | TEXT NOT NULL — e.g. "Finance > Operations > Utilities" | | full_abbreviation | TEXT NOT NULL — e.g. "FIN-OPS-UTIL" | | depth | INTEGER DEFAULT 0 | | updated_at | TIMESTAMPTZ |

5.3 Business rules / key logic

- **rebuildCategoryPath** (categories.js:21) is the internal engine: a recursive CTE (WITH RECURSIVE path_cte) walks from the given category up through parent_id to the root, then string_aggs the names (joined by " > ") and abbreviations (joined by"-") in root-to-leaf order, and upserts the result into category_paths (ON CONFLICT (category_id) DO UPDATE). Called every time a category is created or updated.

- **Child must match parent's module** — createCategory explicitly rejects a child category whose module doesn't equal its parent's module, preventing e.g. a FINANCE category being nested under a DOCUMENT category.
- **No true delete** — categories are only ever deactivated (is_active flag via updateCategory), never removed, preserving referential integrity for historical transactions that still point at old category IDs.
- **Tree vs. flat responses**: GET /categories builds and returns a nested tree by default (parent/children structure built in JS from the flat query result); pass ?flat=true to get the flat array instead (used by most modal category dropdowns, which then client-side-filter by module).
- Every transaction/contribution/transfer/grant/loan/investment record requires a non-null category_id (enforced by transactions.category_id NOT NULL at the schema level) — categories aren't optional metadata, they're mandatory classification.

5.4 API endpoints (cms/src/routes/categories.js, prefix /api/categories — note controller logic lives directly in the routes file; there is no separate categoriesController.js)

Method	Path	Permission	Description
GET	/	any authenticated user	List categories, optional ?module=FINANCE filter, optional ?flat=true
POST	/	CATEGORY_MANAGE	Create a category (validates module enum, non-empty name/abbreviation ≤20 chars, optional parent_id)
PATCH	/:id	CATEGORY_MANAGE	Update name/abbreviation/description/is_active; rebuilds path afterward

5.5 Frontend

There is **no standalone Categories page** — category management is one tab (CategoriesTab) inside cms-frontend/src/pages/settings/SettingsPage.jsx (tab key categories, label "Categories", TagIcon): - Module filter chips: FINANCE / DOCUMENT / EVENT / INVESTMENT / GENERAL — switching modules reloads the flat category list for that module. - "Add Category" button opens an inline form (not a modal) with: Parent Category select (indented by depth, only shown when adding — not editable once created), Category Name, Abbreviation (auto-uppercased, max 20 chars), Description. - Table lists every category for the selected module with visual indentation by depth (" └" connector), Full Path column, Abbreviation (monospace), Depth, and a per-row Edit icon that reopens

the same inline form pre-filled (parent cannot be changed on edit — only name/abbreviation/description). - All actions on this tab are gated by CATEGORY_MANAGE. - Elsewhere in the app, category selection is purely consumptive: every finance-related modal (Contribution, Expense, Inflow, Transfer) fetches categoriesAPI.getAll({ flat: true }) and client-side filters to c.module === 'FINANCE' to populate its category dropdown, displaying c.full_path || c.name.

5.6 Known issues (Categories)

- No delete endpoint exists (by design, per the "never truly delete" pattern above) — worth confirming this is understood as intentional rather than a missing feature.
- updateCategory's SQL uses COALESCE($1, name) etc. for name/abbreviation, but description and is_active are passed through directly without COALESCE — meaning an update call that omits description or explicitly sends null/undefined for it will actually **clear** the description. In practice the frontend always sends a description default and never sends is_active on edit, so this hasn't surfaced as a live bug, but it's an inconsistency worth a closer look.

Cross-module notes relevant to all five sections

- **Dual-post / atomic accounting pattern**: every money-moving operation in these modules (postTransaction calls, transfer's two legs + up to two bank-charge legs) runs inside a single withTransaction(…) wrapper — Postgres transaction — so a failure partway through rolls back everything, never leaving one leg posted without its pair.
- **Reference code format**: [MODULE_CODE]-[CATEGORY_ABBREV]-[YYYYMM]-[00001], generated by generateReference() (cms/src/services/referenceService.js) under a locked reference_sequences row per (module_code, category_abbrev, year_month) triple, guaranteeing no duplicate sequence numbers even under concurrent requests. Every reference also gets an unguessable 10-character public_id for safe external quoting/searching without revealing volume or ordering.
- **Audit logging**: every state-changing action in these five modules calls logAction(…) (services/auditService.js) with an ACTIONS.* constant and MODULES.FINANCE (or MODULES.SYSTEM for exchange rates/categories), inside the same DB transaction as the business change itself. #### 6. GRANTS

6.1 Purpose

Tracks external, non-repayable funding the club receives (or gives out) — grants from donors, government programs, or partner organizations — including multi-tranche disbursement schedules (e.g. a grant paid in three installments over a year) and, where applicable, refund/clawback tracking.

6.2 Data model

grants: id, reference_id, grantor_name, grant_purpose, total_amount, currency_id, category_id, status (PENDING/ACTIVE/COMPLETED/CANCELLED), start_date, end_date, notes, created_by, created_at.

grant_tranches: id, grant_id (FK), tranche_number, amount, expected_date, received_date (nullable — null means not yet received), account_id (which account it lands in), transaction_id (FK to the posted transaction once received), status (PENDING/RECEIVED/OVERDUE), notes.

Relationships: one grant has many tranches; each received tranche links 1:1 to a transactions row via postTransaction.

6.3 Business rules / key logic

- A grant is created with its total amount and expected tranche schedule up front; each tranche is a separate planned disbursement.
- Receiving a tranche (receiveTranche) posts a CREDIT transaction (inflow_type='GRANT') against the chosen account via postTransaction, stamps received_date, links transaction_id, and flips status to RECEIVED.
- If a tranche's expected_date passes without being received, it can be flagged/queried as OVERDUE (status transition, not an automatic cron — the scheduled "overdue checks" job mentioned in MONTHLY_REPORT_CRON/scheduled jobs table covers loans primarily; grants overdue tracking is more query-driven).
- Grant refunds (clawbacks) post a DEBIT transaction with inflow_type='GRANT_REFUND' against whichever account the funds are pulled from.
- Reference codes use module code GRANT, tranche receipts get their own reference under the receiving account's module code.

6.4 API endpoints (cms/src/routes/grants.js, prefix /api/grants)

Method	Path	Permission	Description
GET	/	FINANCE_VIEW_ALL	List grants, filter by status
GET	/:id	FINANCE_VIEW_ALL	Grant detail incl. tranche schedule
POST	/	FINANCE_TRANSACTION_CREATE (Treasurer/Asst. Treasurer roles)	Create grant + tranche schedule
PATCH	/:id	same	Edit grant details
POST	/:id/tranches/:trancheId/receive	same	Post receipt of a tranche

6.5 Frontend

cms-frontend/src/pages/grants/GrantsPage.jsx — list of grants with progress (tranches received / total), detail drawer showing the tranche schedule table with a "Mark Received" action per pending tranche (opens a small modal for account + received date).

6.6 Known issues (Grants)

- Overdue tranche detection is largely manual/query-based rather than an automated notification — worth confirming whether members expect a proactive reminder here (unlike loans, which do have automated overdue-check cron jobs).

---

7. LOANS

7.1 Purpose

Covers both directions of lending: loans the club **gives** to members (member borrows from the club, repays with interest) and loans the club **receives** from external sources (club borrows, repays a lender). Full interest calculation (daily/simple/compound), repayment scheduling, and status tracking live here.

7.2 Data model

loans_given: id, reference_id, borrower_id (FK users), principal_amount, interest_rate, rate_type (DAILY/SIMPLE/COMPOUND), interest_period, disbursement_date, expected_end_date, account_id (disbursing account), status (PENDING/ACTIVE/OVERDUE/COMPLETED/DEFAULTED), category_id, notes, created_by.

loan_repayment_schedule (for loans given): id, loan_id, installment_number, due_date, principal_due, interest_due, total_due, amount_paid, status (PENDING/PAID/OVERDUE/PARTIAL).

loan_repayments: id, loan_id, schedule_id (nullable — supports off-schedule/lump-sum payments), amount, principal_portion, interest_portion, payment_date, transaction_id, received_by.

loans_received: id, reference_id, lender_name, principal_amount, interest_rate, rate_type, disbursement_date, expected_end_date, account_id (receiving account), status, notes.

loan_received_repayments: mirrors loan_repayments but for money the club pays back out to an external lender — amount, principal_portion, interest_portion, payment_date, transaction_id.

7.3 Business rules / key logic — exact formulas from loanService.js

- **Daily rate derivation**: dailyRate = annualRate / 365 (simple daily accrual basis) — used as the base unit for DAILY rate_type loans.
- **Simple interest**: interest = principal × rate × (days / 365) — no compounding; the same principal accrues the same daily amount for the loan's life.

- **Compound interest**: applies the rate periodically (per the loan's interest_period, e.g. monthly) and re-bases the principal each period: newBalance = balance × (1 + periodRate), repeated per period elapsed.
- **Applicable rate resolution**: getApplicableRate() picks the correct rate for interest calculation as of any given date — supports the "loan-rate-amendment" feature (a loan's rate can be changed mid-term via a rate-history mechanism), meaning older accrued interest uses the old rate and newer accrual uses the amended rate. This directly relates to SYSTEM_DIAGNOSTIC_REPORT.md Critical Fix #6 ("no loan-rate-amendment UI") — the backend service supports rate history; the UI to actually change a rate was the missing piece, now fixed.
- **Schedule generation**: generateRepaymentSchedule() splits the total loan (principal + total interest) into N equal installments across the loan term, computing each installment's principal/interest split proportionally.
- **Repayment splitting**: when a payment comes in, splitRepayment() allocates it first to any outstanding interest, then to principal (standard amortization convention), updating the relevant schedule row's amount_paid and status.
- **Status determination**: determineLoanStatus() compares today's date against the schedule to classify a loan as ACTIVE (on track), OVERDUE (a scheduled installment's due date has passed without full payment), or COMPLETED (all installments PAID).
- **Disbursement** posts a DEBIT (loan given) or CREDIT (loan received) transaction via postTransaction, inflow_type = LOAN_DISBURSED or LOAN_RECEIVED respectively, subject to the same floor-limit/zero-balance checks as any other transaction.
- **Repayment** posts the inverse: LOAN_REPAYMENT_IN (member repaying the club) or LOAN_REPAYMENT_OUT (club repaying a lender), split further at the ledger-category level between principal and interest portions where reporting requires that distinction.
- **Overdue checks**: part of the scheduled-jobs suite (see Section 5, Infrastructure) — a cron job periodically re-evaluates loan schedules and flags/notifies overdue installments.

7.4 API endpoints (cms/src/routes/loans.js, prefix /api/loans)

Method	Path	Permission	Description
GET	/given	FINANCE_VIEW_ALL	List loans given
GET	/given/:id	FINANCE_VIEW_ALL	Detail + schedule + repayment history
POST	/given	Treasurer/Asst. Treasurer	Create + disburse a loan given
POST	/given/:id/repayments	Treasurer/Asst. Treasurer	Record a repayment
PATCH	/given/:id/rate	Treasurer/Asst. Treasurer	Amend the interest rate (rate-history

			preserving)
GET	/received	FINANCE_VIEW_ALL	List loans received
POST	/received	Treasurer/Asst. Treasurer	Create + receive a loan
POST	/received/:id/repayments	Treasurer/Asst. Treasurer	Record a repayment to the lender

7.5 Frontend

cms-frontend/src/pages/loans/LoansPage.jsx — tabbed Given/Received views; each loan row expands to a repayment schedule table with per-installment status coloring (green PAID, red OVERDUE, gray PENDING); "Record Repayment" modal supports both scheduled and off-schedule lump-sum payments; a rate-amendment action (Treasurer only) opens a small modal to set a new rate effective from a chosen date.

7.6 Known issues (Loans)

- Diagnostic report Critical Fix #6 (no loan-rate-amendment UI) is confirmed **fixed** — the rate-amendment modal now exists and calls the preserved rate-history backend logic.
- Compound interest period handling depends on interest_period being set consistently at loan creation; there's no UI validation preventing an inconsistent period/rate_type combination beyond basic required-field checks — worth a closer look if compound loans see real usage.

---

8. INVESTMENTS

8.1 Purpose

Tracks the club's investments in external projects/ventures — capital committed, returns received, and (at the schema level) project milestones — separate from loans (which expect a repayment schedule) since investment returns are typically irregular and tied to project performance rather than a fixed schedule.

8.2 Data model

investments: id, reference_id, project_name, description, amount_invested, currency_id, account_id (funding source), investment_date, expected_return, status (ACTIVE/COMPLETED/WRITTEN_OFF), category_id, notes, created_by.

investment_returns: id, investment_id, amount, return_date, transaction_id, notes.

investment_milestones (backend-only, per diagnostic report): id, investment_id, milestone_name, target_date, completed_date, status, notes.

8.3 Business rules / key logic

- Investing capital posts a DEBIT transaction (inflow_type='OTHER_INCOME'-adjacent; effectively an outflow categorized under Investment) from the funding account via postTransaction.
- Receiving a return posts a CREDIT transaction (inflow_type='INVESTMENT_RETURN') to a chosen account.
- Investment milestones exist as a full CRUD schema/controller layer but — per SYSTEM_DIAGNOSTIC_REPORT.md Moderate item #12 — have **no frontend UI**; they're only reachable via direct API calls today.
- Status is set manually (ACTIVE/COMPLETED/WRITTEN_OFF) rather than auto-derived from returns received vs. expected.

8.4 API endpoints (cms/src/routes/investments.js, prefix /api/investments)

Method	Path	Permission	Description
GET	/	FINANCE_VIEW_ALL	List investments
GET	/:id	FINANCE_VIEW_ALL	Detail + returns history
POST	/	Treasurer/Asst. Treasurer	Create + post capital outflow
POST	/:id/returns	Treasurer/Asst. Treasurer	Record a return
PATCH	/:id	Treasurer/Asst. Treasurer	Edit / change status
GET/POST/PATCH	/:id/milestones…	Treasurer/Asst. Treasurer	Milestone CRUD — **backend only, no frontend UI**

8.5 Frontend

cms-frontend/src/pages/investments/InvestmentsPage.jsx — investment cards/list with amount invested, total returns to date, simple ROI display, status badge; "Record Return" modal. No milestone UI (confirmed gap, matches diagnostic report).

8.6 Known issues (Investments)

- Confirmed per diagnostic report: investment_milestones is a fully built backend feature with **zero frontend surface** — either intentionally deferred or an oversight; worth a decision on whether to build the UI or remove the unused backend routes.

---

9. REQUISITIONS

9.1 Purpose

A general-purpose internal request/approval workflow — originally for expense requisitions (a member/officer requests funds for something), expanded to also carry acknowledgement-style requests: **Contribution Acknowledgement**, **Savings Deposit**, and (v1.26.0) **Side Fund Contribution** — the mechanism by which a regular Shareholder now records money they've already handed over, since direct posting of any of these was restricted to Treasurer/Assistant Treasurer. One workflow engine, four purposes.

9.2 Data model

requisitions: id, reference_id, requisition_type (EXPENSE / CONTRIBUTION_ACKNOWLEDGEMENT / SAVINGS_DEPOSIT / SIDE_FUND_CONTRIBUTION — the last widened onto the CHECK constraint in v1.26.0), requested_by, amount_requested, amount_approved, currency_id, category_id, description, purpose, contribution_date (the date the underlying payment/deposit was actually made — required for all three acknowledgement-style types), required_by_date, priority, status (PENDING/APPROVED/REJECTED), account_id (target account, for EXPENSE type only), transaction_id, reviewed_by, reviewed_at, review_notes, created_at.

For all three acknowledgement-style types, the requester submits amount + contribution_date + a free-text purpose/notes field describing how they paid; no month/period picker is offered even for the side fund type — the existing oldest-unpaid-first cascade in applySideFundPayment sorts out which period(s) a side fund acknowledgement covers.

9.3 Business rules / key logic

- Module code resolution for requisitions is done **manually** in the controller rather than via the shared resolveModuleCode() helper used elsewhere (flagged as a Minor item in the diagnostic report — inconsistent but not incorrect, since requisitions aren't tied to a specific account the way transactions are).
- Approving an EXPENSE requisition posts a DEBIT transaction via postTransaction against the chosen account (subject to the same floor-limit/zero-balance rules as any transaction) and flips status to APPROVED.
- Approving a CONTRIBUTION_ACKNOWLEDGEMENT requisition routes into creditShareholderContribution, posting a CREDIT against the Primary account and recomputing shareholding percentages (Section 4.2.3).
- Approving a SAVINGS_DEPOSIT requisition does **not** post any money movement itself — it hands off to createPendingFlexibleDeposit, which still needs a Treasurer/Assistant Treasurer's separate financial sign-off via the Savings module (Section 4.11) before it actually lands in the member's balance.
- **Approving a SIDE_FUND_CONTRIBUTION requisition (v1.26.0)** routes into creditSideFundContribution/applySideFundPayment (Section 4.10.3) — posts a CREDIT into the side fund's parent account and applies it to the requester's own dues oldest-unpaid-first, exactly like every other side fund payment entry point.

- Rejecting any type requires a reason, notifies the requester (bell + email), and leaves no transaction posted.
- Only Treasurer/Assistant Treasurer (or holders of the relevant finance permission) can approve/reject; any authenticated member can submit a requisition of any type for themselves. A pending requisition can be edited by its requester or the Treasurer/Assistant Treasurer before it's approved.

9.4 API endpoints (cms/src/routes/requisitions.js, prefix /api/requisitions)

Method	Path	Permission	Description
GET	/	scoped — own requisitions for regular members, all for Treasurer/Directors /Admin	List
GET	/:id	same scoping	Detail
POST	/	any authenticated member (not Auditor)	Submit a new requisition (either type)
POST	/:id/approve	Treasurer/Asst. Treasurer	Approve → posts the transaction
POST	/:id/reject	Treasurer/Asst. Treasurer	Reject with reason

9.5 Frontend

cms-frontend/src/pages/requisitions/RequisitionsPage.jsx — "New Requisition" button opens a type-toggle modal with four buttons (Money Request / Acknowledge My Contribution / Request Savings Deposit / Acknowledge Side Fund Payment, v1.26.0 adding the last); list view with status badges and a type badge per acknowledgement type; Treasurer/Director view shows Approve/Reject actions inline, with the approval modal skipping account selection for all three acknowledgement types (money always resolves to the relevant module's own account automatically).

9.6 Known issues (Requisitions)

- Confirmed Minor item from diagnostic report: manual module-code resolution in the requisitions controller instead of reusing resolveModuleCode() — cosmetic/consistency issue only, not a functional bug.

Cross-module notes (Grants, Loans, Investments, Requisitions)

- All four modules post their real money movements through the same postTransaction() choke point as every other module (Section 4.2), so the "never negative," floor-limit, and audit-logging guarantees apply uniformly here too.

- All four use the shared reference-code system (GRANT, loan module codes, INV, requisition module codes) and the shared category system (module=FINANCE or INVESTMENT).
- Notifications for approvals/rejections/receipts in all four modules go through the shared notify()/notifyMany() service — bell + best-effort email, non-blocking, sent only after the DB transaction commits. #### 10. SIDE FUND

Rewritten in full for v1.26.0 — v1.25.0 still allowed an unattributed lump sum into the envelope ("direct/batch inflow"), which broke per-member overdue tracking, since not every shilling in the pool could be traced back to a specific member's obligation. v1.26.0 closes that gap: every inflow is now tied to a member's own dues, no exceptions.

10.1 Purpose

A required-for-all-shareholders monthly contribution scheme — every active shareholder owes a fixed amount each calendar month into a shared pool ("the side fund"), used for day-to-day club expenses. It is not its own bank account: the pool is an "envelope" balance layered inside one existing Primary or Secondary account (its parent_account_id), so every contribution and expense is a completely ordinary transaction on that real account, dual-posted alongside an envelope-balance update so the two numbers can never drift apart. The system tracks, per member and per month, whether that due was paid, partially paid, still pending, or defaulted — supports a different due amount for individual members, lets an overpayment carry forward to future months instead of being capped, and surfaces the member's side fund standing on their own Profile page and dashboard.

v1.26.0 — strictly per-member attribution. Every contribution to the fund, however it enters the system, is now always tied to a specific member's own dues — there is no longer any way to add an unattributed lump sum. This is what makes accurate overdue tracking possible: the fund is one pool, but each member's contribution toward it is tracked and chased individually.

10.2 Data model

side_fund_config (singleton row, id always 1): is_active, parent_account_id (FK accounts), currency_id, monthly_amount (the company-wide default due), current_balance (the envelope balance), updated_by, updated_at.

side_fund_dues: id, user_id, period ('YYYY-MM'), amount_due, amount_paid, status (PENDING / PARTIAL / PAID / DEFAULTED), due_date (v1.26.0 — the last day of this due's own period month, stored explicitly rather than recomputed from period each time, so overdue amounts can be reported per member precisely and consistently), transaction_id (the contribution transaction, once paid with real money), paid_date, paid_from_credit (v1.25.0 — TRUE if this due was settled by auto-drawing down banked credit rather than a new payment), recorded_by, notes. One row per member per month — UNIQUE (user_id, period).

side_fund_expenses: id, reference_id, transaction_id, amount, description, expense_date, recorded_by. Links a normal EXPENSE transaction on the parent account back to the side

fund so the envelope balance can be decremented and the spend shows up in the fund's own history.

side_fund_member_overrides (v1.25.0): user_id (PK), monthly_amount, set_by, set_at. One row per member who pays a different fixed amount than the company default (e.g. a hardship reduction) — no row means "on the company default." Only affects dues generated from the point it's set onward, never past periods.

side_fund_member_credit (v1.25.0): user_id (PK), credit_balance, updated_at. A running balance of money a member has paid ahead of what they owed — banked here rather than lost, and automatically applied to their own future dues as they're generated.

side_fund_credit_ledger (v1.25.0): id, user_id, delta (positive = banked from an overpayment, negative = applied to a specific due), reason, related_due_id, created_at. An auditable trail of every credit movement, separate from side_fund_dues itself.

10.3 Business rules / key logic

- **Monthly due generation** (jobs/scheduler.js, 00:15 on the 1st of each month): creates one side_fund_dues row per active shareholder for the new month if the fund is active, using that member's side_fund_member_overrides amount if one is set, otherwise the company-wide side_fund_config.monthly_amount. Also computes and stores due_date (v1.26.0) as the last day of that period's own month. Safe to re-run (ON CONFLICT (user_id, period) DO NOTHING).
- **Default marking** (00:20 on the 1st, right after generation): any due from the month that just ended still PENDING or PARTIAL is marked DEFAULTED — a record that it wasn't paid on time, not a block on paying it later. Every payment path accepts a payment against a due in any unpaid status.
- **Dual-posting**: every real money movement — a due payment or a side-fund expense — posts one completely ordinary transaction on the parent account (visible in that account's own Transactions ledger like any other entry) *and*, in the same DB transaction, adjusts side_fund_config.current_balance by the same amount, so the ledger and the envelope total can never disagree.
- **Shared oldest-first payment-application service (v1.26.0)** — services/sideFundService.js's applySideFundPayment(client, { userId, amount, transactionId, referenceCode, paidDate, recordedBy }) is now the single choke point every side fund payment path funnels through, regardless of how the money physically arrived. It always applies strictly oldest-unpaid-period-first (PENDING, PARTIAL, and DEFAULTED all count), settling as much of each due as the payment covers before moving to the next; whatever's left after every existing due is fully paid gets banked into side_fund_member_credit, with a side_fund_credit_ledger entry recording why. The caller is responsible for posting the actual ledger CREDIT and incrementing side_fund_config.current_balance — this function only ever touches side_fund_dues/side_fund_member_credit/side_fund_credit_ledger. Four entry points call it:
    1. sideFundController.recordDuePayment — Treasurer records one member's payment (PATCH /dues/:id/pay, the due row clicked just identifies *who's*

paying; the cascade still applies across that member's whole standing, not only the clicked row).

2. sideFundController.bulkPayDues (v1.26.0, below) — mark-all-as-paid batch entry.
3. transactionsController.recordContribution (v1.26.0, below) — the side fund slice of a Transactions contribution.
4. requisitionsController.approveRequisition's SIDE_FUND_CONTRIBUTION branch (v1.26.0, below).

- **Bulk pay-all-dues (v1.26.0)** — PATCH /dues/bulk-pay, for the common case where most/all members paid on time in one sitting. Accepts { category_id, paid_date, payments: [{ user_id, amount }] }; posts **one** pooled ledger CREDIT transaction for the sum of every payment (the single real deposit/collection event, visible once in the account's own ledger), then loops applySideFundPayment once per member against that shared transactionId — so the pool sees one transaction, while side_fund_dues/side_fund_credit_ledger still track every member's own contribution individually. Each member's amount is editable in the request, not locked to "exactly what's owed" — a treasurer can tick someone off the batch or adjust an amount before submitting.
- **Transactions contribution side fund split (v1.26.0)** — POST /transactions/contributions accepts an optional side_fund_amount, sliced out of the total amount: the side fund portion is credited to that same member's own dues via applySideFundPayment (its own separate ledger transaction, into the side fund's parent account — kept distinct from the capital-contribution transaction so the two envelopes never mix), and only the remainder (amount − side_fund_amount) is recorded as the capital contribution via creditShareholderContribution. If the remainder is zero, no contribution row is created at all — the whole amount was a side fund payment.
- **Requisitions SIDE_FUND_CONTRIBUTION type (v1.26.0)** — a member can request/acknowledge a side fund payment already made, the same way they already could for a capital contribution or savings deposit (Section 4.9). Just an amount and a date — no month picker; approval routes into applySideFundPayment, and the existing oldest-unpaid-first cascade sorts out which period(s) it covers.
- **Per-member overdue summary (v1.26.0)** — GET /overdue/me (own) and GET /overdue (SIDE_FUND_VIEW, every member) return overdue_count/overdue_amount computed from dues that are DEFAULTED, or still PENDING/PARTIAL past their own due_date. Deliberately a separate read endpoint rather than a change to getMyDues/getAllDues's existing response shape, so nothing already consuming those endpoints breaks.
- **No unattributed inflow (v1.26.0)** — the v1.25.0 "Add Funds Directly" feature (recordDirectInflow, POST /side-fund/inflows) has been removed entirely, along with its frontend button/modal. There is now no way to add money to the envelope that isn't tied to a specific member's own dues — inflow_type='SIDE_FUND_DIRECT_IN' remains in the transactions.inflow_type CHECK constraint only so historic rows from before this version stay valid; no new transaction can be posted with it.

- **Automatic credit draw-down**: immediately after the monthly due-generation job creates a new due for a member who holds banked credit, it draws that credit down against the brand-new due right away (paid_from_credit = TRUE, status PAID or PARTIAL depending on how much credit covers) — this is what "the balance is distributed to cater for the following months" means in practice. No new ledger transaction is posted for this step, since the money already moved into the account back when the credit was originally banked; only the side_fund_dues row and a side_fund_credit_ledger entry record the reallocation.
- **Per-member overrides** are opt-in and forward-only: setting or clearing one only changes dues generated from that point on, exactly like a change to the company-wide default.
- Changing the parent account is only allowed while the envelope balance is zero, so money is never silently "moved" without an actual transaction.
- All Side Fund date fields that represent money actually moving (expense_date, a due's paid_date, a bulk-pay/requisition paid_date/contribution_date) reject future dates — see Section 26.7.

10.4 API endpoints (cms/src/routes/sideFund.js, prefix /api/side-fund)

Method	Path	Permission	Description
GET	/settings	any authenticated user	Fund status, envelope balance, monthly default
PATCH	/settings	SIDE_FUND_MANAGE	Activate/deactivate, set parent account, set company default amount
GET	/dues/me	any authenticated user	Own due history, most recent period first
GET	/dues	SIDE_FUND_VIEW	All members' dues
PATCH	/dues/:id/pay	SIDE_FUND_CONTRIBUTION_RECORD	Record one member's payment — cascades oldest-unpaid-first, banks remainder as credit
PATCH	/dues/bulk-pay	SIDE_FUND_CONTRIBUTION_RECORD	v1.26.0 — mark several/all members' due as paid in one pooled transaction, per-row amounts editable
GET	/overdue/me	any authenticated	v1.26.0 — own

		user	overdue count/amount
GET	/overdue	SIDE_FUND_VIEW	v1.26.0 — every member currently overdue
GET	/expenses	SIDE_FUND_VIEW	Spending history
POST	/expenses	SIDE_FUND_EXPENSE_RECORD	Record an expense drawn from the fund
GET	/overrides	SIDE_FUND_MANAGE	Every active shareholder + their override amount, if any (v1.25.0)
PUT	/overrides/:userId	SIDE_FUND_MANAGE	Set a member's custom monthly amount (v1.25.0)
DELETE	/overrides/:userId	SIDE_FUND_MANAGE	Clear a member's override — back to the company default (v1.25.0)
GET	/credit/me	any authenticated user	Own banked credit balance (v1.25.0)
GET	/credit	SIDE_FUND_VIEW	Every member currently holding banked credit (v1.25.0)

POST /inflows (direct/batch top-up) no longer exists — removed in v1.26.0.

10.5 Frontend

cms-frontend/src/pages/sideFund/SideFundPage.jsx — a standalone page (/side-fund) with a tabbed layout: **My Dues** (own due history, plus a red "overdue" banner when overdue_amount > 0), **All Members** (a "Record Payment" action per outstanding due, plus a red overdue summary panel listing every member currently overdue, SIDE_FUND_VIEW/SIDE_FUND_CONTRIBUTION_RECORD), **Spending History**, **Member Credit** (every member currently holding banked credit, SIDE_FUND_VIEW), and **Member Overrides** (every shareholder with an inline editable override amount, SIDE_FUND_MANAGE). Summary cards show the envelope balance, the company-wide monthly due, and which account it lives inside; a member with banked credit sees a green "You have banked side fund credit" callout. Recording a payment that overpays shows a result screen listing every period it settled and any amount banked as credit, rather than closing silently. Settings (activate/deactivate, parent account, default amount) are edited

via a modal within this same page, not a separate Settings tab. The v1.25.0 “Add Funds Directly” button/modal is gone (v1.26.0).

Bulk Pay Dues modal (v1.26.0) — a “Bulk Pay Dues” button (SIDE_FUND_CONTRIBUTION_RECORD) opens a period picker plus a checklist of every member with an outstanding due for that period, pre-ticked with their full outstanding amount but with each row’s amount directly editable and each row individually untickable, before one category/date and a single submit records the whole batch.

Elsewhere in the app (v1.26.0): - **Transactions** (TransactionsPage.jsx) — the Record Contribution modal gains an optional “Side Fund Portion” field (shown only while the fund is active) that slices an amount out of the total before it’s submitted; the contribution amount that will actually be recorded is shown live underneath. - **Requisitions** (RequisitionsPage.jsx) — a third request-type button, “Acknowledge Side Fund Payment,” alongside Money Request and Acknowledge My Contribution, with the same amount+date (no month picker) pattern as Contribution Acknowledgement. - **Dashboard** — the Treasurer/Admin dashboard’s existing Side Fund card (DashboardPage.jsx, fed by accountsController’s side_fund_allocation) was already correct and unchanged. The plain Shareholder dashboard (ShareholderDashboard.jsx) previously had no side fund visibility at all — it now shows a “My Side Fund” card (own overdue amount if any, else banked credit if any, else “Up to date”), linking to /side-fund. - **Profile** (ProfilePage.jsx, Section 23) — unchanged from v1.25.0: current-period due status and banked credit still shown alongside the Shareholding summary.

10.6 Known issues (Side Fund)

- None currently open.

---

11. SAVINGS

11.1 Purpose

Handles the personal savings pool — members deposit into the single, dedicated SAVINGS account (Section 4.1), and later “handouts” pay members back out of it, e.g. at year-end. Deliberately separate from the Primary account so club operating funds and member personal savings are never commingled.

11.2 Data model

savings_deposits: id, reference_id, user_id, amount, currency_id, deposit_date, category_id, notes, transaction_id, created_by.

savings_handouts: id, reference_id, user_id, amount, handout_date, category_id, notes, transaction_id, created_by, approved_by (if an approval step applies).

11.3 Business rules / key logic

- Deposits always post a CREDIT to the single SAVINGS account (inflow_type='SAVINGS_DEPOSIT_IN') via postTransaction; the SAVINGS account has no floor limit (Section 4.1.3), so any deposit is always accepted so long as it's a positive amount.
- Handouts post a DEBIT (inflow_type='SAVINGS_HANDOUT_OUT') — subject to the ordinary "never go negative" check (a member can't be handed out more than the SAVINGS account currently holds in aggregate, since it's a single pooled account, not per-member sub-balances at the ledger level — per-member totals are tracked via the savings_deposits/savings_handouts rows themselves, summed).
- **Fixed this session's diagnostic-report predecessor**: the previously reported crash where a savings handout could be posted with category_id: null (violating transactions.category_id NOT NULL) is marked **FIXED** in SYSTEM_DIAGNOSTIC_REPORT.md — the handout flow now requires/derives a valid category before calling postTransaction.
- A member's savings balance is a derived figure: sum of their savings_deposits minus sum of their savings_handouts, not a stored running balance.
- **Fixed (v1.26.1) — a handout could be paid from any account, not just Savings.** createSavingsHandout used to trust an account_id sent from the request body, checking only that it was *some* active account — so a handout could be recorded against the Primary account or any Secondary/operational account, money that was never actually sitting in the member's savings. It's now resolved server-side via the same getSavingsAccount(client) helper every other savings entry point already used (deposits, fixed-term withdrawals) — a handout is always paid out of the one dedicated SAVINGS account, full stop, and the client can no longer choose otherwise. The "Pay From Account" dropdown was removed from RecordHandoutModal (SavingsPage.jsx) to match — there's nothing to choose. POST /savings/handouts no longer accepts account_id at all.

11.4 API endpoints (cms/src/routes/savings.js, prefix /api/savings)

Method	Path	Permission	Description
GET	/deposits	FINANCE_VIEW_ALL (or own records for regular members)	List deposits
POST	/deposits	Treasurer/Asst. Treasurer	Record a deposit
GET	/handouts	same scoping	List handouts
POST	/handouts	Treasurer/Asst. Treasurer	Record a handout — always paid from the SAVINGS account, resolved server-side (v1.26.1); no

			account_id in the request
GET	/balance/:userId	scoped	A member's derived savings balance

11.5 Frontend

cms-frontend/src/pages/savings/SavingsPage.jsx — per-member savings summary cards, deposit/handout history table, "Record Deposit"/"Record Handout" modals (Treasurer roles only); a regular Shareholder sees only their own savings history and balance.

11.6 Known issues (Savings)

- The category_id null-crash fix (Critical Fix, per diagnostic report) should be spot-checked again after any future refactor of the handout flow, since it was a real production-crash bug once already.
- Sections 11.1/11.2 above describe an earlier, simpler design (a flat savings_deposits table, immediate handouts) that predates the current flexible/fixed-term split, the member-confirms-before-it-posts handout workflow (PENDING_CONFIRMATION → confirmSavingsHandout/rejectSavingsHandout), savings_balances/member_savings, and Savings Pool inflows — all implemented in code but not yet reflected here. Flagged for a fuller rewrite of this section, same treatment Section 4.10 (Side Fund) already got.

---

12. DIVIDENDS

12.1 Purpose

Distributes profit to shareholders proportional to their shareholding percentage (from shareholding_registry, Section 4.14). A Treasurer declares a total dividend pool against a company account; every current shareholder's share is calculated automatically and, once approved, credited directly into that shareholder's own internal Savings balance (v1.22.0) — real, withdrawable money via the existing Savings handout flow, not just a calculated record for someone to action outside the system.

12.2 Data model

dividends: id, reference_id, account_id (the declaring/source account), currency_id, category_id, total_amount, period_label, declaration_date, payment_date, status (PENDING/PAID/CANCELLED), notes, created_by, approved_by, approved_at. v1.22.0 adds transaction_id (the debit leg from the source account), savings_transaction_id (the credit leg into the Savings account), and exchange_rate (the rate used to convert into the Savings account's currency — 1 if they already match).

dividend_distributions: one row per shareholder per dividend — id, dividend_id, user_id, shares_at_time, percentage_at_time (both snapshotted at declaration, so a later

shareholding change never rewrites an already-declared dividend), amount (the declared share, in the dividend's own currency), status (PENDING/PAID), transaction_id, paid_at. v1.22.0 adds credited_amount (what was actually added to savings_balances, in the Savings account's currency) and exchange_rate (copied from the parent dividend, kept per row for a self-contained audit trail).

12.3 Business rules / key logic

Declaration (declareDividend) snapshots every current shareholder's percentage from shareholding_registry and pre-computes each member's proportional share of total_amount, inserting one dividend_distributions row per shareholder (status PENDING). Declaration fails if shareholding percentages don't sum to within 0.01 of 100% — a data-integrity guard, not a business rule to work around.

Editing (editDividend, PATCH, PENDING only) — the declarer or any Treasurer can adjust the account, category, amount, period, date, or notes before approval. If total_amount changes, every distribution is deleted and recalculated from scratch against *today's* shareholding percentages — safe because nothing has been paid out yet at this stage.

Approval (approveDividend) — the money-moving step, and the core of the v1.22.0 change. Posts two ledger legs in the same transaction: 1. **DEBIT** the declaring account for the full total_amount (inflow_type = 'DIVIDEND_OUT') — the company's side of "the dividend was paid". 2. **CREDIT** the single Savings account (Section 4.11) with that same total, converted into the Savings account's currency (inflow_type = 'DIVIDEND_SAVINGS_IN').

Then, for every shareholder: their declared share (amount) is converted by the same rate and added to their own savings_balances.principal_balance (creating the row via getOrCreateSavingsBalance if they've never saved before), the distribution row is marked PAID with its credited_amount/exchange_rate recorded, and a best-effort notification is sent ("Dividend credited to your savings").

Currency conversion is manual, not automatic. The system has exactly one Savings account, so every shareholder's balance shares its currency. If the dividend's declaring account uses a different currency, the Treasurer must supply exchange_rate when approving — the same manual-entry precedent already used for cross-currency Transfers (Section 4.3), and for the same reason: currency_exchange_rates (Section 4.4) is explicitly documented as display-only and is never used to calculate real money movements. If the currencies already match, no rate is needed and 1 is used automatically. getAllDividends/getDividendById both expose a needs_exchange_rate flag (comparing the dividend's currency against the live Savings account currency) so the frontend can decide whether to ask for a rate before showing the approve confirmation.

Approval requires the Savings account to already exist (Section 4.11's "set up once" step) — it fails with a clear message otherwise, the same guard every other Savings-module action uses.

Fixed (v1.26.2) — approving a same-currency dividend crashed the backend. When needs_exchange_rate is false, there's nothing for the Treasurer to fill in, so the frontend called dividendsAPI.approve(id, undefined). axios sends no request body at all for undefined — no body, no Content-Type header — so Express's JSON body parser never runs, and req.body arrives as undefined server-side rather than {}. const { exchange_rate } = req.body then throws TypeError: Cannot destructure property 'exchange_rate' of 'req.body' as it is undefined, an unhandled error surfaced to the user as the generic "Something went wrong." Fixed on both sides: the frontend now always sends {} when there's no rate to send, and approveDividend now defaults with req.body || {} so any future caller sending no body at all degrades safely instead of crashing.

12.4 API endpoints (cms/src/routes/dividends.js, prefix /api/dividends)

Method	Path	Who	Description
GET	/	FINANCE_VIEW_ALL	List dividends (includes needs_exchange_rate per row)
GET	/:id	FINANCE_VIEW_ALL	Detail + per-shareholder distribution breakdown
POST	/	Treasurer	Declare a dividend (snapshots percentages)
PATCH	/:id	Declarer or Treasurer	Edit a still-PENDING dividend
POST	/:id/approve	Treasurer	Approve — posts both legs, credits every shareholder's Savings balance. Body: exchange_rate (required only if needs_exchange_rate is true)

12.5 Frontend

cms-frontend/src/pages/dividends/DividendsPage.jsx — dividend list with a "Rate needed to credit savings" hint on PENDING rows where currencies differ; declare/edit modal; a dedicated Approve modal that shows the declared total, the amount that will land in Savings, and (when currencies differ) the exchange-rate input with a live converted-total preview; a detail modal showing each shareholder's declared share and, once paid, their actual credited amount in the Savings account's currency.

12.6 Known issues (Dividends)

- Since distribution amounts are snapshotted at declaration time (and recalculated only if the total is edited pre-approval), any shareholding correction discovered *after* approval requires a manual/administrative fix — there's no automatic re-snapshot or recalculation mechanism once money has moved. This mirrors the same deliberate design already used for shareholder contributions and reversals elsewhere in the system.
- This module has one Savings account and therefore one shared currency for every shareholder's credited amount — there is no per-shareholder currency preference. If that ever needs to change, the conversion logic in approveDividend (currently one rate applied uniformly) would need to become per-shareholder.

---

13. SHARE CERTIFICATES

13.1 Purpose

Generates and emails an official, styled PDF share certificate to a shareholder — the one document in the whole system produced via server-side **Puppeteer** (a real headless Chrome instance rendering an HTML template to PDF) rather than the client-side print-to-PDF pattern (exportUtils.js) used everywhere else, because it needs to be emailed as an attachment (not just printed locally) and needs pixel-perfect, print-quality rendering (seals, signatures, decorative borders).

13.2 Data model

share_certificates: id, reference_id, user_id, certificate_number (distinct from the general reference-code system — a dedicated human-facing certificate numbering scheme), shares_held_at_issue, percentage_at_issue, issue_date, issued_by, pdf_generated (boolean), emailed_at (nullable).

13.3 Business rules / key logic

- Certificate generation snapshots the shareholder's shares_held/percentage from shareholding_registry at issue time (same "snapshot, don't live-link" pattern as dividends).
- **Puppeteer pipeline**: an HTML template (populated with company branding — logo/colors from company_settings, Section 4.20 — plus the shareholder's name, shares, percentage, certificate number, issue date, and an official-looking border/seal design) is rendered by a headless Chrome instance launched server-side into a PDF buffer, which is then both saved (for later re-download) and attached to an outgoing email via the shared email service.
- This is the **only** place in the codebase that uses Puppeteer — everywhere else that produces a "PDF" actually relies on the browser's native print-to-PDF via window.print() triggered from a formatted HTML preview (exportUtils.js's printDocument/previewDocument pattern, Section 4.19).

- Re-issuing/regenerating a certificate for the same shareholder creates a new share_certificates row (new certificate number) rather than overwriting — preserves a full historical record of every certificate ever issued.

13.4 API endpoints (cms/src/routes/certificates.js, prefix /api/certificates)

Method	Path	Permission	Description
GET	/	FINANCE_VIEW_ALL (or own certificates for regular members)	List certificates
POST	/generate/:userId	Secretary/Admin	Generate + email a certificate (runs the Puppeteer pipeline)
GET	/:id/download	scoped	Re-download a previously generated PDF

13.5 Frontend

cms-frontend/src/pages/certificates/CertificatesPage.jsx (or embedded in the Shareholding view) — "Generate Certificate" action per shareholder (Secretary/Admin), certificate history list per member with a download icon; the generation action shows a loading state since Puppeteer rendering + emailing takes noticeably longer than a normal request.

13.6 Known issues (Share Certificates)

- Puppeteer being the sole heavyweight server-side rendering dependency in an otherwise lightweight Node/Express app is worth flagging operationally: it increases memory footprint and cold-start time on the starter Render plan, and is the most likely candidate if the backend service ever needs a larger plan than starter as usage grows (cross-reference: Section 8, Infrastructure, and the earlier scaling-advice discussion about resource growth).

---

14. SHAREHOLDING

14.1 Purpose

The authoritative registry of who owns what percentage of the club — recalculated fresh from actual contribution history every time a contribution is posted (Section 4.2.3), rather than manually maintained, so it can never silently drift out of sync with the real contribution ledger.

14.2 Data model

shareholding_registry: id, user_id (unique — one row per shareholder), shares_held, percentage (NUMERIC, 4 decimal places), last_recalculated_at.

14.3 Business rules / key logic

- **Always derived, never directly edited**: as covered in Section 4.2.3, every approved contribution triggers a full recompute across *all* shareholders (sum each member's approved shareholder_contributions, divide by the grand total, write shares_held/percentage) — there is no manual "set a member's percentage" admin action, by design, to prevent the registry from ever disagreeing with the underlying contribution history.
- A brand-new contributor gets an auto-created shareholding_registry row (starting at 0 shares) on their very first contribution.
- percentage is stored to 4 decimal places for precision in dividend/certificate calculations that multiply it against large pool amounts.
- The GET /api/users/shareholding endpoint (Section 6, Users) exposes this company-wide registry — this was one of the two endpoints patched with blockAuditor this session (Section 3, RBAC Summary) after being found open to any authenticated user, including Auditor, by original "convenience" design.

14.4 API endpoints

Shareholding itself has no dedicated route file — it's read via GET /api/users/shareholding (Section 6, Users module) and updated only as a side effect of creditShareholderContribution (Transactions module, Section 4.2). There is no direct write endpoint.

14.5 Frontend

Surfaced in multiple places rather than one dedicated page: a "Shareholding" tab/section (likely under Users or a Reports view) listing every shareholder with their current shares/percentage, sortable; individual percentage figures also feed the Dividends and Share Certificates modules' snapshot logic.

14.6 Known issues (Shareholding)

- The full-registry recompute on every single contribution (rather than an incremental per-user update) is O(number of shareholders) work on every contribution post — fine at the club's current membership scale, but worth keeping in mind if membership grows into the hundreds+ (cross-reference the earlier scaling-advice conversation about database/computation growth).

Cross-module notes (Side Fund, Savings, Dividends, Certificates, Shareholding)

- **Permission constants used across this cluster** (for quick reference): FINANCE_VIEW_ALL (read most of this cluster), FINANCE_TRANSACTION_CREATE/role-based Treasurer checks (deposits, handouts, side fund moves), SYSTEM_CONFIG (side fund creation), Secretary/Admin roles specifically for certificate generation (a Secretary-specific duty, distinct from Treasurer's financial duties).

- All five modules snapshot-and-post through the same ledger discipline as every other financial module — no exceptions, no bypass of postTransaction.
- Certificates are the one place email delivery is integral to the feature (not just a notification side-effect) — if GMAIL_USER/GMAIL_APP_PASSWORD env vars are ever misconfigured (Section 8, Infrastructure), certificate generation would still produce/save the PDF but the emailing step would fail (best-effort pattern, consistent with notify()'s non-blocking design elsewhere). #### 15. EVENTS

15.1 Purpose

The club's shared calendar — meetings, deadlines, and general events — visible to members (subject to the Auditor gating fixed this session) with upcoming-event surfacing on the TopBar/dashboard.

15.2 Data model

events: id, reference_id, title, description, event_type (MEETING/DEADLINE/GENERAL), event_date, event_time, location, category_id, created_by, created_at, is_cancelled.

event_attendees (if tracked): event_id, user_id, rsvp_status.

15.3 Business rules / key logic

- GET /events/upcoming (used by the TopBar) returns events within the next N days (default 7), sorted ascending — this is the endpoint that was unconditionally called for every authenticated user, including Auditor, before this session's blockAuditor fix (Section 3, RBAC Summary; Section 4.1 note on TopBar gating).
- Events tied to a MEETING type often pair with the Documents module's generated Meeting Agenda / Meeting Minutes templates (Section 4.17).
- Cancelling an event sets is_cancelled = TRUE rather than deleting, preserving history.

15.4 API endpoints (cms/src/routes/events.js, prefix /api/events)

Method
GET
GET
POST
PATCH

15.5 Frontend

cms-frontend/src/pages/events/EventsPage.jsx — calendar/list view, "New Event" modal (Secretary/Admin), event detail with a link to generate a Meeting Agenda/Minutes document if the event is a MEETING type. Upcoming events also surface as a TopBar dropdown widget (now hidden for Auditor).

15.6 Known issues (Events)

- The TopBar visibility leak (Section 3) is the main historical issue here — confirmed fixed this session via blockAuditor on events.js plus the frontend isAuditor gate in TopBar.jsx.

---

16. DOCUMENTS

16.1 Purpose

A general file-storage/document-record module (meeting minutes uploads, resolutions, contracts, miscellaneous attachments) distinct from the print-generated templates covered in Section 4.19 — this module is about storing and categorizing *uploaded* files, not producing new ones from data.

16.2 Data model

documents: id, reference_id, title, description, file_path, file_size, mime_type, category_id (module='DOCUMENT'), uploaded_by, upload_date, is_confidential (gates visibility to certain roles).

16.3 Business rules / key logic

- Files are stored on local disk under UPLOAD_DIR (Section 8, Infrastructure) — flagged in the earlier scaling-advice conversation as **not persistent** across Render redeploys/restarts on the current hosting setup, a known operational risk worth revisiting as the system scales (cross-reference DEPLOYMENT_GUIDE.md's uploads warning).
- is_confidential documents are filtered out of listings for roles below a certain access level (exact gating varies by document type/purpose).
- Upload uses the shared middleware/upload.js (multer-based), with MAX_FILE_SIZE_MB enforced (Section 8).

16.4 API endpoints (cms/src/routes/documents.js, prefix /api/documents)

Method
GET
POST
GET
DELETE

16.5 Frontend

cms-frontend/src/pages/documents/DocumentsPage.jsx — category-filterable file browser, upload modal (drag-and-drop), confidential-document lock icon, download action.

16.6 Known issues (Documents)

- Local-disk, non-persistent upload storage is the headline risk here (same issue raised in the earlier scaling conversation) — a durable object-storage solution (e.g. S3-compatible) is the natural next step once the club depends on this module for anything irreplaceable.

---

17. REPORTS

17.1 Purpose

Generates the club's financial and membership reports — monthly summaries (auto-emailed via the MONTHLY_REPORT_CRON scheduled job, Section 8), and on-demand reports covering contributions, loans, dividends, and overall financial position.

17.2 Data model

Reports are largely **computed on demand** from existing tables (transactions, shareholder_contributions, loans_given, etc.) rather than stored — no dedicated reports table; any persistence is limited to a log of "report sent" events if applicable.

17.3 Business rules / key logic

- The monthly report cron (default 0 8 1 * * — 8am on the 1st of each month, Section 8) compiles a financial summary and emails it to configured recipients (likely Directors/Treasurer/Admin) via the shared email service.
- On-demand reports in the frontend use the same exportUtils.js print-to-PDF pattern as other documents (Section 4.19) rather than a server-generated file, for most report types.
- **Fiscal quarter labelling (v1.25.0)** — the General, Personal, and "My" report generators (getGeneralReport/getIndividualReport/getMyReport) each attach report.fiscal_quarter via fiscalService.getQuarterForDate() (Section 19.6), looked up against the last day of the reported month. Purely a label shown alongside the calendar period — it never changes any of the report's actual figures, which stay strictly calendar-month based as they always have.

17.4 API endpoints (cms/src/routes/reports.js, prefix /api/reports)

Method	Path	Permission	Description
GET	/financial-summary	FINANCE_VIEW_ALL	Aggregate financial position over a date range
GET	/contributions	FINANCE_VIEW_ALL	Contribution report
GET	/loans	FINANCE_VIEW_ALL	Loan portfolio report
GET	/dividends	FINANCE_VIEW_ALL	Dividend distribution report

17.5 Frontend

cms-frontend/src/pages/reports/ReportsPage.jsx — report-type selector, date-range filter, on-screen summary tables/charts, "Export" button (print-to-PDF via exportUtils.js).

17.6 Known issues (Reports)

- No items specific to this module surfaced in the diagnostic report beyond the general "raw toFixed() usage" minor formatting note that applies broadly across the app's currency-display code (Section 9, Known Issues Registry).

---

18. NOTIFICATIONS

18.1 Purpose

The bell-icon notification system — every significant event in the app (approvals needed, approvals completed, rejections, contributions recorded, documents ready, audit engagement updates) can generate an in-app notification and, optionally, a paired email, through one consistent entry point.

18.2 Data model

notifications: id, user_id, title, message, notification_type, is_read, related_record_type, related_record_id, created_at.

18.3 Business rules / key logic

- **notify(userId, {...}) / notifyMany([userIds], {...})** (services/notificationService.js) is the single call site every module uses — always **best-effort and non-blocking**: called only *after* the real business transaction has already committed, and any failure (e.g. email send failure) is caught and logged, never allowed to roll back or fail the parent operation.
- This pattern is why, throughout every module section above, "notifies X" is described as happening after a DB transaction, never as part of it.
- The bell dropdown (TopBar) polls/fetches on route change; the Auditor-gating fix this session (Section 3, TopBar note) means Auditor's bell now only ever shows audit-submission-related notifications, never company-wide approval/event notifications.

18.4 API endpoints (cms/src/routes/notifications.js, assumed prefix /api/notifications)

Method
GET
PATCH
PATCH

18.5 Frontend

The bell dropdown lives inside TopBar.jsx itself (not a separate page) — unread count badge, dropdown list, "Mark all read," footer text that is now conditional on Auditor status (Section 3).

18.6 Known issues (Notifications)

- None specific beyond the general pattern already documented — the best-effort/non-blocking design is a deliberate and sound choice, not a gap.

---

19. SETTINGS / COMPANY BRANDING

19.1 Purpose

Lets an Admin configure company identity — name, address, logo, brand colors, mission/vision/core values — which then flows into the sidebar, generated documents (Section 4.19), share certificates (Section 4.13), and emails.

19.2 Data model

company_settings: id (singleton row), company_name, company_address, primary_color, accent_color, description, mission, vision, core_values, logo_path, updated_by, updated_at.

19.3 Business rules / key logic

- Read (GET /settings/company) is open to **any authenticated user** — deliberately, since the sidebar/topbar/generated-documents need company name/branding on every single page load, not just for admins. (Notably, this route was *not* included in the blockAuditor rollout — Auditor legitimately needs to see company branding to use the External Audit portal UI at all.)
- Write (PATCH /settings/company) is gated with requireRoles(['Admin']) directly — a deliberate requireRoles choice (not the configurable permission system) because company identity is treated as foundational configuration, the same reasoning applied to contribution-recording restrictions in v1.4.0 per the route file's own code comments.
- Logo upload (POST /settings/company/logo) uses the shared uploadSingle multer middleware, storing into the branding upload subfolder.
- This is also the module referenced in the earlier "Categories" section as living in the same SettingsPage.jsx file as the Categories tab — Settings is a multi-tab page, not just branding.

19.4 API endpoints (cms/src/routes/settings.js, prefix /api/settings)

Method	Path	Permission	Description
GET	/company	any authenticated user (including Auditor)	Read branding/company info

PATCH	/company	requireRoles(['Admin'])	Update company settings
POST	/company/logo	requireRoles(['Admin'])	Upload/replace the logo

19.5 Frontend

cms-frontend/src/pages/settings/SettingsPage.jsx — tabbed interface; a "Company" (or "Branding") tab with name/address/colors/mission/vision/values fields and a logo uploader, alongside the Categories tab (Section 4.5) and likely Roles/Permissions management (Section 6).

19.6 Fiscal Quarters (v1.25.0)

Lets an Admin define the company's own financial-year quarters as fully custom date ranges — not constrained to equal three-month blocks, so a financial year that doesn't follow the calendar year (or quarters of uneven length) is supported. Purely a labelling/lookup table: it never changes any of the system's actual calendar-month-based figures, only how a date is described.

Data model — **fiscal_quarters**: id, label (e.g. "FY2025/26 — Q1"), start_date, end_date (CHECK (end_date >= start_date)), created_by, created_at. Deliberately does **not** enforce non-overlapping ranges at the database level — fully custom ranges were chosen over rigid non-overlap enforcement, so overlapping quarters are possible if an Admin configures them that way (see lookup rule below).

Lookup rule (services/fiscalService.js, getQuarterForDate(date)) — finds every configured quarter whose range contains the given date; if more than one matches (an overlap), the most recently-starting one wins. Reports (Section 17) call this once per report generation, against the *last day* of the reported month (the closing date the report's figures are as-of), and attach the result as report.fiscal_quarter ({ id, label, start_date, end_date }, or null if no quarter covers that date) — shown on both the General and Personal report screens as a small label next to the period, purely informational.

API endpoints (cms/src/routes/settings.js, prefix /api/settings/fiscal-quarters) — reads are open to any authenticated user (Reports needs the label on every report view, not just for Admins); writes are Admin only:

Method
GET
POST
PUT
DELETE

Frontend — a "Fiscal Quarters" tab in SettingsPage.jsx alongside Stamps and Membership Agreement: a simple list of configured quarters with inline edit, plus an "Add

Fiscal Quarter" form (label, start date, end date). Reports (ReportsPage.jsx) shows the matched quarter's label as a small pill next to the report period, when one exists.

19.7 Known issues (Settings)

- None specific surfaced by research beyond the general note that this is one of the few routes intentionally left open to Auditor — worth remembering during any future audit of blockAuditor coverage so this isn't mistakenly "fixed" into inaccessibility.

---

20. GLOBAL SEARCH

20.1 Purpose

A single search bar (TopBar) that queries across multiple record types (transactions, users, documents, events, etc. — by reference code, name, or description) so users don't need to know which module a record lives in to find it.

20.2 Data model

No dedicated table — queries run live against each relevant module's existing tables, likely via ILIKE/full-text matching against reference codes, names, and descriptions, unioned into a single result set.

20.3 Business rules / key logic

- Results are scoped by the searching user's permissions — a search doesn't bypass module-level access control (e.g. a regular Shareholder searching won't see other members' private financial detail beyond what they could already view via the module's own list endpoint).
- This is the Global Search button in the TopBar that was wrapped in {!isAuditor && (…)} this session (Section 3) — Auditor has no legitimate use for company-wide search and it was hidden outright rather than merely access-controlled server-side, since it has no meaningful scoped result set for that role.

20.4 API endpoints (cms/src/routes/search.js, prefix /api/search)

Method
GET

20.5 Frontend

The search icon/input in TopBar.jsx, opening a dropdown of grouped results by record type, each clickable to navigate to the relevant record's detail view.

20.6 Known issues (Global Search)

- No issues specific to this module surfaced by research beyond the now-fixed Auditor visibility (Section 3).

---

21. DOCUMENT GENERATION / EXPORT PIPELINE (exportUtils.js)

21.1 Purpose

The client-side pattern used almost everywhere in the app to produce a "PDF" — rather than a server-rendered file, a formatted HTML template is built in the browser and handed to the browser's native print dialog (window.print()), which the user then saves as PDF. This avoids running a heavy server-side rendering dependency (like Puppeteer, Section 4.13) for the vast majority of documents, reserving that heavier approach only for the one case that truly needs it (emailed share certificates).

21.2 Key functions

- **printDocument(templateFn, data)** — builds the HTML via the given template function, opens a hidden iframe or new window, writes the HTML, and triggers window.print().
- **previewDocument(templateFn, data)** — same HTML generation, but renders it in an on-screen modal/preview first rather than immediately printing, letting the user review before printing/exporting.
- **GENERATED_RENDERERS map** — a lookup table from document/template type to its renderer function, covering (per research): MEETING_AGENDA, MEETING_MINUTES, RECEIPT, RESOLUTION, AUDITOR_FEEDBACK. Each entry is a template function that takes structured data (e.g. an event record for a Meeting Agenda) and returns print-ready HTML with company branding (Section 4.20) applied.
- Every module's "Export"/reference-click-to-preview behavior described throughout Section 4 (transactions, transfers, requisitions, reports, etc.) routes through this same shared pipeline — it is the single mechanism, not a per-module reimplementation.

21.3 Known issues (Export pipeline)

- Being entirely client-side and print-dialog-based means exported "PDFs" are only as good as what the user's browser print-to-PDF renderer produces — no server-side guarantee of pixel-perfect output the way Puppeteer provides for certificates. This is a deliberate, reasonable trade-off (speed/simplicity vs. one-off rendering quality) rather than a defect, but worth understanding when comparing certificate output quality to every other document type in the system.
- **Fixed in v1.25.1 — every generated-document preview/download was broken.** documentsController.downloadDocument and auditController.previewEngagementDocument both return a SYSTEM_GENERATED document's fields wrapped in the standard { success, message, data: {…} } envelope (sendSuccess), but the two frontend consumers — DocumentsPage.jsx's openDocument() and AuditorPortalPage.jsx's handlePreviewDocument() — were reading document_type/template_data/title off the *top-level* parsed JSON instead of .data, so GENERATED_RENDERERS[payload.document_type] was always undefined and every preview/download (regardless of document type) failed with "This document type can't be reconstructed for preview/download." Fixed by unwrapping .data before

the renderer lookup in both files. This was a universal bug, not specific to any one document type — anyone who generated a Meeting Minutes, Meeting Agenda, Receipt, Resolution, or Auditor Feedback document and then tried to preview/download it afterward would have hit this. #### 22. AUTHENTICATION

22.1 Purpose

Handles login, session management (JWT access + refresh tokens), two-factor authentication (TOTP), and password reset — the front door of the system.

22.2 Data model

Auth-relevant columns live mostly on **users** (Section 6.2) plus: **refresh_tokens**: id, user_id, token_hash, expires_at, revoked, created_at, device_info. **password_reset_tokens**: id, user_id, token_hash, expires_at, used. **user_2fa** (or columns on users): totp_secret, totp_enabled, backup_codes.

22.3 Business rules / key logic

- **JWT access token**: short-lived (JWT_EXPIRES_IN, default 15m, Section 8). **Refresh token**: long-lived (JWT_REFRESH_EXPIRES_IN, default 30d), stored server-side (hashed) so it can be revoked (e.g. on logout or password change).
- **authenticate middleware** (middleware/auth.js) verifies the JWT signature/expiry, then **re-loads the full current user record from the database on every single request** — roles, permissions, and is_active status — rather than trusting stale claims baked into the token at login time. This means a role change, permission change, or account deactivation takes effect on the user's *very next request*, not merely at their next login. This is a deliberate, security-conscious design choice worth calling out explicitly, since it's not the default behavior most JWT tutorials implement.
- **2FA (TOTP)**: optional per-user, standard app-name TOTP_APP_NAME (Section 8) shown in authenticator apps (e.g. Google Authenticator) during setup; once enabled, login requires the current 6-digit code in addition to the password.
- **Password reset**: token-based, single-use, time-limited; per SYSTEM_DIAGNOSTIC_REPORT.md, the **UI** for this flow was a previously-missing Critical item, now marked **FIXED**.
- **Bcrypt** password hashing, BCRYPT_ROUNDS configurable (default 12, Section 8).
- **401-refresh interceptor**: the frontend Axios interceptor automatically attempts a token refresh on a 401 response and retries the original request once — per the diagnostic report, there is a known **bypass gap** (Moderate item #14) where certain request paths don't go through this interceptor consistently, meaning some 401s may not trigger an automatic refresh/retry and could surface as an unexpected logout instead. Still open as of the diagnostic report's date.

22.4 API endpoints (cms/src/routes/auth.js, prefix /api/auth)

Method	Path	Description
POST	/login	Email/password (+ TOTP code if 2FA enabled) →

		access + refresh token
POST	/refresh	Exchange a valid refresh token for a new access token
POST	/logout	Revoke the current refresh token
POST	/forgot-password	Send a password-reset email
POST	/reset-password	Consume a reset token, set a new password
POST	/2fa/setup	Generate a TOTP secret + QR data for enrollment
POST	/2fa/verify	Confirm enrollment with a code
POST	/2fa/disable	Turn off 2FA (requires current password/code)

22.5 Frontend

cms-frontend/src/contexts/AuthContext.jsx — holds the current user, hasRole()/hasPermission() helpers used throughout the app (including this session's new isAuditor = hasRole('Auditor') pattern in TopBar.jsx), login/logout actions, and the Axios interceptor wiring. Login page, 2FA setup/verify pages, forgot/reset-password pages under cms-frontend/src/pages/auth/.

22.6 Known issues (Authentication)

- Diagnostic report Critical Fix confirmed: password-reset UI now exists (was previously missing).
- Diagnostic report Moderate item #14 (401-refresh interceptor bypass on some paths) — **still open**, not addressed this session.

---

23. USERS

23.1 Purpose

Member/officer account management — profile data, role assignment, activation/deactivation, and the shareholding view (Section 4.14) exposed via this module's routes.

23.2 Data model

users: id, email (unique), password_hash, first_name, last_name, phone, profile_picture_path, is_active, totp_enabled, created_at, last_login_at.

user_roles (join table): user_id, role_id, assigned_by, assigned_at — a user can hold multiple roles simultaneously (e.g. Treasurer + Director).

23.3 Business rules / key logic

- **/me** and **/roles** are deliberately left open to *every* authenticated user including Auditor (Section 3, RBAC Summary / users.js note) — a user always needs to read their own profile and know their own roles regardless of what other access they've been granted.
- **/shareholding** was the second leak this session found and fixed with blockAuditor (Section 3) — company-wide ownership percentages, inappropriate for the Auditor role.
- Deactivating a user (is_active = FALSE) doesn't delete their account or historical records — it blocks login and, per the authenticate middleware's live re-fetch behavior (Section 22.3), takes effect immediately on their very next request even if they still hold a valid, unexpired JWT.
- Role assignment supports multiple roles per user (e.g. someone can be both Secretary and a Director) — access is the union of all permissions/role-checks across every role they hold.
- Avatar/profile picture upload uses the shared multer upload middleware into a profiles subfolder; per diagnostic report Minor item, there's a "stale Avatar spot" — a UI location that doesn't reflect an updated avatar without a manual refresh (cosmetic).

23.4 API endpoints (cms/src/routes/users.js, prefix /api/users)

Method	Path	Permission	Description
GET	/me	any authenticated user (incl. Auditor)	Own profile
PATCH	/me	any authenticated user (incl. Auditor)	Update own profile/avatar
GET	/roles	any authenticated user (incl. Auditor)	Own assigned roles
GET	/	USER_VIEW_ALL (or similar)	List all users
GET	/:id	USER_VIEW_ALL	Single user detail
PATCH	/:id	Admin	Edit a user, assign/remove roles, activate/deactivate
GET	/shareholding	blockAuditor + FINANCE_VIEW_ALL-class permission	Company-wide shareholding registry (Section 4.14)

23.5 Frontend

cms-frontend/src/pages/users/UsersPage.jsx — member directory table with role badges, active/inactive status, Admin-only "Edit"/"Assign Roles"/"Deactivate" actions; a personal "My Profile" page for self-service edits, accessible to every role including Auditor.

23.6 Known issues (Users)

- Diagnostic report Minor item: a stale avatar display spot somewhere in the UI that doesn't refresh immediately after upload — cosmetic, not addressed this session.
- /shareholding leak — **fixed this session** (Section 3).

---

24. ROLES & PERMISSIONS (RBAC)

24.1 Purpose

The configurable side of access control — while the 9 system roles themselves are fixed/seeded (Section 3), *what each role can actually do* is governed by a many-to-many role_permissions mapping that an Admin configures through the UI, letting the club tune access without code changes.

24.2 Data model

roles: id, name (9 seeded values — Admin, Director, Treasurer, Assistant Treasurer, Secretary, Assistant Secretary, Coordinator, Shareholder, Auditor), description, is_system_role.

permissions: id, code (e.g. FINANCE_VIEW_ALL, FINANCE_TRANSACTION_CREATE, SYSTEM_CONFIG, CATEGORY_MANAGE, FINANCE_TRANSFER_CREATE, FINANCE_TRANSFER_APPROVE, FINANCE_FLOOR_LIMIT_UPDATE, ...), description, module_group.

role_permissions (join table): role_id, permission_id, granted_by, granted_at. **Ships with zero rows** — every single grant, for every role including Admin, must be configured manually post-install (Section 3, restated here as this is the module that actually performs that configuration).

24.3 Business rules / key logic

- **Two enforcement mechanisms coexist in the codebase**: requireRoles([...]) — hard-coded role-name string match, OR logic across the list — and requirePermissions([...]) / requireAnyPermission([...]) — permission-*code* match against whatever's been granted via role_permissions, AND/OR logic respectively. Which one a given route uses is decided per-route by whoever wrote it, and the two systems are **not** kept in sync automatically — a route using requireRoles(['Treasurer']) is completely unaffected by anything configured in the Roles & Permissions UI, since it never consults role_permissions at all. This is documented as SYSTEM_DIAGNOSTIC_REPORT.md Moderate finding #7, and remains an open architectural inconsistency, not fixed this session.

- Practical implication for whoever administers the system: **before assuming a permission toggle in Settings → Roles → Permissions will change what a role can do**, check whether the specific route in question actually reads role_permissions (requirePermissions) or is hard-wired to a role name (requireRoles) — Section 4's per-module endpoint tables throughout this Bible note which mechanism each route uses specifically so this can be checked without reading source code.
- Because role_permissions starts empty, **a fresh install of this system, including for Company B, grants literally nobody any permission-gated action until an Admin manually visits Settings → Roles → Permissions and grants them** — this is the single most important "day one" configuration step referenced by both DEPLOYMENT_GUIDE.md and GOING_LIVE_GUIDE.md, and is repeated here for completeness.
- The Auditor role is the sole intentional exception to "configure permissions here" — it is designed to hold **zero** role_permissions grants; its access model is entirely the /api/audit/* engagement-scoping system (Section 4.25) plus the blockAuditor denial-list (Section 3), not anything configured through this screen.

24.4 API endpoints (cms/src/routes/roles.js and/or permissions.js, likely under /api/roles, /api/permissions)

Method
GET
GET
GET
POST
DELETE

24.5 Frontend

A "Roles & Permissions" tab inside SettingsPage.jsx (alongside Categories and Branding, Sections 4.5/4.20) — role selector, checkbox/toggle grid of every permission code grouped by module, save action; Admin-only.

24.6 Known issues (Roles & Permissions)

- The requireRoles vs requirePermissions inconsistency (Moderate #7) is the headline open item — it means the Roles & Permissions screen does not fully control access across the whole app, only the subset of routes actually written against requirePermissions/requireAnyPermission. Resolving this would require auditing every route file and deciding, module by module, which model should apply — a nontrivial refactor, not something to undertake casually.

25. INTERNAL AUDIT LOG (audit_log)

25.1 Purpose

The system's own permanent, append-only record of "who did what, when" — completely separate from, and not to be confused with, the **External Audit Portal** (Section 4.25/Section 5), which is a member-facing workflow for engaging an outside Auditor. This module is purely internal/technical: every meaningful state change in the app writes one row here.

25.2 Data model

audit_log: id, user_id (who performed the action, nullable for system/cron-initiated actions), action (an ACTIONS.* constant, e.g. CONTRIBUTION_RECORDED, TRANSFER_APPROVED, SYSTEM_CONFIG_CHANGED), module (a MODULES.* constant, e.g. FINANCE, SYSTEM, USERS), record_type, record_id, description, metadata (JSONB — flexible extra detail per action type), ip_address, created_at.

25.3 Business rules / key logic

- **logAction(...)** (services/auditService.js) is the single entry point every module calls — always **inside the same database transaction** as the business change itself (unlike notify(), which deliberately runs *after* commit) — so an audit-log write can never succeed while the actual change it's describing fails to commit, or vice versa.
- Rows are never edited or deleted — a genuinely append-only table, matching the same "corrections are new entries, not edits" philosophy used for financial transactions (Section 4.2.3's reversal pattern).
- metadata (JSONB) lets each action type store whatever extra structured detail is useful (e.g. old value/new value for a settings change) without needing a new column per action type.

25.4 API endpoints (cms/src/routes/system.js or a dedicated audit-log route, likely under /api/system/audit-log or /api/audit-log)

Method	Path	Permission	Description
GET	/	Admin (or SYSTEM_CONFIG)	Paginated, filterable audit trail (by user, module, action, date range)
GET	/:id	Admin	Single entry detail incl. full metadata

25.5 Frontend

An "Audit Log" view (likely under Settings or a dedicated System page), Admin-only — filterable table (user, module, action, date range), expandable rows for metadata detail. This is a distinct screen from the External Audit Portal's own submission/review UI (Section 5) despite the similar naming — worth being explicit about that distinction

anywhere this Bible or the app itself uses the word "audit," since the system now has two genuinely different features sharing that word.

25.6 Known issues (Internal Audit Log)

- No functional issues surfaced by research. The main risk worth flagging for the future is unbounded growth — like the transactions table, audit_log grows forever with no archival/pruning mechanism, which the earlier scaling-advice conversation already identified as a general "watch this as the system grows" item (Section 8/9 cross-reference).

Cross-module notes (Auth, Users, Roles & Permissions, Internal Audit Log)

- The **live-reload authenticate pattern** (Section 22.3) is the single most important security property underpinning this entire cluster: it's *why* deactivating a compromised account, revoking a role, or fixing a permission grant takes effect immediately rather than only at next login — worth remembering when troubleshooting "I changed X but the user still seems to have old access," since the far more likely explanation is a requireRoles-vs-requirePermissions mismatch (Section 24.3) than a caching/reload delay.
- This cluster is where the blockAuditor fix's two targeted route-level applications live conceptually (/shareholding in users.js) even though the middleware itself is defined in auth.js's file (Section 22) — cross-referenced from Section 3 for anyone looking for the code, not just the policy. #### 26. INFRASTRUCTURE & CROSS-CUTTING TECHNICAL PATTERNS

26.1 Purpose

Everything that isn't a single business module but underpins all of them: the scheduled-job runner, email delivery, the reference-code generator, file uploads, and the two-company Render deployment architecture.

26.2 Scheduled jobs (node-cron)

All jobs are registered at backend startup (likely in server.js or a dedicated jobs/ bootstrap file) and run in-process — meaning **the backend web service itself is the job runner**; if it's ever scaled to multiple instances, every job would fire once per instance unless a locking mechanism is added (this exact risk was raised in the earlier scaling-advice conversation as a "watch this" item).

Job	Schedule (cron)	Purpose
Monthly report	MONTHLY_REPORT_CRON (default 0 8 1 * * — 8am, 1st of month)	Compiles + emails the financial summary report (Section 4.17)
Loan overdue check	(daily, exact expression per loanService.js/job file)	Re-evaluates loan repayment schedules, flags overdue installments,

		notifies
Audit engagement reminder	(per this session's build — reminds Auditors of upcoming access-expiry / pending submissions)	Built this session as part of the External Audit workflow (Section 4.25)
Grant tranche overdue check	(if implemented as a job rather than query-only)	Flags tranches past their expected date
Reference-sequence maintenance / misc housekeeping	as applicable	

(Exact cron expressions beyond the two confirmed above — MONTHLY_REPORT_CRON and the audit reminder job built this session — should be verified directly against the current jobs/ source if precise scheduling detail is ever needed operationally; this table reflects what's confirmed from research plus this session's own build.)

26.3 Email delivery

- Sent via Gmail SMTP using GMAIL_USER / GMAIL_APP_PASSWORD (an app-specific password, not the real Gmail account password — required because Gmail blocks plain password SMTP auth for third-party apps).
- Used for: notification emails paired with bell notifications (notify(), Section 4.18), the monthly report, share certificate delivery (Section 4.13, the one place email is integral to the feature rather than a side notification), password reset, and audit engagement communications (Section 4.25).
- Failure mode is always best-effort/non-blocking except where a feature's entire purpose *is* the email (certificates) — even there, the PDF is generated and saved regardless of whether the send succeeds.

26.4 Reference-code system (recap + infrastructure detail)

- services/referenceService.js — generateReference(moduleCode, categoryAbbrev) locks a row in reference_sequences keyed by (module_code, category_abbrev, year_month) via INSERT … ON CONFLICT (…) DO UPDATE SET last_sequence = last_sequence + 1 RETURNING last_sequence, guaranteeing atomic, gap-free-per-key sequencing even under concurrent requests without a separate locking table.
- Every generated reference also gets a public_id — a random unguessable ~10-character string, stored alongside the human-readable reference — used anywhere a reference needs to be exposed externally (e.g. printed on a document, given to an Auditor) without revealing internal volume/ordering information the sequential reference number would leak.
- resolveModuleCode() — resolves which module-code prefix an account's transactions use (Section 4.1.3) — is the one piece of this system with per-module variation; most other modules use a fixed module code (GRANT, TRF, etc.) rather than deriving it from a linked account.

26.5 File uploads

- middleware/upload.js — multer-based, single shared middleware (uploadSingle(fieldName, subfolder)) used across Documents, Settings/branding, Users/avatars, and the External Audit Portal's document uploads.
- Stored under UPLOAD_DIR (default ./uploads) directly on the backend service's local disk, size-capped by MAX_FILE_SIZE_MB (default 10).
- **Not persistent across redeploys/restarts on Render's current setup** — flagged in both DEPLOYMENT_GUIDE.md and the earlier scaling-advice conversation as a real operational risk once the club depends on uploaded files (logos survive because they're re-uploadable trivially; member-uploaded documents and audit evidence files would not survive a redeploy without a durable storage solution).

26.6 Deployment architecture (Render Blueprints)

Two fully independent Blueprint files exist in the repo root, one per company, sharing the exact same codebase:

	Company A (render.yaml)	Company B (render.company-b.yaml)
Backend service	cms-backend	cms-b-backend
Frontend service	cms-frontend	cms-b-frontend
Database	cms-db (db cms, user cms)	cms-b-db (db cms_b, user cms_b)
Company name (env)	ZWECK TUKULA Ltd	COMPANY_B_NAME_HERE placeholder → frontend REACT_APP_COMPANY_NAME currently set to INVESTABO GLOBAL INVESTMENTS LIMITED
Company initials	ZT	IGI
TOTP app name	ZWECK TUKULA CMS	COMPANY_B_NAME_HERE CMS (placeholder, not yet updated to match IGI)
REACT_APP_VERSION	1.26.2	1.26.2 (kept in sync manually — a pure display string, nothing reads it programmatically)

Both Blueprints provision: 1 Postgres database (basic-256mb plan, ~$6-7/mo — the paid successor to the retired "starter" DB plan, chosen specifically because this app holds real financial records and the Free tier's 90-day expiry is unacceptable), 1 backend web service (starter plan — specifically *not* Free, because Free spins down after 15 minutes idle and would silently kill the scheduled cron jobs), 1 frontend static site (free — static sites don't spin down the same way and cost nothing on Render). Total: roughly **$13-14/month per company**.

Build-time URL wiring: both the backend's FRONTEND_URL and the frontend's REACT_APP_API_URL are constructed inline in their buildCommand/startCommand (e.g. FRONTEND_URL="https://${FRONTEND_HOST}") rather than set as plain static env vars, because Render's fromService/property: host mechanism only resolves at deploy time and Create React App bakes REACT_APP_* values into the static build output at *build* time — so it has to be present at that exact moment, not set afterward.

sync: false env vars (must be filled in by hand in the Render dashboard post-deploy, never committed): GMAIL_USER, GMAIL_APP_PASSWORD, COMPANY_EMAIL, COMPANY_ADDRESS (backend); REACT_APP_COMPANY_ADDRESS (frontend). JWT_SECRET/JWT_REFRESH_SECRET use generateValue: true — Render generates a random secret at first deploy, never stored in the repo.

Frontend SPA routing fix: both frontend Blueprints include a rewrite rule (source: /* → destination: /index.html) — without it, refreshing the browser on any inner route (e.g. /investments/5) returns a 404 from the static host instead of letting React Router handle it client-side.

26.7 Future-date validation (v1.25.0)

A system-wide rule: the system cannot accept a financial entry dated in the future. Implemented as one reusable validator, notFutureDate (middleware/validate.js), attached with .custom(notFutureDate) onto the existing express-validator isISO8601() chain for a given field — not a new middleware layer, just one extra link added to chains that already existed. It compares calendar dates only (the date portion of the ISO string against today's date string), so a value of "today" always passes regardless of the time of day or the submitting browser's timezone.

Scope — deliberately narrower than "every date field." The rule is applied only to fields that represent *when money actually moved or an event actually happened*, not to fields that are legitimately forward-looking or scheduled. Applied (guarded): contribution_date, value_date, expense_date, paid_date/payment_date, received_date, disbursement_date, declaration_date, handout_date, deposit_date, entry_date, return_date, actual_end_date — across Transactions, Transfers, Side Fund, Loans, Grants, Investments, Dividends, Requisitions, Service Fees, and Savings. Deliberately **not** applied: due_date (loan repayments, investment milestones), start_date/end_date/expected_end_date of terms and agreements, first_coupon_date, event_date, required_by_date, access_expires_at, effective_from (rate-change dates, which can legitimately be scheduled ahead), maturity_date. The distinction is judgment, not a mechanical sweep of every isISO8601() field — each field was checked individually for what it actually represents.

Frontend mirrors it, but the backend is the real gate. Every guarded <input type="date"> also has max={today} set (across the corresponding pages — Transactions, Transfers, Side Fund, Loans, Grants, Investments, Dividends, Requisitions, Service Fees, Savings) so the browser's native date picker won't offer a future date in the first place. This is a UX nicety only; validateRequest rejects a future date server-side (422,

"This date cannot be in the future") regardless of what the client sends, exactly like every other validation rule in this system.

26.8 Known issues (Infrastructure)

- Company B's render.company-b.yaml still contains unfilled COMPANY_B_NAME_HERE/CB-style placeholders in the **backend** section (TOTP_APP_NAME: "COMPANY_B_NAME_HERE CMS", COMPANY_NAME: "COMPANY_B_NAME_HERE") even though the **frontend** section has already been updated with the real name (INVESTABO GLOBAL INVESTMENTS LIMITED / IGI) — these two should be reconciled (update the backend placeholders to match, or confirm the backend COMPANY_NAME/TOTP_APP_NAME values are meant to differ from the frontend's display name) before Company B's next deploy, since a mismatched TOTP app name would appear in every Company B user's authenticator app.
- Local-disk upload storage (Section 26.5) and single-instance-only cron execution (Section 26.2) are the two infrastructure items most likely to need attention as either company scales — both already flagged in the earlier scaling-advice conversation with this same reasoning, repeated here for a single authoritative reference point.
- No automated test suite exists (also raised in the scaling-advice conversation) — every fix and feature in this system, including everything built this session, has been verified via manual node --check/syntax validation and logical code review rather than automated regression tests. This is a reasonable trade-off at the current scale/team-size (a solo non-developer owner plus an AI developer) but is worth revisiting if the system is ever handed to additional developers.

---

27. EXTERNAL AUDIT PORTAL

27.1 Purpose

The one place in the entire system a genuine outsider — a real, external auditing firm engaged by the club, not a member or officer — gets a login at all. Everything else in this Bible assumes an internal user; this module exists specifically to give a scoped, time-boxed, read-mostly window into the club's finances for the sole purpose of an audit, plus a structured way for that auditor to submit their findings and have them formally reviewed and archived. This was built entirely in this engagement (v1.20.0, with the visibility-leak hardening in v1.20.1 covered in Section 3), and I (Claude) am writing this section directly from the source rather than from a research subagent, since I built it myself and know it firsthand.

The core design principle, stated once so it doesn't need repeating in every subsection below: **the Auditor role itself grants nothing.** Holding the role only unlocks the /api/audit/* route tree; every single one of those endpoints then re-derives what a specific auditor can see from the audit_engagement_* join tables — which accounts, which date range, which documents — rather than trusting anything about the role in general. An Admin explicitly attaches an auditor to an engagement; that attachment, not the role, is the real permission boundary.

27.2 Data model

audit_engagements — one row per audit assignment: id, name, description, period_start, period_end (the date range being audited — every scoped query clamps to this), access_expires_at (optional hard cutoff, independent of the audit period itself), status (ACTIVE/REVOKED), created_by, revoked_by, revoked_at, created_at.

audit_engagement_accounts — join table: engagement_id, account_id. Defines exactly which of the club's accounts (Section 4.1) this engagement can see transactions for. An engagement with no rows here would see nothing — Admin must explicitly select at least one account when creating an engagement (enforced by validation: account_ids must be a non-empty array).

audit_engagement_users — join table: engagement_id, user_id, added_by, added_at. Defines which login(s) are attached to this engagement. Adding a user here also auto-grants them the Auditor role if they don't already hold it — but critically, the person must **already have a registered account**; this endpoint never creates one from scratch (they self-register normally, requesting the Auditor role during registration, and an Admin then attaches them to the specific engagement).

audit_engagement_documents — join table: engagement_id, document_id, added_by, added_at. Defines which existing documents rows (Section 4.16) an Admin has explicitly shared with this engagement — separate from, and in addition to, the auto-archived documents a finished/approved submission generates (below).

audit_engagement_comments — id, engagement_id, user_id, comment_text, created_at, submission_id (nullable). A comment with submission_id IS NULL is "staged" — written by the auditor but not yet part of a formal submission round. Once finish is called, every staged comment for that engagement gets stamped with the new submission's id in one atomic update, locking it into that round.

audit_submission_files — id, engagement_id, file_path, file_name, file_size_bytes, mime_type, uploaded_by, uploaded_at, submission_id (nullable, same staged/locked pattern as comments), document_id (nullable — populated only once the submission is fully approved and archived into a real documents row).

audit_submissions — id, engagement_id, submitted_by, submitted_at, status (SUBMITTED/APPROVED/REJECTED), director_approved_by, director_approved_at, secretary_approved_by, secretary_approved_at, rejected_by, rejected_at, rejection_reason, feedback_document_id (the generated AUDITOR_FEEDBACK document, populated on finalization).

audit_extension_requests — id, engagement_id, requested_by, current_access_expires_at, requested_new_access_expires_at, reason, status (PENDING/APPROVED/REJECTED), reviewed_by, reviewed_at, reviewer_notes, created_at.

users also carries three auditor-specific profile columns used only by this module: auditor_company_name, auditor_company_initials, auditor_contact_phone —

required (all three) before an auditor can comment, upload a file, or finish an audit, since they feed directly into the reference-code prefix and the archived feedback document.

27.3 Business rules / key logic

Engagement access gate (assertEngagementAccess) — every single auditor-facing endpoint calls this first. It checks three things in order: (1) the requesting user is actually attached to this specific engagement via audit_engagement_users, (2) the engagement's status is still ACTIVE (not revoked), (3) if access_expires_at is set, it hasn't passed yet. Any failure throws a 403. This one function is the entire security model for the auditor-facing half of the module — nothing downstream re-derives access differently.

Scoped transaction/summary queries clamp in SQL, not JavaScript. getEngagementTransactions and getEngagementSummary both intersect any caller-supplied from_date/to_date against the engagement's own period_start/period_end using GREATEST/LEAST directly in the SQL, with an explicit code comment explaining why: Postgres returns DATE columns as JS Date objects, and comparing those against a plain 'YYYY-MM-DD' query-string with JS's </> operators does not behave as expected (the string doesn't coerce to a timestamp the same way Date does) — so the actual period boundary is enforced entirely in SQL, where the types are unambiguous. An auditor's date filter can only narrow the visible range, never widen it past what the engagement was scoped to.

Auditor profile completeness gate. Before an auditor can add a comment, upload a report file, or finish an audit, isAuditorProfileComplete() requires all three of auditor_company_name, auditor_company_initials, and auditor_contact_phone to be filled in (via the ordinary PATCH /api/users/me, not a special audit-portal endpoint). This exists because those exact fields feed buildAuditorModuleCode() (below) — without them, the reference codes on an eventual submission would be meaningless.

Reference-code prefix derivation (buildAuditorModuleCode) — combines the auditor's first name and their firm's initials into a sanitized, uppercase, alphanumeric-only string capped at 20 characters (the width of reference_sequences.module_code), falling back to 'AUDITOR' if the result would otherwise be empty. Example: auditor "John", firm initials "KPMG" → module code JOHNKPMG, producing reference codes like JOHNKPMG-FEEDBACK-202607-00001 for the generated feedback document and JOHNKPMG-REPORT-202607-00001 for each archived report file. This is the one place in the whole reference-code system where the module-code segment is derived from a *person*, not a fixed constant or an account's own prefix.

Stage → Finish → Dual-approve → Archive lifecycle, the heart of the module: 1. **Stage.** The auditor adds comments (POST …/comments) and/or uploads report files (POST …/report-files) one at a time, any number of times. Each lands with submission_id = NULL. A staging action is blocked outright if a submission for that engagement is already sitting in SUBMITTED status — you cannot add to a round that's already under review; you have to wait for it to be approved or rejected first. 2. **Finish.** POST …/finish requires the profile-completeness gate to pass and requires **at least one** staged comment or file to exist (an empty "finish" is rejected). It then, in a single DB transaction: inserts one

audit_submissions row, and atomically re-points every currently-staged comment and file at that new submission id — locking that batch together as one reviewable round. The auditor gets an immediate confirmation email/notification; every active Director and Secretary gets a “needs your approval” notification (best-effort, fired after the transaction commits, never blocking the response). 3. **Dual approval.** POST /submissions/:id/approve — a Director **and** a Secretary must each independently approve. The endpoint is role-aware: if the caller holds both roles, one call fills both slots at once; if they’ve already filled their applicable slot, calling again returns a 409 conflict rather than double-counting. The moment both slots are filled (checked inside a SELECT … FOR UPDATE-locked transaction to prevent a race between two near-simultaneous approvals), the submission is finalized in the same transaction: - A real documents row is created for the feedback itself (document_type='AUDITOR_FEEDBACK', source='SYSTEM_GENERATED', rendered client-side from template_data the same way every other generated document works, Section 4.21) — this is the one place AUDITOR_FEEDBACK referenced in the GENERATED_RENDERERS map (Section 4.21) actually gets produced. - A real documents row is created for **each** uploaded report file (document_type='AUDIT_REPORT', source='UPLOADED', pointing at the file already on disk from step 1 — no re-upload happens at finalization). - Every one of those new documents gets its own reference code via the shared generateReference() service and is linked back with linkReferenceToRecord() and inserted into audit_engagement_documents — meaning the finished, approved report becomes visible in the engagement’s own document list going forward, alongside anything an Admin shared manually. - This is deliberately the **only** point at which anything from this workflow becomes a permanent, referenced record: documents.reference_id is NOT NULL at the schema level, so nothing staged or merely submitted-but-not-yet-approved could ever have been written into the documents table directly, even by a bug — the dual-approval step is structurally, not just procedurally, the gate. - The submission flips to APPROVED, feedback_document_id is set, and the auditor receives a final confirmation email. 4. **Single-reviewer rejection.** Unlike approval, rejection needs **only one** Director or Secretary — POST …/reject with a required reason immediately flips the submission to REJECTED and emails the auditor the reason. The auditor can then stage a fresh round of comments/files and finish again; nothing about a rejected round is reused.

Extension requests — a lighter-weight, single-purpose parallel workflow: an auditor whose access_expires_at is approaching (or has a legitimate reason to need more time) submits a requested new expiry date plus a reason. Only one PENDING request per engagement is allowed at a time. A Director or Secretary approves (which directly updates audit_engagements.access_expires_at to the requested date) or rejects (with optional reviewer notes) — either action is a single-reviewer decision, no dual sign-off required here, unlike the submission-approval workflow.

Document preview mirrors the ordinary Documents module’s UPLOADED-vs-SYSTEM_GENERATED split (Section 4.16): an UPLOADED document streams the file from disk via res.download; a SYSTEM_GENERATED one returns its template_data for the frontend to render, exactly like every other generated document in the system. Every view of an engagement document is itself audit-logged (AUDIT_DOCUMENT_VIEWED)

— a deliberate extra layer of internal record-keeping specifically because an outside party is the one looking.

Reviewer preview of staged (not-yet-archived) files. Before a submission has been fully approved, its uploaded files have no documents row yet — so previewSubmissionFile (used by the Director/Secretary review page) reads straight from audit_submission_files, bypassing the normal Documents-module download path entirely. This is a deliberate, narrow exception: reviewers need to actually look at a report before approving it, but the file isn't a "real," referenced document until that approval happens.

27.4 API endpoints (cms/src/routes/audit.js, prefix /api/audit)

Admin — engagement management (requireRoles(['Admin']), same direct-role-check treatment given to Settings and floor limits — foundational configuration, not a day-to-day permission):

Method	Path	Description
GET	/engagements	List all engagements with account/user/document counts
GET	/engagements/:id	Full detail — accounts, attached users, shared documents
POST	/engagements	Create (name, description, period, optional access expiry, ≥1 account)
PATCH	/engagements/:id	Full-replace edit (blocked once revoked)
POST	/engagements/:id/revoke	Revoke (blocks all further auditor access immediately)
POST	/engagements/:id/users	Attach an auditor by email (auto-grants the Auditor role if needed)
DELETE	/engagements/:id/ users/:userId	Detach (does not strip the Auditor role itself — they may be on other engagements)
POST	/engagements/:id/ documents	Manually share an existing document with the engagement
DELETE	/engagements/:id/ documents/:documentId	Un-share

Auditor — scoped read-only access (requireRoles(['Auditor']), every function re-checks engagement membership):

Method	Path	Description
GET	/my-engagements	Engagements this login is attached to
GET	/engagements/:id/allowed-accounts	The account whitelist for this engagement
GET	/engagements/:id/transactions	Paginated, period-and-account-clamped transaction ledger
GET	/engagements/:id/documents	Documents shared with (or archived into) this engagement
GET	/engagements/:id/documents/:documentId	Preview/download one (membership-checked)
GET	/engagements/:id/summary	Opening/closing balances, totals, category breakdown, full transaction list — feeds the downloadable summary PDF

Auditor — submission workflow (v1.20.0):

Method	Path	Description
GET / POST	/engagements/:id/comments	List / add a staged comment
GET / POST	/engagements/:id/report-files	List / upload a staged report file
DELETE	/engagements/:id/report-files/:fileId	Remove a staged file (own uploads only, only while unsubmitted)
GET	/engagements/:id/submissions	This engagement's submission history + review status
POST	/engagements/:id/finish	Bundle everything staged into one submission for review
POST	/engagements/:id/extension-requests	Request more access time
GET	/engagements/:id/extension-requests	Own extension-request history

Director / Secretary — submission review (v1.20.0):

Method	Path	Description

GET	/submissions	List submissions, optional ?status= filter
GET	/submissions/:id	Full detail incl. comments and files
GET	/submissions/:id/files/:fileId	Preview a staged (not-yet-archived) file
POST	/submissions/:id/approve	Record this reviewer's sign-off; finalizes once both are in
POST	/submissions/:id/reject	Single-reviewer rejection with required reason
GET	/extension-requests	List, optional ?status= filter
POST	/extension-requests/:id/approve	Approve — updates the engagement's access expiry directly
POST	/extension-requests/:id/reject	Reject with optional notes

27.5 Frontend

Three distinct surfaces, matching the three audiences above: - **Admin engagement management** (built in this engagement's earlier phase) — create/edit engagements, pick accounts, attach/detach auditor logins by email, share/un-share documents, revoke. - **Auditor portal** (/audit — the one route an Auditor is force-redirected to no matter what URL they visit, per AppLayout.jsx's central guard, Section 4.16/Layout note) — a profile-completeness gate blocks the comment/upload/finish actions until the three required fields are filled in; scoped transaction ledger and summary view (with a downloadable PDF built from getEngagementSummary's data via the same auditSummaryTemplate()/print pipeline as everything else, Section 4.21); a staging area for comments and file uploads; a "Finish Audit" action that bundles the current staged batch into a submission; a submission-history view showing SUBMITTED/APPROVED/REJECTED status and, on rejection, the reason given; an extension-request form. - **Director/Secretary review page** (/audit-review, built this session) — lists pending submissions and pending extension requests, lets a reviewer open a submission to read its comments and preview its staged files before approving or rejecting, and shows the dual-approval progress (which of Director/Secretary has signed off so far) so a reviewer can see at a glance whether they're the first or second sign-off.

27.6 Known issues (External Audit Portal)

- This is the newest module in the system (built this engagement) and has not yet been exercised with a real external auditor in production — worth treating any edge case discovered during the club's first real audit cycle as an expected first-use finding rather than a sign something was rushed.

- The blockAuditor middleware rollout (Section 3) was applied to 18 other route files specifically *because* this module's introduction of a genuinely external, zero-permission role exposed pre-existing "open to any authenticated user" routes that had never mattered before. Any brand-new route added anywhere else in the system in the future should be written with the question "would an Auditor legitimately need this?" in mind from day one, rather than relying on a future audit to catch another accidental leak.

---

28. STAFF ACCESS & SERVICE FEES (ADMINISTRATIVE OFFICER ROLE)

28.1 Purpose

Introduced in v1.21.0 for hired/contracted staff who do real, ongoing ground-work for the company — the concrete example that shaped the design was someone acting as a de facto secretary: recording meeting minutes, liaising with authorities in person, but who is **not** a company member or shareholder and must never see financial data by default. This is a genuinely different shape of access than every other role in the system, which is why it's a brand-new role (**Administrative Officer**) rather than a variant of the existing Secretary role.

Two independent halves make up this module: 1. **Access control** — what an Administrative Officer can see day to day (Events and Documents, in full, minus Financial-category documents), plus a narrow exception mechanism (staff_document_grants) for the rare case where they legitimately need one specific financial document. 2. **Compensation** — a recurring monthly **service fee** and an expense **reimbursement** request/approval flow (service_fee_agreements, service_fee_payments, service_reimbursement_requests).

Terminology note — read this before touching this module. Everything here is deliberately called a "service fee" to a "contracted service provider," never "salary," "payroll," or "employee." Whether a specific hire should legally be treated as an employee or an independent contractor is a real question with tax and labour-law consequences that varies by country and by the actual working relationship — it is not something this software can or should decide. "Service fee" was chosen specifically because it doesn't presume an answer either way. If you are setting this up for a real hire, confirm the correct legal classification with an accountant or lawyer first; this module's naming is not legal advice and carries no weight in that determination.

28.2 Data model

Table	Purpose
staff_document_grants	One row per (document, user) exception. granted_by/granted_at always set; revoked_at/revoked_by set on revoke — rows are **never deleted**, so there's a permanent record of who could see what and when (UNIQUE(document_id,

	user_id); re-granting after a revoke reactivates the same row rather than inserting a second one).
service_fee_agreements	One row per person's standing monthly arrangement: monthly_amount, currency_id, account_id (which account pays it), category_id, start_date/end_date, status (ACTIVE/ENDED).
service_fee_payments	One row per actual monthly payment made against an agreement, linked to the real transactions row it produced.
service_reimbursement_requests	One row per expense reimbursement request: amount, description, expense date, optional receipt upload, status (PENDING/APPROVED/REJECTED), reviewer notes.

Deliberately no "engagement" wrapper, unlike the External Audit Portal's audit_engagements. An audit is a time-boxed relationship with a natural start and end; an ongoing staff relationship isn't, so staff_document_grants is a simple, direct, standing per-document grant instead of something scoped to a fixed period.

28.3 Business rules / key logic

Default access — no code, only configuration. Both routes/events.js and routes/documents.js already gated their create/view endpoints with the fine-grained requirePermissions([…]) pattern (not a hard-coded role check), so giving an Administrative Officer full Events access and document-upload/generate access required **zero code changes** — just granting EVENT_VIEW, EVENT_CREATE, DOCUMENT_VIEW, DOCUMENT_UPLOAD, and DOCUMENT_GENERATE to the role through Settings → Roles → Permissions after the migration runs (remember: role_permissions ships empty for every role, Section 3).

Financial-document filtering (documentsController.js) is where the real restriction lives, since Documents is a mixed-category module (not everything in it is financial): - isFinanceCategoryAbbrev() treats a document as financial if its category path's full_abbreviation is exactly 'FIN' or starts with 'FIN-' — matching on the category tree rather than a hardcoded ID means any future Financial sub-category is automatically covered without a code change. - getAllDocuments adds a WHERE-clause exception for an Administrative Officer: show the document if it's *not* financial, **or** if there's a matching active row in staff_document_grants. - getDocumentById / downloadDocument call a shared assertDocumentVisible() check that throws a 403 with a clear message ("Ask an Admin to grant you access to this specific document") if a financial document is opened directly by ID without a grant. - Everything else (Accounts, Transactions, Transfers, Grants, Loans, Investments, Requisitions, Savings, Side Fund, Dividends, Reports, Search,

shareholding) is blocked outright by blockFinanceRestricted (Section 3) — there is no per-item exception mechanism for those, only for Documents.

Granting/revoking a document exception (staffAccessController.js, Admin-only): grantDocument checks for an existing (possibly revoked) row first and re-activates it with an UPDATE rather than risking the UNIQUE(document_id, user_id) constraint with a second INSERT. revokeGrant soft-revokes (sets revoked_at), never deletes.

Service fee payments and approved reimbursements both go through postTransaction() — the same single choke-point every other money movement in the system uses (Section 4.26) — so the "never go negative" rule and floor-limit checks apply here exactly as they would to any other outgoing transaction. transactions.inflow_type was widened to add 'SERVICE_FEE_OUT' and 'SERVICE_REIMBURSEMENT_OUT'.

Who can do what, financially, in this module: - **Admin** creates, amends, or terminates a service fee agreement (who, how much, which account, which category). No separate currency field exists anywhere in this module (v1.21.1) — an account can only ever hold one currency, so the paying account chosen IS the currency; currency_id is derived server-side from account_id on both create and amend (if the paying account changes on amend, currency_id is recomputed to match), the same pattern every other money-recording endpoint in this system already follows. - **Treasurer / Assistant Treasurer** (Admin also allowed) record the actual monthly payment against an active agreement, and review (approve/reject) reimbursement requests — approving one immediately posts the real transaction and pays it, in the same step. Viewing the Agreements list (and recording a payment) is available to this role too, not just Admin — only *creating or amending* an agreement is Admin-only. - **The contracted person themselves** (any authenticated user — this isn't restricted to the Administrative Officer role specifically, since a service fee arrangement could in principle be set up for anyone) sees their own agreement and payment history, and submits reimbursement requests with an optional receipt upload.

Amending and terminating an agreement (v1.21.1) both go through the same PATCH /agreements/:id endpoint the create form's Admin-only sibling modal uses on the Agreements tab: "Amend" edits monthly amount, paying account, category, and notes (who it's for and the original start date don't change); "Terminate" is a focused, separate action that sets status = 'ENDED' with a required end date, after which no further monthly payment can be recorded against that agreement. Both are logged (SERVICE_FEE_AGREEMENT_UPDATED).

28.4 API endpoints

Staff Access (cms/src/routes/staffAccess.js, prefix /api/staff-access):

Method	Path	Who	Description
GET	/grants	Admin	List grants — filter by document_id and/or user_id; active-only unless include_revoked=t

			rue
POST	/grants	Admin	Grant one document to one user
DELETE	/grants/:id	Admin	Revoke (soft)
GET	/my-documents	Self	Documents granted to the caller
GET	/my-documents/:documentId	Self	Preview/download a granted document (access checked against the grant, not DOCUMENT_VIEW)

Service Fees (cms/src/routes/serviceFees.js, prefix /api/service-fees):

Method	Path	Who	Description
GET	/agreements	Admin, Treasurer, Assistant Treasurer	List all agreements
GET	/agreements/:id	Admin, Treasurer, Assistant Treasurer	Full detail incl. payment history
POST	/agreements	Admin	Create a new agreement
PATCH	/agreements/:id	Admin	Edit / end an agreement
POST	/agreements/:id/pay	Treasurer, Assistant Treasurer, Admin	Record a monthly payment (defaults to the standard monthly amount)
GET	/my-agreement	Self	The caller's own agreement + payment history
POST	/reimbursements	Self	Request a reimbursement (with optional receipt upload)
GET	/my-reimbursements	Self	The caller's own reimbursement history
GET	/reimbursements	Admin, Treasurer, Assistant Treasurer	Review queue, optional ?status=

			filter
GET	/reimbursements/:id/receipt	Owner or reviewer	Download the attached receipt
POST	/reimbursements/:id/approve	Treasurer, Assistant Treasurer	Approve — posts and pays the transaction in one step
POST	/reimbursements/:id/reject	Treasurer, Assistant Treasurer	Reject with a required reason

28.5 Frontend

- **Sidebar/TopBar** — an Administrative Officer keeps a normal, multi-page sidebar (unlike the Auditor's single-page forced redirect); every finance-adjacent nav item is simply hidden (Sidebar.jsx's isAdminOfficer check), and the TopBar's account-balance widget is skipped for the same role while upcoming events still load.
- **/service-fees** (ServiceFeesPage.jsx) — three tabs: "My Service Fee" (everyone — own agreement, payment history, a "Request Reimbursement" action, own reimbursement history), "Agreements" (Admin + Treasurer/Assistant Treasurer — list, create via Admin-only modal, record-payment action), "Reimbursement Requests" (Treasurer/Assistant Treasurer — review queue with approve/reject actions and a receipt-download link, with a pending-count badge on the tab itself).
- **Documents page** — an Admin-only "Grant staff access" icon action on each row opens a small modal showing who currently has access to that document (with per-person revoke) and a picker to grant it to someone new.

28.6 Post-install configuration (do this after running the migration)

This role, like every role in this system, ships with zero permissions granted. After migration_v1.21.0.sql runs: 1. Restart the backend so it picks up the new routes. 2. Assign the Administrative Officer role to the relevant user via Users → Assign Role. 3. Grant EVENT_VIEW, EVENT_CREATE, DOCUMENT_VIEW, DOCUMENT_UPLOAD, DOCUMENT_GENERATE to the Administrative Officer role via Settings → Roles → Permissions — without this step the role can log in but can't do anything yet. 4. Create a Service Fee agreement for them via /service-fees → Agreements → New Agreement. 5. Grant individual Financial-category documents as needed via Documents → (the new "Grant staff access" icon on a row).

28.7 Known issues / open items

- Like the External Audit Portal, this module has not yet been exercised with a real hired staff member in production — treat the first real use as the actual test.
- The Documents-module financial filter matches on category *path* (FIN/FIN-%), not a fixed category ID, so it's self-maintaining as new Financial sub-categories are added — but a Financial-category document that gets **moved** to a non-Financial category (or

vice versa) after creation would immediately change visibility for this role. There's no audit-log entry specifically for that category-reassignment case today.

---

29. Digital Consent & Multi-Signatory Approval (v1.23.0)

29.1 Purpose

Two related problems this module solves:

1. **Digital consent + a real signature, once per member.** Before v1.23.0 there was no record of a member ever agreeing to anything, and no signature stored anywhere in the system — a document could say "Approved by Jane Doe" but had no actual mark from Jane attached to it. Now, once a new member's role is assigned (the same point the v1.21.1 pending-approval gate hands off to), they're taken to a one-time Consent screen: draw a signature (a signature pad, not a photo/scan upload — chosen for a consistent result on any device, no scanner needed), then read and consent to the company's Membership Agreement. Neither step can be skipped, and it happens exactly once per member — an Admin editing the Membership Agreement's wording later does **not** force anyone who already consented to consent again (that's a deliberate choice, not an oversight — see 29.3).
2. **More than one person's signature required on some documents.** Resolutions, Loan/Grant Agreements, and Share Certificates can now require several specific *roles* (not specific people — whoever currently holds the role) to each sign before the document counts as approved. This is opt-in per document type, configured by an Admin in Settings → Signatories; a document type with nothing configured behaves exactly as it always did (one approver, done).

29.2 Data model

- **users.signature_path / signature_updated_at** — the member's current signature image (PNG), stored on disk the same way photo_path is, referenced by URL (/uploads/signatures/…). Redrawing it (Profile → Signature) overwrites this — it does **not** retroactively change any document already signed, because signing takes a snapshot (see below).
- **membership_agreement** — a singleton row (id = 1, same pattern as savings_settings/side_fund_config) holding the current agreement content and a version number that increments every time an Admin edits it. Editing does not invalidate existing consents.
- **member_consents** — one row per member, ever (user_id is UNIQUE) — agreement_version consented to, consented_at, ip_address, user_agent. Its mere existence for a user is what requireConsent checks.
- **signature_requirements** — Admin-configured: document_type (RESOLUTION / LOAN_AGREEMENT / GRANT_AGREEMENT / SHARE_CERTIFICATE) + role_id + is_active. A document type with zero active rows has no multi-signature requirement.

- **document_signatures** — the generic signing-slot table, reused for two different kinds of "thing being signed" via target_type:
  - target_type = 'DOCUMENT', target_id = a documents.id (Resolutions, Loan/Grant Agreements).
  - target_type = 'CERTIFICATE_ROUND', target_id = a certificate_signing_rounds.id (see 29.5) — **one signature covers every certificate in that round**, not one signature per certificate. Each row is one required role's slot: required_role_id, status (PENDING/SIGNED), signed_by, signature_snapshot_path (a **copy** of the signer's users.signature_path taken at the moment they sign — this is what makes a later signature redraw safe), signed_at.
- **documents.fully_signed / fully_signed_at** — set once every required role's slot for that document is SIGNED. For a document type with no signature requirement configured, this is set immediately by the original single-approver approveDocument call, same moment status flips to FINAL.
- **certificate_signing_rounds** — one row per (certificate_type, period_label) batch, e.g. ('MONTHLY', '202608'). share_certificates.signing_round_id links each certificate issued in that batch to it.

29.3 Business rules / key logic

Consent is a hard gate, applied the same way the v1.21.1 pending-approval gate is. A new requireConsent middleware sits immediately after requireAssignedRole in every route file that already had it (the same ~22 files, plus the equivalent privileged routes in users.js) — a role-assigned member who hasn't consented gets a 403 from the backend and, on the frontend, AppLayout.jsx redirects them to /consent before they ever see the Sidebar/TopBar/Dashboard, exactly like a zero-role account gets redirected to /pending-approval. The onboarding chain is now: verify email → get a role assigned → consent + sign → full access. **This applies to every existing member too, not just new ones** — there is no way to know a member who used the system before v1.23.0 already implicitly agreed to anything, so everyone goes through the same one-time gate once, on their next login after this migration runs. giveConsent requires users.signature_path to already be set (the frontend enforces drawing a signature before the "Agree and continue" button becomes clickable, but the backend checks independently too).

A document-signature slot belongs to a ROLE, not a person. signSlot looks up which of the caller's *currently held* roles matches a still-PENDING slot for the target, and fills that slot — so if the Treasurer changes mid-year, whoever holds the role at signing time is the one who signs, not whoever held it when the document was created. Signing is first-come-first-served among anyone holding a required role; the moment the last required slot is SIGNED, the target flips to fully signed automatically (no separate "finalize" step needed).

approveDocument (Resolutions, Loan/Grant Agreements) branches on whether signature_requirements exist for that document_type, checked fresh on every call: - **Nothing configured** → unchanged from before this feature existed: one call flips status straight to FINAL, sets approved_by/approved_at/fully_signed/fully_signed_at all at

once. - **Something configured** → the call instead opens the required signing slots (idempotent — calling Approve again just returns the current signing status, it doesn't reset anything) and the document stays DRAFT until POST /documents/:id/sign is called by someone holding each required role. approved_by/approved_at end up recording whoever supplied the **last** signature, not whoever clicked Approve first.

Certificate signing rounds (Section 29.5) follow the identical opt-in rule — configure signature_requirements for SHARE_CERTIFICATE and the monthly/annual batch holds until signed; leave it unconfigured and certificates keep emailing immediately, exactly as they did before this feature existed.

29.4 API endpoints

Method	Endpoint	Access	Purpose
GET	/users/me/membership-agreement	Any authenticated user (pre-consent)	Current agreement text/version + this user's own consent status
POST	/users/me/consent	Any authenticated user (pre-consent)	Give one-time consent — requires a signature already saved
PATCH	/users/me/signature	Any authenticated user (pre-consent)	Save/redraw signature — body { signature_data_url } (base64 PNG from the SignaturePad)
PATCH	/settings/membership-agreement	Admin	Edit the agreement text (bumps version, does not force re-consent)
GET / PUT	/settings/signature-requirements[/:documentType]	Admin	Read / replace which roles must sign a document type
POST	/documents/:id/sign	DOCUMENT_APPROVE permission	Fill the caller's role's pending slot on a document
GET	/documents/:id/signatures	Any authenticated user	Current signing status for a document
GET	/certificates/rounds	Treasurer / Assistant Treasurer / Admin	List signing rounds

GET	/certificates/ rounds/:id	Treasurer / Assistant Treasurer / Admin	One round + its signature status
POST	/certificates/ rounds/:id/sign	Any role-assigned, consented member	Fill the caller's role's pending slot on a round — signSlot itself enforces that the caller actually holds a required role

29.5 Monthly/annual certificate signing rounds, in detail

The existing issueCertificatesForAllShareholders pipeline (used by both the scheduled cron jobs and the Admin "Issue Now" button) is unchanged in *when* it runs and *what* it calculates — it still issues one share_certificates row per active shareholder with their current figures. What changed is what happens next:

1. Every certificate issued in one run is grouped into a single certificate_signing_rounds row for that (certificate_type, period_label).
2. If signature_requirements has active roles configured for SHARE_CERTIFICATE, the round's signing slots open and its configured signatories are notified — **no certificates are emailed yet**.
3. Each signatory calls POST /certificates/rounds/:id/sign once — this signs the **round**, not each certificate individually, which is what makes a round of, say, forty shareholders' certificates practical to approve (one click, not forty).
4. The moment the last required role signs, every certificate in the round is re-rendered — the PDF's three previously-blank signature lines are replaced with however many roles are configured, each showing that signer's actual signature image and name — and emailed to its holder, same as the original immediate-email behaviour just deferred until now.

A new scheduled job, scheduleCertificateSigningReminders (cms/src/jobs/scheduler.js), runs daily at 08:00 during the last week of every month (0 8 24-31 * *) and reminds — by notification and email — any signatory who still has a pending slot on a still-OPEN round. This is the system's answer to "signatories approve every last week of the month once accounts are up to date": the round itself opens whenever certificates are issued (still the 1st of the month, unchanged), but the last-week reminder is specifically timed to prompt sign-off once that month's activity has settled, rather than rushing it on day one.

v1.23.1 addendum — document signature slots now email too. At launch, certificate-round notifications already carried an email block, but Resolution/Loan/Grant Agreement document signature slots had no notification mechanism at all — a signatory only found out their signature was needed by noticing the amber Signatures icon on the Documents page. A new shared helper, notifyPendingSignatories (signatureService.js), looks up

everyone currently holding a still-PENDING required role for a target (deduplicated, since one person can hold more than one required role) and sends each of them both a bell notification and an email via the existing notifyMany mechanism. It's called from two places: approveDocument the moment signature slots first open (guarded by an alreadyHadSlots check so clicking Approve again on an already-open document never re-emails everyone), and a new daily job, scheduleDocumentSignatureReminders (cms/src/jobs/scheduler.js, 30 8 * * *), which reminds anyone still pending on any document with open slots — the document equivalent of the certificate-round reminder job above. Code-only change, no schema migration.

29.6 Frontend

- **SignaturePad.jsx** (components/common) — a small canvas-based draw pad using the native Pointer Events API (mouse, touch, and stylus all work identically, no external drawing library needed), exporting a white-background PNG data URL. Used on both the Consent page and Profile → Signature.
- **ConsentPage.jsx** (/consent, standalone — no Sidebar/TopBar, same shape as PendingApprovalPage.jsx) — draw-and-save a signature, read the Membership Agreement, check a box, submit. AppLayout.jsx redirects any role-assigned-but-not-consented user here.
- **Profile → Signature tab** — redraw the signature later; existing signed documents are unaffected (they hold their own snapshot).
- **Settings → Signatories** — per document type, checkboxes for which roles are required; **Settings → Membership Agreement** — a plain textarea to edit the consent text, with the one-time-consent caveat repeated in the UI itself.
- **Documents page** — a Signatures icon (amber while pending, green once fully signed) on Resolution/Loan Agreement/Grant Agreement rows opens a modal listing each required role, who (if anyone) signed, their signature image, and a Sign button if the current user's role has an open slot.
- **Reports page → Certificate Signing Rounds panel** (Treasurer/Assistant Treasurer/Admin) — expandable list of rounds, each showing per-role signature status and a Sign action for eligible signatories.

29.7 Known issues / open items

- **Ad-hoc, single-certificate issuance** (POST /certificates, the "download my own certificate now" self-service action) is **not** part of the signing-round gate — it issues and the user can view/print it immediately, same as before this feature existed. Only the batch monthly/annual pipeline goes through a signing round. share_certificates.signing_round_id is NULL for these.
- **Existing members must consent on their next login** after this migration runs, including whoever is currently the system's Admin — there is no bootstrap exemption. This is intentional (see 29.3) but worth expecting rather than being surprised by.
- **No re-consent on agreement changes.** If the Membership Agreement's wording changes materially later (not just typo fixes), there is currently no mechanism to prompt already-consented members to review the new version — member_consents.agreement_version records what they agreed to, but nothing

acts on a mismatch with the current membership_agreement.version. A future version could add this if it becomes a real governance need.

- **Company stamp/seal** was explicitly scoped out of this version — only personal, per-role signatures exist; there is no separate official company seal image applied automatically. *(Resolved in v1.24.0 — see Section 4.30.)*

---

30. Company Stamps & Seals (v1.24.0)

30.1 Purpose

Personal signatures (Section 4.29) prove *who* signed something. This module adds the *company's own mark* — a Treasury stamp, a Secretariat seal, or any other department's stamp an Admin uploads — automatically attached to a document once it is fully approved/signed, the same way a physical rubber stamp gets pressed onto a finished paper document. It's a separate concept from a person's signature and is applied by the system itself, not by any individual member.

30.2 Data model

- **company_stamps** — one row per uploaded stamp image: name (e.g. "Treasury", "Secretariat"), file_path (/uploads/stamps/…, same URL-path convention as the company logo), mime_type (restricted to image/png / image/svg+xml — the two transparent-background-friendly formats, so a stamp overlays cleanly without a white box around it), is_active (soft-deactivate only, same convention as roles.is_active — a deactivated stamp is never hard-deleted because already-stamped documents still reference it via document_stamps_applied).
- **document_stamp_requirements** — Admin-configured: document_type + stamp_id + is_active, mirroring signature_requirements's exact shape. A document type with zero active rows is never stamped — fully opt-in, same rule as multi-signatory approval. Covers the *full* documents.document_type list plus SHARE_CERTIFICATE, not just the four signable types, since stamping is described more broadly ("the important documents as specified by the company") than signing is.
- **A database-enforced business rule**: a **partial unique index** (idx_one_active_stamp_per_share_cert, on document_stamp_requirements(document_type) WHERE document_type = 'SHARE_CERTIFICATE' AND is_active = TRUE) makes it structurally impossible for SHARE_CERTIFICATE to have more than one active stamp assigned at a time — this is how "the monthly share certificate only gets a treasury stamp" is enforced. Deliberately **not** hardcoded by stamp name (no WHERE name = 'Treasury' anywhere) — whichever single stamp an Admin assigns to that slot is the one that applies, so renaming or replacing the Treasury stamp later doesn't require a code change. Every other document type may carry several active stamps at once (e.g. a Resolution could get both a Secretariat and a Director's seal).
- **document_stamps_applied** — a snapshot of which stamp(s) actually got baked onto a specific document/round the moment it became fully approved/signed, using

the identical target_type/target_id polymorphic shape as document_signatures ('DOCUMENT' → a documents.id, 'CERTIFICATE_ROUND' → a certificate_signing_rounds.id). This is what makes the feature safe against later config changes — if an Admin swaps which stamp is assigned to Resolutions next month, every Resolution stamped before that change keeps showing the stamp it actually got, not the new one.

- **company_settings.stamps_enabled** (v1.24.1) — a single master on/off switch for the whole feature, sitting alongside the company's other branding toggles (logo, colors) in the same singleton row. Defaults FALSE. This is separate from, and layered on top of, the per-document-type opt-in above — an Admin can freely upload stamps and assign document_stamp_requirements while this is off, none of it takes effect on a real document until the switch itself is on.

30.3 Business rules / key logic

A company-wide switch gates everything else (v1.24.1). stampService.areStampsEnabled() reads company_settings.stamps_enabled and applyStamps() checks it first, before even looking at document_stamp_requirements — if the switch is off, applyStamps() returns an empty array immediately and no document_stamps_applied row is ever written, regardless of what's configured for that document type. This means the feature can be fully set up in advance (stamps uploaded, document types assigned) and only switched on when the company is actually ready, and can be switched off again later without losing any of that configuration — turning it off does not remove stamps already applied to documents finalised while it was on, it only stops new ones from being stamped.

Stamping happens automatically, at the exact moment something becomes fully approved — never earlier, never by a manual action. There is no "apply stamp" button anywhere; a new stampService.applyStamps(targetType, targetId, documentType) is called from every place in the codebase that already flips something to fully-approved/fully-signed: - documentsController.approveDocument — the single-approver finalisation path (non-signable types, or signable types with nothing configured in Section 4.29's signature_requirements). - documentsController.signDocument — the moment the *last* required signature lands on a multi-signatory document. - certificatesController.signRound — the moment the last required signature lands on a certificate signing round. - certificateService.issueCertificatesForAllShareholders — the immediate-email path, for when nothing is configured in signature_requirements for SHARE_CERTIFICATE (the round is effectively "approved" the instant it's created, since nobody needs to sign it).

applyStamps itself is idempotent (an ON CONFLICT … DO NOTHING insert into document_stamps_applied) and best-effort, exactly like notification-sending elsewhere in this codebase — a stamp-application failure is caught and logged but never blocks or rolls back the actual approval/signing action it's attached to.

Rendering — the stamp image is overlaid near the signature area: - **Certificates (server-rendered PDF, certificateService.renderCertificateHtml)** — the function already

takes a signatures parameter (Section 4.29); it now also takes stamps, rendered as an absolutely-positioned <img class="stamp"> inside a .stamp-wrap div that also contains the existing .signatures block, using the same resolveAbsoluteAssetUrl() helper puppeteer needs to load images from outside its own page context. - **Client-rendered documents (exportUtils.js)** — a shared stampOverlay(data) helper renders the same kind of overlay from data.stamps (an array the caller populates), wrapped by a new .stamp-overlay-wrap CSS class added to the shared base stylesheet (getBaseStyles()) so every template can opt in with one wrapping div. Wired into resolutionTemplate and shareCertificateTemplate — the two templates most directly named in the original request. DocumentsPage.jsx's openDocument() (the shared preview/download handler) fetches GET /documents/:id/stamps and merges the result into template_data **only when doc.fully_signed is true** — a draft document is never re-rendered with a stamp on it, even if one is configured for its type.

30.4 API endpoints

Method	Endpoint	Access	Purpose
POST	/settings/stamps	Admin	Upload a stamp (multipart, fields stamp + name)
GET	/settings/stamps	Admin	List every stamp, active and deactivated
PATCH	/settings/ stamps/:id/ deactivate	Admin	Soft-deactivate a stamp (also turns off any requirements using it)
GET / PUT	/settings/stamp-requirements[/:documentType]	Admin	Read / replace which stamp(s) apply to a document type
GET	/documents/:id/ stamps	Any authenticated user	Whichever stamp(s) were actually applied to this document
GET	/certificates/ rounds/:id	Treasurer / Assistant Treasurer / Admin	Now also returns stamps alongside signatures
GET / PATCH	/settings/company	Any authenticated (GET) / Admin (PATCH)	Now also reads/writes stamps_enabled (v1.24.1) alongside the existing branding fields — no new route,

			reuses the existing company-settings endpoint

30.5 Frontend

- **Settings → Stamps tab** (SettingsPage.jsx) — a status banner at the top shows whether stamps are currently ON or OFF company-wide, with a Turn On/Turn Off button (v1.24.1, settingsAPI.updateCompany({ stamps_enabled })); below it, upload a named stamp (name + PNG/SVG file), a gallery of uploaded stamps (active and deactivated, with a Deactivate action), and per-document-type assignment — checkboxes for most types, radio buttons (single-select, with a Clear option) for Share Certificates specifically, matching the database-enforced one-stamp rule in 30.2. All of this configuration UI stays usable while the switch is off, by design.
- **Documents page → Signatures modal** — now also shows whichever stamp was applied, alongside the existing signature list, once the document is fully signed.
- **Reports page → Certificate Signing Rounds panel** — the expanded round view now shows the applied stamp image next to the signature list.

30.6 Known issues / open items

- **Only resolutionTemplate and shareCertificateTemplate render a client-side stamp overlay today.** The stampOverlay() helper and .stamp-overlay-wrap CSS class exist in exportUtils.js for any template to use, but the other document templates (meeting minutes, receipts, requisitions, etc.) haven't been wired up to pass data.stamps through yet — a future session can extend this to any additional stampable type as needed, following the exact same pattern as the two templates already done.
- **Uploaded (not system-generated) documents — e.g. a Loan/Grant Agreement uploaded as a PDF rather than generated from a template — have no client-side rendering path to overlay a stamp onto at all**, since documentsController.js's UPLOADED source type just serves the original file as-is. document_stamps_applied still records that a stamp was "applied" to these (visible via GET /documents/:id/stamps), but the underlying file itself is never visually altered. Same limitation the multi-signature feature already has for uploaded documents (Section 4.29 doesn't visually stamp these either — signatures there are also metadata-only for the UPLOADED source type).
- **Ad-hoc, single-certificate issuance** (POST /certificates, the self-service "download my own certificate now" action) is not part of the signing-round gate (same known issue as Section 4.29.7) and so never has a certificate_signing_rounds.id to look stamps up against — shareCertificateTemplate's stamp overlay will simply show nothing for these, consistent with existing behaviour.

5. Deployment Guide

This section is the Bible's own copy of "how to put this system online," written to stand alone — but the repo root also keeps DEPLOYMENT_GUIDE.md as a shorter, copy-paste-friendly companion for when you just need the exact commands without the surrounding explanation. The two are kept in sync; if they ever disagree, trust whichever was edited most recently (check the file's own timestamp) and update the other to match.

5.1 What's being deployed, and where

Render hosts three things per company: the Node/Express backend (a real long-running server process — needed because this app has scheduled cron jobs, not just request/response), the React frontend (built once into static files), and a managed PostgreSQL database. Render was chosen specifically because it supports long-lived server processes with scheduled background jobs, which serverless platforms (Vercel and similar) are not designed for.

Cost: backend starter plan (~$7/month — deliberately not the Free tier, which spins down after 15 minutes idle and would silently kill the nightly/monthly cron jobs), database basic-256mb plan ($6/month for the instance — deliberately not Free, since Free-tier databases expire after 90 days and this app holds real financial records), frontend static site (free). Render's Blueprint deploy screen also attaches a 15 GB disk to the database by default, billed separately at $0.30/GB/month (~$4.50/month) — so the database actually runs ~$10.50/month total, not just the $6/month instance price. **Total roughly $17.50/month per company deployed** (confirmed directly against Render's Blueprint cost estimate, not just the base per-plan prices — those alone undercount the real bill by the disk cost).

5.2 Before you start

You'll need: a free GitHub account, a free Render account with a card on file (for the paid Starter/Basic plans), and Git installed locally. psql (PostgreSQL's command-line client) is also needed for the one-time schema load in Step 5.4 below — installing just the client tools (not a full local Postgres server) is enough.

5.3 Step 1 — Push the code to GitHub

Render deploys from a Git repository, not from files on your computer directly. 1. Create a new **private** GitHub repository (e.g. cms) — leave it empty, no README/.gitignore/license. 2. From a terminal in the project root (the folder containing both cms/ and cms-frontend/): git init git add . git commit -m "Initial commit" git branch -M main git remote add origin https://github.com/YOUR_USERNAME/cms.git git push -u origin main 3. A .gitignore is already in place at the root and inside both cms/ and cms-frontend/, so node_modules/ and your real local .env (with live secrets) are automatically excluded — nothing extra to do here, just don't delete those .gitignore files.

5.4 Step 2 — Connect Render via Blueprint

1. In the Render dashboard, click **New +** → **Blueprint**.
2. Connect your GitHub account, select the repo you just pushed.
3. Render detects render.yaml at the repo root and previews three resources: cms-backend, cms-frontend, cms-db (Section 4.26.6 explains exactly what each resource's configuration means).
4. Click **Apply**. The database provisions first, then both services build — the first build typically takes 5-10 minutes since it's installing every dependency from scratch.

5.5 Step 3 — Fill in the secrets Render couldn't guess

render.yaml auto-generates safe values (JWT secrets via generateValue: true) but leaves company-specific/genuinely-secret values blank on purpose (sync: false — Section 4.26.6). Set these in the Render dashboard after the first deploy: - **cms-backend** → **Environment**: GMAIL_USER (the sending Gmail address), GMAIL_APP_PASSWORD (a Gmail App Password, not the normal account password), COMPANY_EMAIL, COMPANY_ADDRESS. - **cms-frontend** → **Environment**: REACT_APP_COMPANY_ADDRESS.

Saving each triggers an automatic redeploy with the new values baked in.

5.6 Step 4 — Load the database schema (one-time)

The database exists but starts empty. 1. Render dashboard → **cms-db** → **Connect** → copy the External Connection String. 2. From your local terminal: psql "PASTE_THE_CONNECTION_STRING_HERE" -f cms/schema.sql 3. Expect a long stream of CREATE TABLE/CREATE INDEX/INSERT messages with no errors — this single script builds every table, index, and structural seed row (the 9 system roles, finance/document/event categories, currencies, the company's own name/address) in one shot.

If you're upgrading an existing older-version database instead of starting fresh, run the matching migration_vX.X.0.sql file(s) from cms/ in order instead — each is written to be safe to re-run even if you're unsure whether it already applied.

5.7 Step 5 — Verify it's actually live

1. Visit https://<your-backend-url>.onrender.com/health — expect a small JSON response with "status": "OK". An error here almost always means either an environment-variable typo or Step 5.6 wasn't completed.
2. Visit the frontend URL — you should land on the login page with the company logo showing.
3. A fresh database has no seeded Admin account. Register your own account through the app, then follow the Going Live Guide's Step 4 (Section 6.5 below) to promote yourself to Admin via a one-time direct SQL command.

5.8 Running a second, separate company

render.company-b.yaml (repo root) is a ready-to-use second Blueprint on the exact same codebase, with different service names (cms-b-backend/cms-b-frontend/cms-b-db) so it deploys as a completely independent set of resources rather than colliding with Company A's (Section 4.26.6 has the full side-by-side comparison table). Before pushing, replace the placeholder COMPANY_B_NAME_HERE/CB values with the real second company's details — as flagged in Section 4.26.8, the backend section of that file currently still has unfilled placeholders even though the frontend section was already updated to INVESTABO GLOBAL INVESTMENTS LIMITED/IGI, so double check both halves match before deploying. In the Render dashboard, use **New +** → **Blueprint** → same GitHub repo → set the **Blueprint Path** field to render.company-b.yaml. Then repeat Steps 5.5-5.7 exactly, pointed at the new services and the new database.

5.9 Ongoing maintenance

- **Code changes**: push to main — every service watching that branch (both companies', if a second is set up) redeploys automatically.
- **Schema changes**: a new migration_vX.X.0.sql gets added to cms/. The code only needs writing once; the migration needs running once **per live database** (cms-db, and cms-b-db too if applicable) — this is the one place having two companies genuinely doubles the manual work. Once the system holds real data, follow the full safe-update checklist in Section 5.10 rather than just running the file straight against the live database.
- **Custom domain**: Render's **Settings** → **Custom Domains** on the frontend service. If you switch domains, revisit whether the backend's CORS check (which currently trusts whatever cms-frontend's Render-assigned URL resolves to) needs updating to trust the custom domain too.
- **Uploaded files**: not persistent across redeploys on the current setup (Section 4.26.5/4.26.8) — a Render Persistent Disk add-on or a move to external object storage (e.g. S3) is the fix once this matters operationally.

5.10 Updating the system safely once it holds real data

Everything above assumes an empty or test database. Once real members, contributions, loans, and dividends are sitting in the live database, "update the system" changes meaning: it's no longer "reload schema.sql," it's "change a live database out from under real records without corrupting or losing any of them." This section is the standing policy for every update from that point forward — it is not a one-time step, it applies to every future change, forever.

The one rule everything else follows: additive, not destructive. A safe change adds something (a new column, a new table, a widened constraint). An unsafe change removes or narrows something a real row already depends on (dropping a column, deleting rows, shrinking a constraint, renaming a column outright). When a change looks unsafe on its face, it gets split into safe steps spread across two or more releases (see the rename example below) rather than done in one shot.

Step 1 — every schema change is a migration file, never a direct edit. A new file cms/migration_vX.X.0.sql is written describing only the *change* — not the whole database. It is the only thing ever run against the live database; schema.sql is updated too, in the same session, but purely as documentation of what a brand-new install should look like — it is never re-run against a database that already has data, because CREATE TABLE and its seed INSERTs would either fail outright (table already exists) or duplicate seed rows.

Step 2 — every migration is written to be idempotent (safe to run more than once). This project already follows this pattern (see migration_v1.22.0.sql from this session) and it continues for every future one: - New columns: ALTER TABLE x ADD COLUMN IF NOT EXISTS y … — running it twice does nothing the second time, instead of erroring. - New/changed constraints: a DO $$ … $$ block that first checks pg_constraint for whether the constraint already has the right definition before touching it, rather than a blind DROP CONSTRAINT/ADD CONSTRAINT that would error on a second run. - New tables: CREATE TABLE IF NOT EXISTS. This matters because it means a migration can always be re-run if there's ever doubt about whether it already applied, without any risk of double-applying it.

Step 3 — a column is never renamed or removed directly. If a field genuinely needs to be renamed or dropped, it happens over two separate releases, never one: 1. *Release A*: add the new column alongside the old one, backfill it from the old column's existing values, update the application code to read/write the new column while the old one is simply left in place (still populated, just unused). 2. *Release B* (a later session, only after confirming nothing still depends on the old column — grep the codebase for it): drop the old column. Skipping straight to a rename or drop in one step is the single most common way a migration destroys real data, because anything still reading the old name breaks the moment it disappears, with no way back except a restore.

Step 4 — back up before running any migration against the live database. One command, no exceptions, every time:

```
pg_dump "CONNECTION_STRING" > backup-before-vX.X.0-$(date +
%Y%m%d).sql
```

This is the actual safety net — even a well-written migration can hit something unexpected on real data (a NOT NULL column with no sensible default for existing rows, for instance). A backup taken thirty seconds beforehand makes any mistake fully reversible: psql "CONNECTION_STRING" -f backup-file.sql against a fresh database restores everything exactly as it was.

Step 5 — test the migration against a copy first, not against the real database. Before running a new migration_vX.X.0.sql against the live cms-db, run it once against a throwaway copy loaded from that same backup (a local Postgres instance, or a temporary Render database spun up just for this and deleted afterward). This surfaces exactly the kind of failure Step 4 protects against, but *before* it can affect real data rather than after.

Step 6 — verify after applying, don't just trust a clean exit. A migration finishing without an error message is necessary but not sufficient. Spot-check with a targeted query afterward — e.g. after the dividends migration, SELECT COUNT(*) FROM dividend_distributions WHERE credited_amount IS NOT NULL AND status != 'PAID'; should return 0, since only paid distributions should have a credited amount. Every future migration should have its own version of this kind of sanity check run once, by hand, right after applying it.

Step 7 (once genuinely live) — turn on Point-in-Time Recovery. Render's paid Postgres plans support PITR (a rolling window that lets the database be restored to any specific moment, not just the last manual backup) — see render.com/docs/disks and the Render Postgres docs for current retention windows and pricing. This is worth enabling once the system is handling real club funds day-to-day, as a second safety net on top of the manual pg_dump habit above, not a replacement for it.

Putting it together, the standing checklist for every future update once live: 1. Write migration_vX.X.0.sql — additive only, IF NOT EXISTS/idempotent, no direct rename/drop. 2. Mirror the same change into schema.sql (for future fresh installs) — never the other way around. 3. pg_dump the live database. 4. Run the migration against a throwaway copy first; fix anything that breaks. 5. Run it for real against cms-db (and cms-b-db too, if Company B is live). 6. Run a targeted verification query confirming existing rows still look correct. 7. Update this Bible's relevant module section + Section 8's Version History (Section 9's existing rule) in the same session.

---

6. Going Live Guide

Companion to Section 5 — this is specifically about the moment you switch from "testing this system" to "real members are using it with real money," including safely clearing out whatever test data accumulated during development. The repo root also keeps the fuller GOING_LIVE_GUIDE.md as a copy-paste-friendly companion.

6.1 The short version

schema.sql never inserts demo/fake data — only *structural* rows (the 9 roles, finance/document/event categories, currencies, the real company name/address). Anything you'd call "test data" — test users, made-up transactions, fake loans — is stuff created by hand while building and testing the app, sitting in the live database. This section's job is removing that safely without breaking the structure underneath it.

6.2 Step 1 — Which situation are you in?

- **Situation A** — the Render database has never really been touched (never ran schema.sql against it for real, or only poked around briefly). Confirm with psql "CONNECTION_STRING" -c "SELECT COUNT(*) FROM users;" — a 0 result (or a table-doesn't-exist error) means you're genuinely clean; skip straight to Section 6.5 (bootstrap the first Admin).

- **Situation B** — the live database has real test activity (recorded contributions, loans, etc.) sitting in it. This is the common case if the app was demoed on the live URL. Proceed through 6.3.

6.3 Step 2B — Wipe and rebuild (the recommended path)

Two approaches exist: surgically deleting rows table-by-table, or wiping the whole database and reloading schema.sql fresh. **Wiping and reloading is recommended** — with roughly 50 interconnected tables and deep foreign-key chains (a transaction references a category, an account, and a user; a loan references an account; and so on), deleting rows by hand in the correct dependency order is genuinely easy to get wrong, and a partial failure can leave the database in a worse, half-deleted state than before.

What gets erased: every user, account, transaction, loan, investment, grant, dividend, savings record, side fund entry, requisition, event, document, notification, and internal audit log entry — everything anyone has ever saved through the app.

What survives, rebuilt automatically and unchanged: the 9 system roles, every finance/document/event category, currencies, the real company name/address (already the actual seed value in schema.sql, not a placeholder), and the structural shape of every table.

What needs redoing afterward (lives in the database, not in schema.sql): any custom role_permissions grants made via Settings → Roles & Permissions (Section 4.24 — remember role_permissions ships empty even on a fresh install), the uploaded company logo, Side Fund activation, and the Savings interest rate if changed from default.

Procedure: 1. **Back up first** — pg_dump "CONNECTION_STRING" > backup-before-reset-$(date +%Y%m%d).sql. Costs two minutes, restorable later with psql "CONNECTION_STRING" -f backup-file.sql against a fresh database. 2. **Drop and reload** — psql "CONNECTION_STRING" -c "DROP SCHEMA public CASCADE; CREATE SCHEMA public;" psql "CONNECTION_STRING" -f cms/schema.sql 3. **Clear the uploads folder** — required for a genuinely blank slate, since uploaded files live on disk, not in the database; wiping the DB just orphans them, it doesn't delete them. Clear cms/uploads/documents/*, cms/uploads/profiles/*, cms/uploads/branding/* (locally) or the equivalent path via the Render **cms-backend** → **Shell** tab for the live disk (though as noted in Section 4.26.5/5.9, the Starter plan's disk isn't persistent across deploys anyway, so this may already be empty — still worth running once to be sure).

A **targeted delete** (keeping most data, removing only specific test records) is the alternative if you've already spent real time customizing permissions/branding and don't want to redo that — but because the safe deletion order depends entirely on which specific records exist, this isn't a generic copy-paste script; it needs to be worked out case by case against the actual data in front of you.

6.4 Step 3 — Reconciling company identity on a true fresh start

If this deployment is for a genuinely different company (not just your own data after testing), schema.sql's seed row hardcodes company_name = 'ZWECK TUKULA Ltd' /

company_address = 'WAKISO, UGANDA' — a fresh reload shows that until someone changes it. No source file needs editing for this: log in as the new Admin and update it via **Settings → Company** (Section 4.19), the same as the logo.

6.5 Step 4 — Bootstrap the first real Admin user

There is no default/seeded Admin login anywhere in this system — the very first person always has to register normally through the app and then be manually promoted, because granting a role normally requires already being logged in as someone who can grant roles (a chicken-and-egg problem every fresh install hits exactly once). 1. Register a real account through the live app with your own name/email. 2. Verify the email (works correctly even before first login). 3. Promote yourself directly against the database: psql "CONNECTION_STRING" -c " INSERT INTO user_roles (user_id, role_id, assigned_by) SELECT u.id, r.id, u.id FROM users u, roles r WHERE u.email = 'YOUR_EMAIL_HERE' AND r.name = 'Admin'; " (Using yourself as the assigned_by value is a deliberate one-time exception to the normal flow where an existing Admin assigns roles to others.) 4. Log out and back in (or refresh) to pick up the new role — every menu item, including Users and Settings, should now appear. 5. From here on, use **Users → Assign Role** in the app itself for every other person; the manual SQL step is only ever needed once, for the very first Admin, per database.

6.6 Step 5 — Reconfigure what a fresh database doesn't remember

After a full wipe (Section 6.3), walk through once, logged in as the new Admin: - **Settings → Company**: confirm name/address (survive automatically), re-upload the logo if one was set before. - **Settings → Roles & Permissions**: grant FINANCE_FLOOR_LIMIT_UPDATE to Treasurer and Assistant Treasurer (this permission has **no default grant** — Section 4.24.3's "zero seed permissions" fact applies here directly) and review any other permission customizations. - **Accounts** (Section 4.1): create the real Primary account and, if used, the Savings account. - **Side Fund** (Section 4.10): re-activate if used, set the real allocation. - **Savings** (Section 4.11): re-set the interest rate/period if changed from default. - **Users → Assign Role**: bring the real Directors/Treasurer/Secretary etc. on board — each person registers, then gets their role assigned.

6.7 Step 6 — Final go-live checklist

- ☐ Fresh schema.sql run (or confirmed genuinely clean database)
- ☐ Uploads folders cleared (Section 6.3, step 3)
- ☐ Admin bootstrapped and login confirmed working
- ☐ Company name/address/logo correct in Settings
- ☐ FINANCE_FLOOR_LIMIT_UPDATE granted to the right roles
- ☐ Real Primary account created (and Savings account, if used)
- ☐ Real Directors/Treasurer/Secretary users created and roles assigned
- ☐ GMAIL_USER/GMAIL_APP_PASSWORD set so verification/notification emails actually send
- ☐ A test contribution/expense sent and confirmed correct on the Dashboard and in Transactions

- ☐ Email verification and password-reset links tested end-to-end

Once every box is checked, the system is live for real use.

---

7. Known Issues & Technical Debt Registry

This section pulls together every open item mentioned throughout Section 4's per-module "Known issues" notes, plus the standing findings of SYSTEM_DIAGNOSTIC_REPORT.md (dated 19 July 2026, still kept at the repo root as the detailed original write-up), into one place — so nobody has to read all 27 module sections to get a picture of what's actually outstanding. Items are grouped by status, not by module.

7.1 Fixed this engagement (for the historical record)

- **Auditor visibility leak** (Section 3) — TopBar amounts/notifications, GET /api/users/shareholding, and 16 other route files were open to any authenticated user, unintentionally including the new Auditor role. Fixed via the blockAuditor middleware + frontend isAuditor gating in TopBar.jsx. Version 1.20.1.
- All 6 Critical items from SYSTEM_DIAGNOSTIC_REPORT.md: the savings-handout category_id: null crash, the floor-limit permission mismatch (route vs. frontend), reference-code mislabeling across 6+2 call sites, missing password-reset UI, missing transaction-reversal UI, missing loan-rate-amendment UI.

7.2 Open — architectural / worth a deliberate decision

- **requireRoles vs. requirePermissions inconsistency** (Section 4.24.3, SYSTEM_DIAGNOSTIC_REPORT.md Moderate #7) — two enforcement mechanisms coexist per-route with no guaranteed consistency; the Roles & Permissions settings screen only controls the subset of routes actually written against the permission system. Resolving this fully means auditing every route file module by module — a real but non-trivial refactor.
- **Local-disk, non-persistent file uploads** (Sections 4.16.6, 4.26.5, 4.26.8, 5.9) — Documents, avatars, branding logos, and audit-report uploads all live on the backend's local disk, which doesn't survive a Render redeploy on the current plan. A Persistent Disk add-on or a move to external object storage (S3-compatible) resolves this.
- **Single-instance-only scheduled jobs** (Section 4.26.2, and the earlier scaling-advice conversation) — every cron job runs in-process on the one backend web service; scaling the backend to multiple instances would fire every job once per instance without an added locking mechanism.
- **No automated test suite** (Section 4.26.8, scaling-advice conversation) — every fix and feature to date, including everything built this session, is verified via manual node --check/syntax validation and logical review rather than regression tests. Reasonable at the current solo-owner-plus-AI-developer scale; worth revisiting if more people ever touch the code.

- **investment_milestones** (Section 4.8.6, SYSTEM_DIAGNOSTIC_REPORT.md Moderate #12) — a fully built backend feature with zero frontend surface. Needs a decision: build the UI, or remove the unused backend routes.
- **401-refresh interceptor bypass** (Section 4.22.6, SYSTEM_DIAGNOSTIC_REPORT.md Moderate #14) — some request paths don't consistently trigger the frontend's automatic token-refresh-and-retry on a 401, which can surface as an unexpected logout on those paths. Still open.
- **Company B Blueprint placeholders** (Section 4.26.8) — render.company-b.yaml's backend section still has unfilled COMPANY_B_NAME_HERE/CB placeholders in TOTP_APP_NAME and COMPANY_NAME, even though the frontend section was already updated to the real name (INVESTABO GLOBAL INVESTMENTS LIMITED / IGI). Needs reconciling before Company B's next deploy.

7.3 Open — moderate, low-risk, worth fixing when convenient

- **EUR hard-coded in floor-limit audit description** (Section 4.1.6, accountsController.js:594) — floor limits now apply to any account type, but the audit-log description text still always says "EUR" regardless of the account's real currency.
- **GET /exchange-rates/history** has no confirmed frontend consumer (Section 4.4.6).
- **updateCategory's missing COALESCE on description/is_active** (Section 4.5.6) — an update call omitting these could unintentionally clear the description; hasn't surfaced as a live bug because the frontend always sends a default, but is worth tightening.
- **Manual module-code resolution in Requisitions** (Section 4.9.6) instead of reusing the shared resolveModuleCode() helper — cosmetic inconsistency, not a functional bug.
- **Dead contributedBy parameter** in the postTransaction call inside creditShareholderContribution (Section 4.2.6) — silently dropped since postTransaction doesn't destructure it; the real contributed_by write happens via a separate UPDATE immediately after. Harmless but misleading to read.
- **Transfer Approve/Reject buttons shown without a role-specific client-side check** (Section 4.3.6) — any holder of FINANCE_TRANSFER_APPROVE sees the button even if the backend will ultimately reject their specific approval (P2S needs Treasurer, S2P needs Director) — a UX rough edge, not a security gap, since the backend enforces correctly either way.
- **Stale avatar display spot** (Section 4.23.6, SYSTEM_DIAGNOSTIC_REPORT.md Minor) — one UI location doesn't immediately reflect a freshly uploaded avatar without a manual refresh.
- **Overdue grant-tranche detection is query-driven, not proactively notified** (Section 4.6.6) — unlike loans, which have an automated overdue-check cron job.

7.4 Open — worth monitoring as the system scales

(Cross-referencing the dedicated scaling-advice conversation that preceded this Bible, and Section 4.26.8's infrastructure notes.) - Unbounded growth of transactions and audit_log with no archival/pruning strategy yet (Section 4.25.6 note on the internal audit log applies equally to the transactions ledger). - Full shareholding-registry recompute on every single contribution (Section 4.14.6) — O(number of shareholders) work per contribution; fine at current membership scale, worth watching if membership grows substantially. - Puppeteer's memory/cold-start footprint on the starter Render plan (Section 4.13.6) — the one heavyweight server-side rendering dependency in an otherwise light Node app; the most likely reason the backend would ever need a larger plan. - Regulatory/compliance readiness and a more formal database-growth/backup strategy, both raised in the earlier scaling conversation, remain open strategic questions rather than concrete bugs.

7.5 How to use this registry going forward

When a future session fixes one of these, move it from its current subsection into 7.1 with the version number it was fixed in (matching the pattern used for this session's Auditor-visibility fix), and update the relevant module's own "Known issues" note in Section 4 to say "confirmed fixed" rather than deleting the historical context — the same treatment already given to the diagnostic report's six Critical items throughout Section 4.

---

8. Version History

Derived from the cms/migration_vX.X.0.sql files' own header comments — each migration is the authoritative record of what changed in that version, and is still sitting in the repo for anyone who needs the exact SQL. This table is the readable summary; version 1.20.1 (this session) was code-only, so it has no matching migration file.

Version	What changed
1.2.0	Bank-charge fields on transfers, contributed_by on transactions, five new tables (dividends, dividend_distributions, authority_payments, member_savings, requisitions), category_paths backfill.
1.3.0	Bond-specific fields on investments (type, face value, coupon rate/frequency, tax withholding) and a bond_coupons payment-schedule table.
1.4.0	Assistant Treasurer role added; requisitions extended (requisition_type, contribution_date) to support Treasurer-acknowledged capital contributions.
1.5.0	investment_transactions table

	(EXPENSE/INFLOW/TAX ledger entries) and company_settings for editable branding.
1.6.0	In-app bell notifications, share_price_history, and per-account reference_prefix.
1.7.0	About-page fields on company_settings (description/mission/vision/core values) and currency_exchange_rates (display-only conversion, Section 4.4).
1.8.0	share_certificates table (Monthly/Annual certificate records, auto-generated reference numbers).
1.9.0	investments.first_coupon_date (bonds bought mid-schedule), references_registry.public_id (unguessable external IDs, Section 4.26.4); bond coupon tax and category-filter fixes.
1.9.1	company_settings.motto; About page/TopBar now read live Settings → Company values instead of hardcoded placeholders.
1.10.0	Reworked member savings: kept legacy fixed-term deposits, added FLEXIBLE savings with balances/interest accrual and a two-actor deposit/handout approval flow.
1.11.0	Original “side fund” petty-cash-style pool feature — dual-posted contributions/expenses, auto-generated monthly member dues.
1.12.0	Bank detail fields on accounts (is_virtual, bank_name, etc., Section 4.1.2); enforced 1:1 exchange rate for same-currency transfers.
1.13.0	Permissions management UI/routes (previously DB-only, Section 4.24); corrected the SAVINGS_APPROVE permission description.
1.14.0	Dedicated SAVINGS account type introduced (Section 4.1); floor limits generalized to any account type; direct/batch Side Fund inflows;

	savings_pool_inflows.
1.15.0	documents.template_data (JSONB) — the mechanism enabling preview/download/regeneration of system-generated documents (Section 4.16/4.21).
1.16.0	users.gender/users.avatar_choice — uploaded photos or illustrated avatars now actually display instead of falling back to initials.
1.17.0	Fixed the savings-handout confirmation crash by adding the missing savings_handouts.category_id column (this is the exact fix later re-confirmed in SYSTEM_DIAGNOSTIC_REPORT.md and Section 4.11.3/4.11.6).
1.18.0	RECEIPT and RESOLUTION document types enabled end-to-end (widened CHECK constraints, seeded document_templates rows, Section 4.21).
1.19.0	**External Audit Portal, part 1** — new Auditor role plus audit_engagements/audit_engagement_accounts/audit_engagement_users/ audit_engagement_documents (scoped, revocable read-only access, Section 4.27).
1.20.0	**External Audit Portal, part 2** — auditor profile fields, engagement comment log, staged report uploads, dual-approval "Finish Audit" submission workflow with auto-archiving, extension-of-access requests, access-expiry reminder job (Section 4.27 in full).
1.20.1	Fixed the Auditor-visibility leak — blockAuditor middleware added to 18 route files plus GET /users/shareholding, and TopBar.jsx/ProtectedRoute.jsx updated to stop showing company-wide amounts/notifications/search to the Auditor role (Section 3). Code-only change, no schema migration. This CMS Bible itself was also written and first published in this version.

1.21.0	**Administrative Officer role (Section 4.28).** New role for hired/contracted staff — full Events access, Documents access minus Financial-category items (with a per-document grant exception via staff_document_grants), and a full "service fee" compensation module (service_fee_agreements/service_fee_payments/service_reimbursement_requests) deliberately not named "payroll" for legal-classification reasons. Generalized blockAuditor into blockRoles()/blockFinanceRestricted (Section 3) so future restricted roles reuse the same deny-list pattern.
1.21.1	**Zero-role account gate (Section 3).** New requireAssignedRole middleware closes the gap between "email verified" and "Admin approved" — a freshly verified account with no role assigned no longer reaches any of the "open to any authenticated user" endpoints (which previously included real company account balances). Rolled out to 22 route files plus GET /users/shareholding. New self-service GET /users/me/role-request endpoint and standalone PendingApprovalPage.jsx (/pending-approval, no Sidebar/TopBar) that AppLayout.jsx redirects such accounts to, with a 30-second background poll so approval takes effect without a manual re-login. Code-only, no schema migration.
1.22.0	**Currency auto-derivation, service-fee amend/terminate, dividends-to-savings (Sections 4.12, 4.28).** Service Fees' Create Agreement form no longer asks for a currency — it is always derived server-side from the selected account, matching the pattern already used everywhere else in the system. Service fee agreements can now be amended (edit modal, same PATCH /agreements/:id) and terminated (end-date modal, sets status='ENDED') instead of being fixed once created.

	Dividend approval was rewritten: approving a dividend now automatically credits every shareholder's own Savings balance for their share, via a two-leg transaction (DEBIT the source account, CREDIT the single company Savings account), updating each dividend_distributions row and savings_balances.principal_balance. Because there is one shared Savings account and its currency may differ from the dividend's declared currency, the Treasurer manually enters the exchange rate at approval time when needed (needs_exchange_rate flag) — consistent with the system's existing rule that automatic exchange rates are display-only and never used to move real money. Migration migration_v1.22.0.sql: widens transactions.inflow_type, adds dividends.transaction_id/savings_transaction_id/exchange_rate, adds dividend_distributions.credited_amount/exchange_rate.
1.23.0	**Digital consent & multi-signatory approval (Section 4.29).** Every member now draws a personal signature and consents to a one-time Membership Agreement right after their role is assigned (requireConsent middleware, /consent page) — applies to existing members too, on their next login. Resolutions, Loan/Grant Agreements, and Share Certificates can now require several specific roles to each sign (Settings → Signatories) before counting as approved — opt-in per document type, falls back to the original single-approver flow if nothing is configured. Monthly/annual certificate batches are grouped into signing rounds (certificate_signing_rounds); one signature per required role covers every certificate in the round, and certificates aren't emailed until the round is fully signed, with a new reminder job during the

	last week of each month for anyone still pending. New tables: membership_agreement, member_consents, signature_requirements, document_signatures, certificate_signing_rounds; new columns users.signature_path/signature_updated_at, documents.fully_signed/fully_signed_at, share_certificates.signing_round_id. Migration migration_v1.23.0.sql.
1.23.1	**Document signature slots now email, not just notify (Section 4.29.5 addendum).** Closed a gap where Resolution/Loan/Grant Agreement signature slots had no notification mechanism at all (certificate rounds already emailed) — new notifyPendingSignatories helper sends a bell notification + email both the moment slots open and via a new daily reminder job, scheduleDocumentSignatureReminders (30 8 * * *). Code-only, no schema migration.
1.24.0	**Company stamps & seals (Section 4.30).** Admin uploads named stamp images (Treasury, Secretariat, or any department) in Settings → Stamps and assigns which document type(s) each auto-attaches to once fully approved/signed — opt-in per type, same shape as v1.23.0's signature requirements. Share Certificates are structurally capped at one active stamp (a partial unique index, not a hardcoded stamp name) to enforce "the monthly share certificate only gets a treasury stamp." New tables: company_stamps, document_stamp_requirements, document_stamps_applied. Stamp application is hooked into every place a document/certificate round becomes fully approved (approveDocument, signDocument, signRound, the no-

	signature-required certificate path), and rendered via certificateService.renderCertificateHtml (server-side PDFs) and a new stampOverlay() helper in exportUtils.js (client-rendered Resolutions and on-demand Share Certificates). Migration migration_v1.24.0.sql.
1.24.1	**Company-wide stamps on/off switch (Section 4.30.2/30.3).** New company_settings.stamps_enabled (default FALSE) lets an Admin turn the entire stamps feature on or off from Settings → Stamps without touching per-document-type configuration underneath it — stampService.applyStamps() checks this first and no-ops entirely while off, regardless of what's configured in document_stamp_requirements. Admins can still upload stamps and assign document types while the switch is off; nothing is applied to a real document/certificate until it's switched on. Reuses the existing PATCH /settings/company endpoint (no new route). Migration migration_v1.24.1.sql.
1.25.0	**Side Fund per-member overrides & overpayment credit, custom fiscal quarters, system-wide future-date validation (Sections 4.10, 4.19.6, 4.26.7).** Side Fund: an Admin/Treasurer can now set a different fixed monthly due for an individual shareholder (side_fund_member_overrides, opt-in, forward-only) instead of everyone paying the single company-wide default; a payment can now exceed what was owed — the extra clears the member's other outstanding dues oldest-first, then anything left over is banked as running credit (side_fund_member_credit, side_fund_credit_ledger) and automatically drawn down against their future months' dues as the monthly due-generation job creates them, with no new

	ledger transaction posted for that reallocation step. Fiscal Quarters: a new Admin-configurable fiscal_quarters table lets the company define its own financial-year quarters as fully custom date ranges (not required to be equal three-month blocks); Reports now attach a matched quarter's label alongside the calendar period, purely as a label. Future-date validation: a single reusable notFutureDate validator, added onto the existing date-field validation chains for every field across the system that represents money actually moving or an event that already happened (contribution/payment/expense/disbursement/deposit dates and similar) — deliberately not applied to legitimately forward-looking fields like due dates or term end dates — backed up on the frontend with max={today} on the matching date pickers. New tables: side_fund_member_overrides, side_fund_member_credit, side_fund_credit_ledger, fiscal_quarters; new column side_fund_dues.paid_from_credit. Migration migration_v1.25.0.sql.
1.25.1	**Fixed: every generated document's preview/download was broken (Section 4.21.3).** DocumentsPage.jsx and AuditorPortalPage.jsx were both reading a SYSTEM_GENERATED document's document_type/template_data/title off the top level of the parsed JSON response, instead of unwrapping the standard { success, message, data } envelope every backend response actually uses — so GENERATED_RENDERERS[payload.document_type] was always undefined and any Meeting Minutes, Meeting Agenda, Receipt, Resolution, or Auditor Feedback document failed to preview or download with "This document type can't be reconstructed," regardless of its actual

	type. Code-only, no schema migration.
1.26.0	**Side Fund: strictly per-member attribution (Sections 4.9, 4.2, 4.10).** Closed the gap where money could enter the fund without being tied to any member (the v1.25.0 “Add Funds Directly” feature) — every shilling in the pool is now always traceable to a specific member’s own dues, which is what makes accurate overdue tracking possible. Removed recordDirectInflow/POST /side-fund/inflows entirely, with no replacement — there is no way to seed an unattributed balance anymore. New shared services/sideFundService.js’s applySideFundPayment() is now the single oldest-unpaid-first payment-application choke point used by all four ways money can now reach a member’s dues: (1) the existing single-due payment (PATCH /dues/:id/pay, now applies across the member’s whole standing, not just the clicked row); (2) new bulk pay-all-dues (PATCH /dues/bulk-pay, one pooled ledger transaction, per-member amounts editable, for the common “everyone paid on time” case); (3) an optional side_fund_amount on a Transactions contribution (POST /transactions/contributions), sliced out of the total and credited to that same member’s dues while the remainder is recorded as the capital contribution; (4) a new SIDE_FUND_CONTRIBUTION requisition type, acknowledged the same way as a contribution or savings deposit. side_fund_dues.due_date (the last day of each due’s own period month) is now an explicit stored column rather than recomputed from period, backing two new overdue-summary endpoints (GET /overdue/me, GET /overdue). The plain Shareholder dashboard (ShareholderDashboard.jsx), which previously had no side fund visibility at all,

	now shows a “My Side Fund” card. Migration migration_v1.26.0.sql: adds side_fund_dues.due_date (NOT NULL, backfilled), widens requisitions.requisition_type to add SIDE_FUND_CONTRIBUTION.
1.26.1	**Fixed: a Savings handout could be paid from any account, not just Savings (Section 4.11.3).** createSavingsHandout trusted a client-supplied account_id, checking only that it belonged to *some* active account — a handout could be recorded against the Primary account or any Secondary/operational account, debiting money that was never actually the member’s savings. Now resolved server-side via getSavingsAccount(), the same helper every other savings entry point (deposits, fixed-term withdrawals) already used — always the one dedicated SAVINGS account, no exceptions. The “Pay From Account” picker was removed from the Record Handout modal to match; POST /savings/handouts no longer accepts account_id. Code-only, no schema migration.
1.26.2	**Fixed: approving a same-currency dividend crashed the backend (Section 4.12.3).** When no exchange rate was needed, DividendsPage.jsx called the approve endpoint with undefined as the request body — axios sends no body and no Content-Type header for undefined, so Express’s JSON parser never ran and req.body arrived undefined server-side instead of {}. const { exchange_rate } = req.body then threw an unhandled TypeError, surfaced to the Treasurer as a generic “Something went wrong.” Fixed on both ends: the frontend now always sends {} when there’s nothing to send, and approveDividend defaults with req.body || {} so any future bodyless caller degrades safely instead of crashing. Code-only, no schema migration.

1.27.0	**Frontend design system — dark mode, gradient banners, dual-scroll tables, mobile pass, back buttons (Section 2.4).** tailwind.config.js now sets darkMode: 'media' — the app follows the OS light/dark setting automatically, no in-app toggle. Dark mode is applied at three levels: dark: variants on every shared component class in index.css (.card, .btn-*, .badge-*, .input, table classes, etc.); a blanket @media (prefers-color-scheme: dark) override block re-targeting the raw Tailwind neutral/alert colour classes (bg-white, text-gray-*, border-gray-*, bg-{colour}-50) used directly across ~40 page files, so pages get dark mode without individual edits; and CSS custom properties (--cms-surface, --cms-text-primary, etc.) for TopBar.jsx's inline-styled neutral surfaces. PageHeader.jsx (23 of ~31 pages) now renders as a gradient banner (blue → indigo → teal, new brand-gradient token) instead of plain text, and gained showBack/backTo props for an in-banner back arrow — wired into GenerateDocumentPage.jsx. Dashboard StatCard icon chips switched from flat tints to small gradients. DataTable.jsx (14 pages) gained a second, synced horizontal scrollbar mirrored above the table so long tables don't require scrolling to the bottom to find one. Mobile: PageHeader actions now wrap on narrow screens, and two hard-coded grid-cols-2 form layouts became responsive. Code-only, no schema migration.

9. How to Keep This Document Alive

This Bible is only worth what it's worth if it stays accurate. The instruction that created it — "it should always be updated when changes that were changed are made or when something is added" — means updating this file is treated as part of finishing a piece of work, not a separate task to get to later. Practically, that means:

When a future session changes code, before considering the work finished: 1. Find the relevant module section in Section 4 (use the numbered list in the Table of Contents, or search this file for the module/table/route name). 2. Update whatever changed — a new endpoint goes in that module's endpoint table, a new business rule goes in that module's "Business rules / key logic" subsection, a schema change goes in "Data model," a bug fix moves the relevant line from Section 7 (Known Issues) into that module's "Known issues" note as "confirmed fixed," etc. 3. If the change is significant enough to warrant a version bump (per the existing convention of bumping REACT_APP_VERSION in both render.yaml and render.company-b.yaml — Section 4.26.6), add a row to Section 8's Version History table. 4. If the change touched deployment/environment configuration, update Section 5 (and the standalone DEPLOYMENT_GUIDE.md, kept in sync per Section 5.1's note). 5. Update the **"Version documented" / "Last updated"** line at the very top of this file.

When a new module is built from scratch, give it its own numbered subsection in Section 4, following the same structure every existing module uses: Purpose, Data model, Business rules/key logic, API endpoints, Frontend, Known issues — and add it to the Table of Contents list.

This file's location: company management system/docs/CMS_BIBLE.md, inside the same Git repository as the code itself — so it's version-controlled, travels with the codebase, and is included in the exact same git add/commit/push workflow as any code change (Section 5.3), rather than living somewhere separate that could drift out of sync unnoticed.

Companion documents: DEPLOYMENT_GUIDE.md, GOING_LIVE_GUIDE.md, and SYSTEM_DIAGNOSTIC_REPORT.md remain at the repo root as focused, standalone references — this Bible folds their content in for a single-document read-through, but doesn't replace them; keep all four in sync when any one changes.

A reminder from Ian's own standing instructions (repeated here since this document exists specifically to satisfy them): keep this Bible detailed enough that a basic beginner — Ian himself, or a future officer of the company with no technical background — could use it to understand what a feature does and why, not just a terse technical index. Favor explaining the *reasoning* behind a design choice, not only stating what the code does.

End of the CMS Bible. Built by Ian and Claude, for ZWECK TUKULA Ltd.